

FOUNDATIONS OF NATURAL LANGUAGE PROCESSING

自然语言处理基础

中文课程讲义

从概率模型与结构预测

到 Transformer 与大语言模型

概率建模 数学直觉 算法实践

笔记作者: VON

课程参考: 2026 春 · 自然语言处理基础 · Yansong Feng

2026 年 10 月

目录

记号与问题地图	vi
第一部分 概率、表示与语言结构	1
第 1 章 从语言问题到可学习的模型	2
1.1 任务、表示与假设空间	2
1.2 联合概率、条件概率与隐变量	3
1.3 学习目标与信息论解释	4
1.4 预测决策与可靠的实验	4
第 2 章 词义消歧与朴素贝叶斯	7
2.1 义项、上下文与任务定义	7
2.2 从贝叶斯公式到线性判别	8
2.3 估计、平滑与完整例子	9
2.4 从词典匹配到上下文表示	10
第 3 章 对数线性模型与学习的几何	12
3.1 从特征到概率	12
3.2 交叉熵梯度为什么是“预测减目标”	13
3.3 凸性、正则化与稳定计算	14
3.4 文本特征与模型边界	14
第 4 章 语言模型、稀疏性与平滑	17
4.1 自回归分解与边界符号	17
4.2 困惑度与不可直接比较的数字	18
4.3 从加性平滑到折扣与插值	18
4.4 Kneser–Ney: 什么词适合出现在新环境	19
第 5 章 分布语义、矩阵分解与主题	22
5.1 从上下文构造共现矩阵	22
5.2 PMI、余弦与低秩几何	23
5.3 潜在主题: 从几何到概率混合	24
5.4 词义、语境与评价的限制	25

第 6 章 通过预测学习词向量	28
6.1 查表、共享参数与上下文预测	28
6.2 softmax 的梯度与计算瓶颈	29
6.3 负采样：把多类预测改为二元识别	30
6.4 负采样为什么连接 PMI	30
第 7 章 神经语言模型与循环序列网络	33
7.1 前馈语言模型与参数共享	33
7.2 卷积与循环：两种组织上下文的方法	34
7.3 梯度消失、爆炸与门控记忆	35
7.4 训练、生成与序列标注	36
第 8 章 序列标注、HMM 与条件随机场	39
8.1 标签体系与隐马尔可夫假设	39
8.2 Viterbi：保留对未来有用的历史	40
8.3 前向后向：对所有解释求和	41
8.4 CRF：保留链结构，放宽观测假设	42
第 9 章 成分句法与 CKY 动态规划	45
9.1 成分、递归与上下文无关文法	45
9.2 概率文法与树的评分	46
9.3 CKY：按跨度长度填表	46
9.4 Inside、神经评分与解析评价	48
第 10 章 依存句法：从动作序列到最优树	50
10.1 有根依存树与投射性	50
10.2 Arc-standard 转移系统	51
10.3 图式解析与最大生成树	52
10.4 投射动态规划、神经评分与评价	53
第 11 章 组合语义与可执行的意义表示	55
11.1 真值条件与一阶逻辑	55
11.2 类型与函数应用如何组成句子	56
11.3 事件、关系图与推理	57
11.4 从自然语言到程序	58
第 12 章 信息检索：从词项匹配到稠密表示	61
12.1 索引、匹配与排序	61
12.2 TF-IDF 与 BM25	62

12.3 双编码器与对比学习	63
12.4 近似搜索与检索评价	64
第二部分 Transformer 与大语言模型	66
第 13 章 从注意力到 Transformer	67
13.1 序列到序列与软对齐	67
13.2 Q、K、V 与缩放点积注意力	68
13.3 多头、残差与前馈网络	69
13.4 位置表示：模型怎样知道词序	70
13.5 计算代价与实现检查	71
第 14 章 预训练：把文本变成监督信号	74
14.1 上下文化表示与迁移学习	74
14.2 BERT：从双侧语境恢复被选中的词	75
14.3 GPT 与 T5：两条生成式路线	76
14.4 训练目标与下游能力之间的距离	77
第 15 章 分词与训练数据：模型实际读到了什么	80
15.1 字符、词与子词	80
15.2 文本抽取与线性化	81
15.3 去重、质量筛选与数据混合	82
15.4 数据泄漏、合成数据与样本打包	83
第 16 章 规模、优化与训练成本	85
16.1 缩放规律与计算预算	85
16.2 梯度、批量与 AdamW	86
16.3 显存由哪些部分构成	87
16.4 稀疏专家与训练监控	87
第 17 章 提示与上下文学习	90
17.1 把任务写成条件生成	90
17.2 少样本示例与上下文学习	90
17.3 提示设计、校准与软提示	91
17.4 推理提示与实验方法	92
第 18 章 生成算法与高效推断	95
18.1 贪心搜索与束搜索	95
18.2 温度、Top-k 与 Top-p 采样	96

18.3 KV cache 与两阶段推断	97
18.4 量化、批处理与推测解码	98
第 19 章 监督微调与参数高效适配	100
19.1 指令数据与回答损失	100
19.2 数据质量、覆盖与训练强度	101
19.3 LoRA 的低秩更新	101
19.4 QLoRA、蒸馏与适配选择	103
第 20 章 偏好学习、RLHF 与 DPO	105
20.1 从比较数据到奖励模型	105
20.2 策略梯度与 KL 约束	106
20.3 PPO 与完整的 RLHF 流程	107
20.4 DPO: 把奖励关系写进策略概率	108
20.5 方法选择与偏好数据的边界	109
第 21 章 推理时计算、可验证奖励与工具使用	111
21.1 多条候选与答案聚合	111
21.2 程序辅助与树搜索	112
21.3 可验证奖励与组相对优化	113
21.4 工具与智能体的状态循环	114
第 22 章 检索增强生成与长上下文	116
22.1 从隐文档到条件生成	116
22.2 切分、召回与重排	117
22.3 证据组织、引用与回答约束	117
22.4 长上下文、迭代检索与系统评价	118
第 23 章 评价: 分数到底说明了什么	120
23.1 分类、结构预测与排序	120
23.2 编辑距离与文本重叠指标	121
23.3 可执行任务与多样本评价	122
23.4 基准设计与大模型裁判	122
23.5 统计波动、污染与错误分析	123
第 24 章 综合实验: 把模型放回任务中	126
24.1 实验一: 从词袋分类到上下文表示	126
24.2 实验二: 序列标注中的结构是否有用	127
24.3 实验三: 一个有证据的课程问答系统	128

24.4 读懂实验结果的共同方法	128
附录 A 推导专题：把几条共同思路连起来	130
A.1 隐变量、下界与 EM	130
A.2 配分函数、边缘与期望梯度	131
A.3 对比学习中的梯度与负例分布	132
A.4 策略梯度的基线、重要性采样与支持集	133
A.5 数值稳定：数学相等不代表浮点计算相同	133
附录 B 算法实验：用小输入检查大公式	135
B.1 概率模型：先检查归一化与边界	135
B.2 动态规划：用穷举作参考答案	136
B.3 神经计算：检查因果性与等价性	136
B.4 偏好与采样：检查目标是否真的一致	137
附录 C 综合习题与解答	139
C.1 概率与分类	139
C.2 表示与语言模型	139
C.3 结构预测与语义	140
C.4 Transformer 与训练	141
C.5 偏好、推理与检索	142
C.6 评价与实验解释	143
参考文献	145

记号与问题地图

语言任务的共同困难，是从有限的观测中推断没有直接写出来的信息。词义消歧推断一个词在当前语境中的含义，句法分析推断词之间的组织方式，语言模型则推断下一步可能出现的文字。我们先研究可解释、可精确求解的模型，再逐步放宽独立性假设，学习更有表达力的表示。

阅读公式时，先区分三个对象：**模型规定哪些答案可能，学习决定如何评分，推断决定怎样寻找答案**。一个模型可以配合多种推断算法；一个动态规划算法也可以为多个模型服务。

记号	含义
$x, x_{1:T}$	输入或输入序列；下标 $1:T$ 包含端点
$y, y_{1:T}$	标签或目标序列； $y_{<t} = y_{1:t-1}$
$\mathcal{V}, V$	词表及其大小；token 是词表中的离散单位
N, T, d, K	样本数、序列长度、表示维度、标签数
$\theta, \mathcal{D}$	可学习参数与数据集
p_θ, q	模型分布与真实 / 目标分布
$E \in \mathbb{R}^{V \times d}$	嵌入矩阵，每一行对应一个 token
$\mathbf{1}[\cdot]$	条件成立为 1，否则为 0
$\log, \exp$	默认使用自然对数与自然指数
$\odot, [a; b]$	逐元素乘法与向量拼接

向量默认写成列向量；批量输入与 Transformer 的序列矩阵采用“每行一个 token”。改变约定只会引入转置，不会改变算法。每章中有特殊记号时，在首次使用处定义。

经典部分的学习路径是：分类建立条件概率，语言模型处理可变长度文本，表示学习共享统计强度，动态规划处理指数大小的结构空间。大模型部分沿着表示、训练数据、训练目标、生成算法与评价展开。检索把两部分连接起来：外部文档既可以作为搜索结果，也可以成为生成时的条件。

第一部分

概率、表示与语言结构

第 1 章 从语言问题到可学习的模型

“我在银行旁边看见了他”包含实体、位置和事件，却没有把它们标成机器可以直接使用的变量。自然语言处理首先要做的，是选定一个问题：我们希望输出一个类别、一系列标签、一棵树，还是另一段文字？一旦输出空间明确，模型、学习算法和评价方法才有共同的对象。

本章不展开应用发展史，而建立贯穿全书的数学框架。尤其要理解：概率不是给模型披上的外衣，它规定了如何组合证据、如何表达不确定性，以及如何从数据中估计参数。

1.1 任务、表示与假设空间

设输入为 $x \in \mathcal{X}$ ，输出为 $y \in \mathcal{Y}(x)$ 。分类任务的 $\mathcal{Y}$ 是固定标签集合；序列标注中，长度为 T 的输入有至多 K^T 个标签序列；句法分析的候选是一组满足树结构约束的分析结果。

一个统一的预测式为

$$\hat{y} = \arg \max_{y \in \mathcal{Y}(x)} s_{\theta}(x, y). \quad (1.1)$$

s_{θ} 是评分函数，不一定是概率。若希望分数有概率解释，可以定义

$$p_{\theta}(y | x) = \frac{\exp s_{\theta}(x, y)}{Z_{\theta}(x)}, \quad Z_{\theta}(x) = \sum_{y' \in \mathcal{Y}(x)} \exp s_{\theta}(x, y'). \quad (1.2)$$

分母称为配分函数。分类时求和只有几个或几万个项；结构预测时，它可能包含指数多个项。很多算法的核心工作，就是在不显式枚举全部结构的情况下计算这个量，或找到最高分的结构。

任务	输出对象	必须保留的信息
词义消歧	一个义项标签	目标词及其上下文
情感分类	正面 / 负面等类别	否定、程度及评价对象
命名实体识别	实体跨度及类型	相邻标签与边界
句法分析	成分树或依存树	层级关系与结构约束
语义解析	逻辑式、查询或程序	变量绑定与组合关系
语言生成	可变长度 token 序列	历史上下文与终止条件

表 1.1 输出空间决定问题的计算结构。

输入表示同样包含假设。词袋把文本变成计数向量，忽略词序；循环网络按顺序更新状态；自注意力允许任意位置建立联系。模型可以学到的规律，既取决于数据，也取决于表示保留了什么。

例如，“狗咬人”和“人咬狗”的词袋相同。任何只接收这个词袋的确定性分类器都无法区分二者。这不是训练不足，而是输入表示已抹去所需信息。增加模型层数不能恢复被表示永久丢掉的差异。

数学直觉 | 先问哪些区别仍然可见

若两个输入被映射为相同的表示 $\phi(x_1) = \phi(x_2)$ ，则任何形如 $g(\phi(x))$ 的模型都对它们输出相同结果。设计表示，就是决定哪些变化应当忽略、哪些变化必须保留。

1.2 联合概率、条件概率与隐变量

从联合概率出发，贝叶斯公式给出

$$p(y | x) = \frac{p(x, y)}{p(x)} = \frac{p(x | y)p(y)}{\sum_{y'} p(x | y')p(y')}. \quad (1.3)$$

其中 $p(y)$ 是先验， $p(x | y)$ 是似然， $p(y | x)$ 是后验。这里的“后验”是关于标签的条件分布，暂时还没有对模型参数做贝叶斯推断。

生成式分类器先描述每一类如何产生输入，再用贝叶斯公式分类。例如，朴素贝叶斯描述体育新闻中各个词的出现概率。判别式分类器直接描述 $p(y | x)$ 或决策边界，例如逻辑回归。**能生成文本与“生成式分类器”不是同一个分类标准**：自回归语言模型可以作为 $p(x)$ 的生成模型，也可以在给定指令后建模 $p(y | x)$ 。

如果数据中有看不见的变量 z ，需要通过求和消去它：

$$p_{\theta}(x) = \sum_z p_{\theta}(x, z). \quad (1.4)$$

隐马尔可夫模型中的隐变量是标签序列，主题模型中的隐变量是主题指派。边缘化保留所有可能解释的总质量；最大化则只保留最好的一个解释。

$$\underbrace{\sum_z p(x, z)}_{\text{所有解释的总概率}} \neq \underbrace{\max_z p(x, z)}_{\text{最强解释的概率}}. \quad (1.5)$$

假设一个句子有两种分析，概率分别为 0.3 和 0.25。它的总概率是 0.55，最佳分析的概率为 0.3。如果另一个句子只有一个概率为 0.4 的分析，按总概率和按最佳分析评分，会产生不同的排序。

条件独立让复杂联合分布可以拆成局部因子。若 $X_1 \perp X_2 | Y$ ，则

$$p(x_1, x_2 | y) = p(x_1 | y)p(x_2 | y). \quad (1.6)$$

这不意味着 X_1 与 X_2 在总体上独立。雨天可能同时导致撑伞和路面潮湿，即使给定天气后两个观测接近独立，不知道天气时它们仍会相关。

链式法则与模型假设的区别

对任意序列，恒有

$$p(x_{1:T}) = \prod_{t=1}^T p(x_t | x_{<t}). \quad (1.7)$$

链式法则没有做近似。把条件改成最近 $n-1$ 个词，才是 n -gram 的马尔可夫假设。把整个历史映射成有限维状态，则是神经语言模型施加的表示约束。区分恒等变形和建模假设，可以避免把一种具体架构误认为概率论的必然要求。

1.3 学习目标与信息论解释

给定独立同分布的训练样本 $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ ，条件最大似然估计为

$$\hat{\theta} = \arg \max_{\theta} \prod_{i=1}^N p_{\theta}(y_i | x_i) = \arg \min_{\theta} -\frac{1}{N} \sum_{i=1}^N \log p_{\theta}(y_i | x_i). \quad (1.8)$$

取对数把乘法变成加法，并改善数值稳定性；除以 N 便于比较不同批量的损失，不改变无正则项时的最优解。若再加入正则项，其系数必须与“求和还是取平均”的约定一致。

令目标标签分布为 $q(y | x)$ ，交叉熵为

$$H(q, p_{\theta}) = - \sum_y q(y | x) \log p_{\theta}(y | x). \quad (1.9)$$

当 q 在正确标签处为 1、其他位置为 0，交叉熵就退化为负对数似然。通过加减 $-\sum_y q(y) \log q(y)$ 得到

$$H(q, p_{\theta}) = H(q) + D_{\text{KL}}(q \| p_{\theta}), \quad (1.10)$$

$$D_{\text{KL}}(q \| p) = \sum_y q(y) \log \frac{q(y)}{p(y)} \geq 0. \quad (1.11)$$

对固定的真实分布 q ，最小化交叉熵就是使模型分布接近它。 $H(q)$ 表示问题本身的不可约不确定性；KL 表示模型额外带来的预测代价。

数学直觉 | 为什么自信地犯错代价很高

把正确答案的概率从 0.8 降到 0.4，损失增加 $\log 2$ ；从 0.02 降到 0.01 也增加 $\log 2$ 。对数损失关心相对概率。若把真实发生的事件赋予零概率，损失发散，这正是平滑和数值稳定计算的重要原因。

训练误差低不等于泛化好。常见的经验风险最小化形式为

$$\hat{\theta} = \arg \min_{\theta} \left[\frac{1}{N} \sum_i \ell_{\theta}(x_i, y_i) + \lambda \Omega(\theta) \right]. \quad (1.12)$$

Ω 可以限制参数大小，也可以来自结构先验。模型越灵活，越能拟合偶然噪声；但不能简单断言参数越多就一定越过拟合，实际结果还受数据量、优化和隐式正则化影响。

数值例子：平均什么非常重要

若两个样本的正确标签概率分别为 0.9 和 0.1，平均负对数似然为

$$-\frac{1}{2}(\log 0.9 + \log 0.1) \approx 1.204. \quad (1.13)$$

先平均概率再取负对数则是 $-\log 0.5 \approx 0.693$ 。两者不同：后一种计算允许一个容易样本掩盖另一个自信错误，因而不是标准最大似然训练目标。

1.4 预测决策与可靠的实验

最高概率标签仅在所有误判代价相同的零一损失下最优。若行动为 a 、真实标签为 y 、代价为 $C(a, y)$ ，贝叶斯决策应最小化条件风险：

$$a^*(x) = \arg \min_a \sum_y C(a, y) p(y | x). \quad (1.14)$$

以二分类为例，假阳性代价为 c_{FP} ，假阴性代价为 c_{FN} 。预测正类的期望代价为 $c_{FP}(1-p)$ ，预测负类为 $c_{FN}p$ ，其中 $p = p(y=1|x)$ 。因此预测正类当且仅当

$$p > \frac{c_{FP}}{c_{FP} + c_{FN}}. \quad (1.15)$$

阈值不是永远等于 0.5，它取决于实际代价；若模型概率未校准，仅代入公式也不能保证最优。

训练集估计参数，验证集选择超参数、阈值与停止时刻，测试集估计固定系统的最终性能。分词器、词表、IDF、标准化统计量、负例池等凡是依赖数据的步骤，都要检查是否间接使用了测试信息。

备注 | 划分单位要与泛化目标一致

同一篇文章的句子、同一用户的消息或同一道题的改写版本，通常不是独立样本。随机按句子划分可能让近重复文本同时进入训练和测试。若关心新作者、新领域或未来数据上的性能，应按对应单位或时间划分。

一个实用基线需要很简单，但不能随意。多数类基线揭示类别偏斜；朴素贝叶斯或线性模型揭示词汇线索能达到的水平；打乱上下文或移除某类输入，可以检查模型是否依赖预期的信息。消融实验应尽量一次改变一个因素，并在相同数据、预算和评价协议下比较。

进一步推导：最大似然怎样得到频率估计

考虑一个只有三个词的词表。训练语料的计数为 (n_1, n_2, n_3) ，总数为 N ，待估概率为 (p_1, p_2, p_3) 。最大似然要解

$$\max_{p_1, p_2, p_3} \sum_{k=1}^3 n_k \log p_k, \quad p_k \geq 0, \quad \sum_k p_k = 1. \quad (1.16)$$

先假设所有计数都大于零，最优概率也位于内部。加入约束乘子：

$$\mathcal{J} = \sum_k n_k \log p_k + \lambda(1 - \sum_k p_k). \quad (1.17)$$

对每个概率求导得 $n_k/p_k - \lambda = 0$ ，因此 $p_k = n_k/\lambda$ 。对 k 求和：

$$1 = \sum_k p_k = \frac{\sum_k n_k}{\lambda} = \frac{N}{\lambda}. \quad (1.18)$$

所以 $\lambda = N$ ，得到 $\hat{p}_k = n_k/N$ 。若某个 $n_k = 0$ ，最大似然允许其概率落在边界零点，这也解释了未见事件为什么会得到零概率。

先验、MAP 与后验均值不是同一估计

给概率向量设置对称 Dirichlet 先验，参数为 α 。其密度与 $\prod_k p_k^{\alpha-1}$ 成正比。观测数据后，后验参数变为 $n_k + \alpha$ 。

后验均值是

$$\mathbb{E}[p_k | \mathcal{D}] = \frac{n_k + \alpha}{N + K\alpha}. \quad (1.19)$$

若后验众数位于内部，MAP 则为

$$\hat{p}_k^{\text{MAP}} = \frac{n_k + \alpha - 1}{N + K(\alpha - 1)}. \quad (1.20)$$

两式相差一个 1。因此，“加一平滑就是均匀 Dirichlet 先验下的 MAP”这一说法并不准确。均匀先验的参数为 1，它的后验均值对应加一平滑；内部 MAP 则回到经验频率。

这一区别来自估计目标不同。后验均值对参数不确定性取平均，MAP 只选择后验密度最高的一点。数据量很大时二者可能接近，小数据时差别更明显。

分类概率如何受到领域先验影响

设训练领域中的正类先验为 π_{old} ，新领域为 π_{new} 。如果只有标签先验改变，而条件分布 $p(x | y)$ 保持不变，则后验优势可以修正为

$$\frac{p_{\text{new}}(1 | x)}{p_{\text{new}}(0 | x)} = \frac{p_{\text{old}}(1 | x)}{p_{\text{old}}(0 | x)} \frac{\pi_{\text{new}}/(1 - \pi_{\text{new}})}{\pi_{\text{old}}/(1 - \pi_{\text{old}})}. \quad (1.21)$$

取对数后，新旧差异只是一个偏置项。比如旧先验为 0.5，新先验为 0.1，旧模型对某样本给出正类概率 0.8。旧后验优势为 4，新后验优势为 4/9，所以新后验为 $4/13 \approx 0.308$ 。

修正的前提很重要。如果新领域中的词语用法也改变， $p(x | y)$ 不再稳定，就不能只改一个先验。概率推导告诉我们何时可行，也明确了何时不能使用。

拒答也可以是一种决策

假设错误预测代价为 1，正确为 0，交给人工复核的代价为 $c \in (0, 1)$ 。若选择最可能标签，条件错误率是 $1 - p_{\text{max}}$ ；复核的代价是 c 。因此仅当

$$p_{\text{max}} \geq 1 - c \quad (1.22)$$

时直接预测更合算。这个规则把“不确定就交给人工”写成明确的风险比较，而不是任意选择一个置信度阈值。它仍依赖概率校准及代价估计。

推演与自检

若系统对两个候选标签给出分数 2 和 1，计算 softmax 后第一类概率为 $e/(e + 1) \approx 0.731$ 。给两个分数同时加 100，结果不变；把分数同时乘 2，第一类概率变为约 0.881。由此可见，**平移不改变 softmax，缩放改变置信程度**。后面温度采样与对比学习中的温度，都建立在这个性质上。

第 2 章 词义消歧与朴素贝叶斯

看到 bank，不能仅凭词形决定它表示银行还是河岸；但看到 mortgage 或 river，歧义便大幅减小。词义消歧是一个适合进入 NLP 的问题：输入和输出都很明确，却能展示上下文、特征、先验、监督与泛化之间的关系。

本章从义项定义出发，建立一个可以手算的分类器。它的独立性假设显然不完全符合语言，却能让我们清楚地看到证据怎样累积，以及哪些工程选择会改变统计模型。

2.1 义项、上下文与任务定义

给定目标词 w 、其出现的上下文 c ，以及该词的候选义项集合 $\mathcal{S}(w)$ ，词义消歧预测

$$\hat{s} = \arg \max_{s \in \mathcal{S}(w)} p(s | w, c). \quad (2.1)$$

词形是类型，一次具体出现是实例；义项介于二者之间。同一个词在不同句子中可以有不同义项，同一个义项也可能由不同词表达。

义项库通常由词典、词汇知识库或人工标注定义。WordNet 使用同义词集组织概念，并包含上下位等关系。实际标注的困难在于粒度：粗粒度的“金融机构 / 河岸”容易区分，细粒度的相关含义有时连人也难稳定判断。

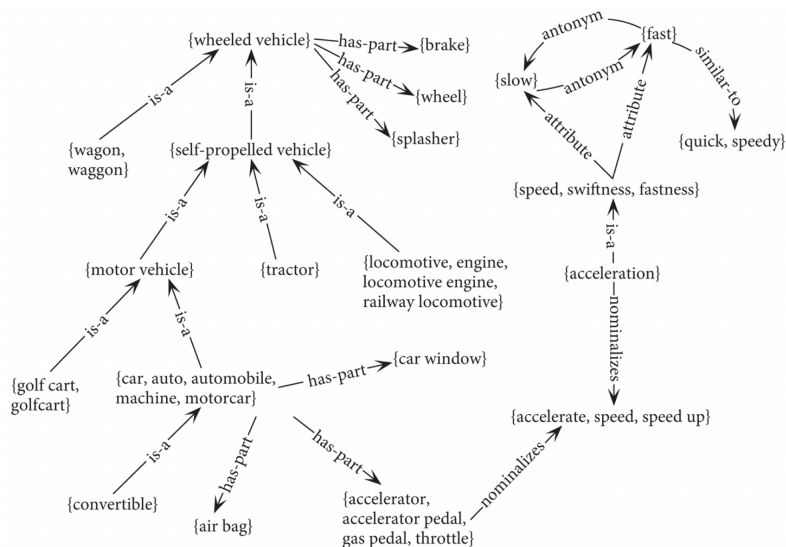


图 2.1 词汇知识库用关系组织词义，而非把词视为孤立编号。（课件 lec2_wsd，第 12 页）

可以使用哪些上下文

局部窗口中的词提示主题，搭配提示固定用法，词性缩小候选范围，句法依存揭示“谁修饰谁”。在“他在苹果工作”中，动词“工作”与介词结构往往比窗口里更远的颜色词重要。文档级主题还可以帮助多个出现共享证据。

可把上下文表示为向量 $x = \phi(w, c)$ 。二值特征表示某词是否出现，计数特征表示出现几次，位置特征区别左邻词与右邻词。例如

$$x_1 = \mathbf{1}[\text{窗口中有 river}], \quad x_2 = \mathbf{1}[\text{左邻词为 central}], \quad x_3 = \mathbf{1}[\text{目标词作名词}]. \quad (2.2)$$

这些特征可以相关；选择朴素贝叶斯时，我们只是用一个简化的概率模型处理它们，并没有声称真实语言独立。

最常见义项基线直接预测训练集中频率最高的义项。如果训练数据中金融义占 80%，一个完全不读上下文的系统就能得到 80% 准确率。因此不能仅凭表面上较高的准确率判断模型是否真正解决了歧义。

2.2 从贝叶斯公式到线性判别

固定目标词后简写标签为 y 、上下文为 x 。朴素贝叶斯假设：**给定义项，各个观测特征条件独立**。若 $x_1, \dots, x_m$ 是离散特征，

$$p(y | x) \propto p(y) \prod_{j=1}^m p(x_j | y). \quad (2.3)$$

比较候选标签时， $p(x)$ 是共同分母，可以省略；如果需要输出校准后的后验概率，仍必须归一化。

对于上下文词计数，设 x_v 是词 v 的出现次数， $\theta_{yv} = p(v | y)$ 。多项式朴素贝叶斯有

$$p(x | y) = \frac{m!}{\prod_v x_v!} \prod_{v \in \mathcal{V}} \theta_{yv}^{x_v}, \quad m = \sum_v x_v. \quad (2.4)$$

在给定输入 x 时，多项式系数与 y 无关，分类分数因此是

$$s_y(x) = \log \pi_y + \sum_v x_v \log \theta_{yv}, \quad \pi_y = p(y). \quad (2.5)$$

它实际上是关于计数向量的线性函数。生成式训练得到的模型，也可以拥有线性的判别边界。

二分类时，将两个标签相减得到对数优势：

$$\log \frac{p(y=1 | x)}{p(y=0 | x)} = \log \frac{\pi_1}{\pi_0} + \sum_v x_v \log \frac{\theta_{1v}}{\theta_{0v}}. \quad (2.6)$$

每个词贡献一个对数似然比。某词在两类中同样常见，贡献接近零；在正类中相对更常见，贡献为正。多个弱线索可以累积成强证据。

数学直觉 | 相对频率比绝对频率更重要

“the” 可能是金融文本中最常见的词，却不一定支持金融义；关键是它在金融义下是否比河岸义下更常见。分类需要比较解释，而不是只看某个解释内部的高频事件。

多项式与伯努利模型

伯努利模型使用 $x_v \in \{0, 1\}$ ，同时建模出现与未出现：

$$p(x | y) = \prod_v \theta_{yv}^{x_v} (1 - \theta_{yv})^{1-x_v}. \quad (2.7)$$

其分数包含 $(1 - x_v) \log(1 - \theta_{yv})$ 。把多项式模型的计数简单截成 0/1，并不等同于完整的伯努利朴素贝叶斯，因为后者利用了“没有出现”的证据。

2.3 估计、平滑与完整例子

设 N_y 为标签 y 的样本数, C_{yv} 为该类上下文中词 v 的总计数。最大似然估计为

$$\hat{\pi}_y = \frac{N_y}{N}, \quad \hat{\theta}_{yv} = \frac{C_{yv}}{\sum_u C_{yu}}. \quad (2.8)$$

如果训练时金融义从未出现 river, 模型就给它零概率。一旦测试上下文包含这个词, 整个金融义的似然变为零, 即使其余证据很强。

加性平滑用伪计数解决这个问题:

$$\hat{\theta}_{yv} = \frac{C_{yv} + \alpha}{\sum_u C_{yu} + \alpha V}, \quad \alpha > 0. \quad (2.9)$$

$\alpha = 1$ 称为加一平滑; 它未必最优, 应在验证集选择。分母加 αV 是为了使整行概率和仍为 1。平滑不是只给没见过的词补一项, 而是把部分概率质量从已见事件转移给其他事件。

义项	money	river	loan
金融义	6	0	4
河岸义	1	8	1

表 2.1 手算用的上下文词频表。两个义项先验均为 $1/2$, 词表仅含这三个词。

加一平滑后, 金融义的词分布为 $(7, 1, 5)/13$, 河岸义为 $(2, 9, 2)/13$ 。对于上下文 money loan, 忽略共同的多项式系数:

$$\tilde{p}(\text{金融}, x) = \frac{1}{2} \frac{7}{13} \frac{5}{13} = \frac{35}{338}, \quad (2.10)$$

$$\tilde{p}(\text{河岸}, x) = \frac{1}{2} \frac{2}{13} \frac{2}{13} = \frac{4}{338}. \quad (2.11)$$

归一化后金融义概率为 $35/39 \approx 0.897$ 。若上下文改成 money river, 两个未归一化分子变为 7 和 18, 河岸义的后验为 $18/25 = 0.72$ 。先验相同并不意味着后验相同, 证据改变了两种解释的相对权重。

重复证据与过度自信

若 money 与 loan 高度相关, 朴素贝叶斯会把两者当作两份独立证据, 可能重复计数。这解释了为什么它的分类准确率不错, 概率却容易过于极端。预测标签正确和概率校准良好, 是两个不同的要求。

算法实现 | 多项式朴素贝叶斯预测

```
import numpy as np

def predict_nb(counts, prior, x, alpha=1.0):
    # counts: [K, V]; x: [V]
    prob = (counts + alpha)
    prob /= prob.sum(axis=1, keepdims=True)
    score = np.log(prior) + np.log(prob) @ x
    z = np.exp(score - score.max())
    return int(score.argmax()), z / z.sum()

C = np.array([[6., 0., 4.], [1., 8., 1.]])
label, posterior = predict_nb(C, [0.5, 0.5], [1, 0, 1])
# label == 0; posterior ~= [0.8974, 0.1026]
```

词表和计数只从训练集建立。测试中的未知词需要预先约定映射到 <unk> 或忽略，不能临时改变 V 却不重新归一化参数。

2.4 从词典匹配到上下文表示

没有足够标注时，可以比较上下文与词典释义。简化 Lesk 算法计算重叠词数：

$$\hat{s} = \arg \max_s |\text{tokens}(c) \cap \text{tokens}(\text{gloss}(s))|. \quad (2.12)$$

去除停用词、词形归一、扩展词典例句或给罕见词更高权重，都会改变匹配效果。它不需要训练分类器，但依赖释义措辞：语义相近而词形不同的表达可能完全没有重叠。

用神经表示可以把相同框架推广成相似度比较。令 $h(w, c)$ 为上下文化的目标词表示， $g(s)$ 为义项释义表示，则

$$p(s | w, c) = \frac{\exp(h(w, c)^\top g(s) / \tau)}{\sum_{s' \in \mathcal{S}(w)} \exp(h(w, c)^\top g(s') / \tau)}. \quad (2.13)$$

这里 $\tau > 0$ 控制分布的集中程度。与每个词单独训练分类器相比，释义编码允许不同目标词共享参数，也更容易使用没有大量标注的义项。

自训练从少量标注开始，将高置信预测加入训练集。其主要风险是错误强化：初始偏差会影响伪标签，伪标签又进一步强化同一偏差。需要按类别控制选择数量、保留独立验证集，并检查高置信是否真的对应高准确率。

“同一篇文章中的同一词通常使用同一义项”和“固定搭配倾向于对应固定词义”可以作为偏好，但不应作为绝对约束。一篇比较银行与河岸的文章，完全可以交替使用两个含义。

评价必须对准义项分布

多类任务中，对每一类分别计算精确率、召回率和 F_1 ，再平均得到宏平均。微平均先累加各类的 TP、FP、FN，再计算指标。在每个样本恰有一个真标签和一个预测标签的完整多类分类中，微平均 F_1 等于准确率。

对稀有义项，应单独查看召回率；对跨领域场景，应关注先验变化。训练语料来自金融新闻时，最常见义项可能是银行；自然地理文本中则未必。**义项频率是语料的性质，不是词语永恒不变的性质。**

完整算例：特征重复会怎样改变置信度

假设目标词有金融义 F 与河岸义 R，先验都为 $1/2$ 。某二值线索 L 在金融义下出现概率为 0.8，在河岸义下为 0.2。看到 L 后，后验优势为

$$\frac{p(F | L)}{p(R | L)} = \frac{0.8}{0.2} = 4, \quad (2.14)$$

所以 $p(F | L) = 0.8$ 。

现在人为复制这个特征，得到 $L_1 = L_2 = L$ ，并让朴素贝叶斯误把它们视为条件独立。模型给出的优势变成

$$\frac{p(F | L_1, L_2)}{p(R | L_1, L_2)} = \frac{0.8^2}{0.2^2} = 16, \quad (2.15)$$

后验变为 $16/17 \approx 0.941$ 。实际上没有得到任何新信息，置信度却显著提高。这是重复证据导致过度自信的一个极端例子。

如果重复十次，错误独立假设会把优势推到 4^{10} 。分类标签一直没有变，因此仅看准确率很难发现问题；负对数似然和校准检查更容易暴露这种过度自信。

二值模型怎样使用没有出现的词

设词表只有 loan 与 river。金融义中，两词出现概率分别为 0.8 与 0.1；河岸义中分别为 0.1 与 0.7。输入中两个词都没有出现，即 $x = (0, 0)$ 。

伯努利模型的条件似然为

$$p(x | F) = (1 - 0.8)(1 - 0.1) = 0.18, \quad (2.16)$$

$$p(x | R) = (1 - 0.1)(1 - 0.7) = 0.27. \quad (2.17)$$

先验相同时，河岸义后验为 $0.27/(0.18 + 0.27) = 0.6$ 。没有出现也能提供证据，因为两类对“缺席”的预测不同。

多项式模型如果把输入表示为零计数，词项对数和为零，只剩先验。它描述的观测过程不同，所以不应期待与伯努利模型给出同样结果。

未知词处理为何必须在训练时决定

假设某类的词频总数为 20，原词表有 5 个词，加一平滑分母为 25。若测试时遇到新词，临时把词表扩成 6，却仍保留旧概率，则旧概率和已经为 1，再给新词正概率后总和就超过 1。

一种一致做法是在训练前把低频词映射为 <unk>，把它作为词表中的普通一项。测试时所有未登录词都映射到该项。另一种做法是明确忽略词表外特征，但这意味着模型只评价已知词，不会从未知词本身获得证据。

未知词比例也应成为错误分析的一部分。若新领域中的大量专业词都映射到同一个 <unk>，模型会丢掉区分能力。此时改进词表、子词或字形特征，比单纯增加训练迭代更有针对性。

第 3 章 对数线性模型与学习的几何

朴素贝叶斯通过“每一类如何产生词语”间接完成分类。但分类任务未必需要完整解释输入：如果我们的目标只是判断新闻属于体育还是财经，直接学习哪些证据支持哪个类别，往往更自然。对数线性模型正是这样一种做法。

它也是理解神经网络输出层、条件随机场和偏好学习的重要桥梁。后面模型越来越复杂，但“线性评分、指数归一化、交叉熵训练”这一局部结构会反复出现。

3.1 从特征到概率

设 $f(x, y) \in \mathbb{R}^m$ 是输入与候选标签的联合特征，参数为 $\theta \in \mathbb{R}^m$ 。分数与条件概率分别为

$$s_\theta(x, y) = \theta^\top f(x, y), \quad p_\theta(y | x) = \frac{e^{\theta^\top f(x, y)}}{\sum_{y'} e^{\theta^\top f(x, y')}}. \quad (3.1)$$

称为“对数线性”，是因为对数概率等于线性分数减去归一化项：

$$\log p_\theta(y | x) = \theta^\top f(x, y) - \log Z_\theta(x). \quad (3.2)$$

它不是说概率本身对参数线性，也不是说整个对数概率都是线性函数。

例如，一个特征可以是“上下文含 loan 且候选标签为金融义”。若条件同时成立，特征为 1；否则为 0。同一个输入对不同候选标签会激活不同的特征。

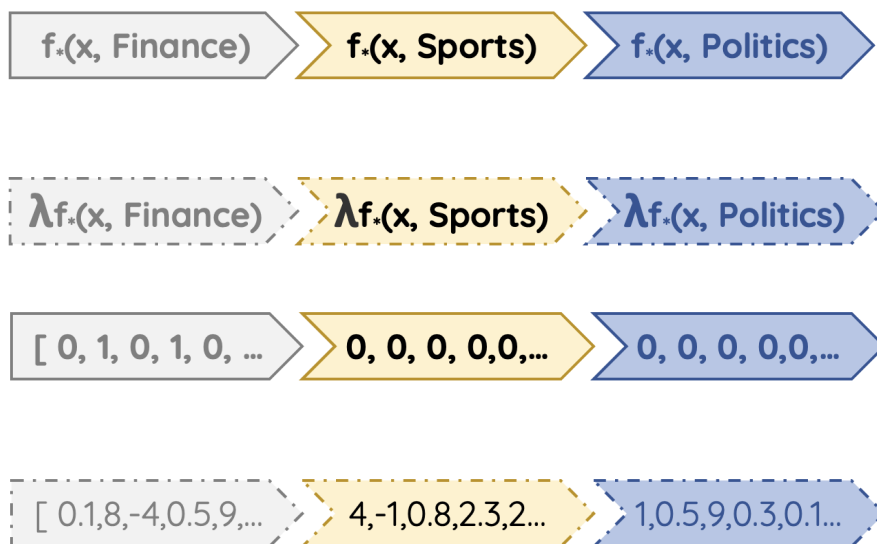


图 3.1 标签相关的特征可组织成不同的参数块。每次计算只激活与候选标签相关的部分。（课件 lec3_loglinear，第 14 页）

更常见的实现把文本先变为 $x \in \mathbb{R}^d$ ，再对每个标签使用一个权重向量：

$$z = Wx + b, \quad W \in \mathbb{R}^{K \times d}, \quad b \in \mathbb{R}^K, \quad p = \text{softmax}(z). \quad (3.3)$$

将所有 w_y 拼成一个大参数向量，并令 $f(x, y)$ 只在第 y 块等于 x ，即可与前一种写法对应。

二分类退化为逻辑回归

令 $z_1 = w_1^\top x + b_1$ 、 $z_0 = w_0^\top x + b_0$ ，则

$$p(y = 1 | x) = \frac{e^{z_1}}{e^{z_0} + e^{z_1}} = \frac{1}{1 + e^{-(z_1 - z_0)}} \quad (3.4)$$

$$= \text{sigmoid}(w^\top x + b), \quad (3.5)$$

其中 $w = w_1 - w_0$ 、 $b = b_1 - b_0$ 。因此逻辑回归只需要一组相对权重。其对数优势为

$$\log \frac{p(y = 1 | x)}{1 - p(y = 1 | x)} = w^\top x + b. \quad (3.6)$$

这条等式把线性几何和概率联系起来：在输入空间沿 w 方向移动，对数优势按固定速度变化。

3.2 交叉熵梯度为什么是“预测减目标”

对一个正确标签为 y 的样本，负对数似然是

$$\ell(\theta) = -\theta^\top f(x, y) + \log \sum_{y'} e^{\theta^\top f(x, y')}. \quad (3.7)$$

先对第一项求导，得到 $-f(x, y)$ 。第二项使用链式法则：

$$\nabla_\theta \log Z = \frac{1}{Z} \sum_{y'} e^{\theta^\top f(x, y')} f(x, y') \quad (3.8)$$

$$= \sum_{y'} p_\theta(y' | x) f(x, y'). \quad (3.9)$$

于是

$$\nabla_\theta \ell = \mathbb{E}_{y' \sim p_\theta(\cdot | x)} [f(x, y')] - f(x, y). \quad (3.10)$$

梯度下降从参数中减去这个差，等价于增强真实标签的特征、削弱模型当前过度期待的特征。

数学直觉 | 学习是在纠正期望

如果某个特征在真实样本中经常出现，而模型预测它出现的频率偏低，就应增加它的权重；如果模型过度使用某个线索，就应降低权重。没有正则项且达到有限的驻点时，经验特征期望与模型期望匹配。

对 softmax 输出层，设 one-hot 目标为 q ，有

$$\frac{\partial \ell}{\partial z_k} = p_k - q_k, \quad \frac{\partial \ell}{\partial W} = (p - q)x^\top, \quad \frac{\partial \ell}{\partial x} = W^\top (p - q). \quad (3.11)$$

最后一式尤其重要：如果 x 由神经编码器产生，误差信号可以继续传回编码器。这正是手工特征模型与端到端神经模型的连接处。

一次数值更新

令 $x = (1, 2)^\top$ ，二分类参数 $w = (0, 0)^\top$ 、 $b = 0$ ，真实标签为 1。初始 $p = 0.5$ ，损失约 0.693，梯度为

$$\nabla_w \ell = (p - y)x = (-0.5, -1)^\top, \quad \nabla_b \ell = -0.5. \quad (3.12)$$

用学习率 0.1 更新后， $w = (0.05, 0.1)^\top$ 、 $b = 0.05$ 。新 logit 为 0.3，正类概率约 0.574，损失下降到约 0.554。这个例子体现了梯度方向如何让正确类别的概率上升。

3.3 凸性、正则化与稳定计算

再次求导，单样本损失的 Hessian 为特征的协方差：

$$\nabla_{\theta}^2 \ell = \mathbb{E}[f f^{\top}] - \mathbb{E}[f] \mathbb{E}[f]^{\top} = \text{Cov}_{p_{\theta}}(f). \quad (3.13)$$

对任意向量 v , $v^{\top} \text{Cov}(f) v = \text{Var}(v^{\top} f) \geq 0$, 所以固定特征下的负对数似然是凸函数。神经网络同时学习 $f_{\phi}(x, y)$ 时, 这个凸性结论通常不再适用。

凸不等于存在有限且唯一的最优解。在可分数据上, 无正则逻辑回归可以让权重不断增大、损失趋于零, 而没有有限参数达到这个下确界。softmax 还存在给所有类别共同加同一线性分数的不识别性。

加入 L_2 正则：

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_i \ell_i(\theta) + \frac{\lambda}{2} \|\theta\|_2^2, \quad \nabla \mathcal{L} = \frac{1}{N} \sum_i \nabla \ell_i + \lambda \theta. \quad (3.14)$$

它抑制过大的权重。在最大后验解释下, 对参数设置零均值高斯先验, 会产生相应的平方惩罚。 L_1 惩罚 $\lambda \|\theta\|_1$ 则更容易得到稀疏解, 但在零点不可微, 需要次梯度或近端方法。

log-sum-exp 技巧

直接计算 e^{1000} 会溢出。利用对任意常数 m 都成立的恒等式

$$\text{LSE}(z) = \log \sum_k e^{z_k} = m + \log \sum_k e^{z_k - m}, \quad (3.15)$$

选择 $m = \max_k z_k$, 所有指数项都不大于 1。不要先计算概率、截断再取对数来替代稳定的 log-softmax; 后者能保留极小概率对应的正确梯度。

算法实现 | 一个小批量的线性分类训练

```
import numpy as np

def step(X, y, W, b, lr=0.1, lam=1e-3):
    z = X @ W.T + b          # [B, K]
    z -= z.max(axis=1, keepdims=True)
    logp = z - np.log(np.exp(z).sum(axis=1, keepdims=True))
    p = np.exp(logp)
    loss = -logp[np.arange(len(y)), y].mean()
    loss += 0.5 * lam * (W * W).sum()
    p[np.arange(len(y)), y] -= 1
    p /= len(y)
    dW, db = p.T @ X + lam * W, p.sum(axis=0)
    return W - lr*dW, b - lr*db, loss
```

此处不正则化偏置, X 的每行是一个样本。梯度中的平均因子与损失保持一致。

3.4 文本特征与模型边界

词袋表示可取原始计数、二值值或 TF-IDF。对文档 d 中的词 t , 一个常用的平滑定义是

$$\text{tfidf}(t, d) = \text{tf}(t, d) \left[\log \frac{N+1}{\text{df}(t)+1} + 1 \right]. \quad (3.16)$$

TF 表示词在文档内的重要程度，IDF 降低普遍出现词的权重。公式有多种约定；比较实验时应固定分词、词频变换和归一化方式。IDF 必须由训练文档计算。

线性模型并不等于只能处理非常简单的问题。字符 n -gram 能捕捉词缀、拼写变体和局部形式；词 n -gram 能编码否定搭配；特征交叉能表达“在某种条件下某词才有意义”。但交叉特征数量会快速增长，神经表示学习的一个动机，就是自动组织这些可共享的组合。

备注 | 可解释系数也依赖表示

较大的正系数表示在其他特征固定时增加该特征会提高某类别的相对分数。若特征尺度不同，直接比较系数大小没有意义；若特征高度相关，权重可能在它们之间重新分配，也不应把系数解释为因果作用。

多标签任务允许一个样本同时属于多个类别，通常每类使用独立 sigmoid 与二元交叉熵。多类任务要求标签互斥，使用 softmax。把二者混用会改变概率空间：softmax 强制所有类别概率和为 1，而独立 sigmoid 不施加此约束。

概率校准与温度

即使准确率不变，模型也可能过于自信。温度缩放使用

$$p_T(y | x) = \text{softmax}(z/T)_y, \quad T > 0. \quad (3.17)$$

固定 T 不改变 argmax ，但会改变概率。可在独立验证集上最小化负对数似然来估计 T 。它只能调整置信度的整体尺度，不能修复错误排序或所有类型的分布偏移。

完整推导：softmax 的 Jacobian

令 $p_i = e^{z_i}/Z$ 、 $Z = \sum_k e^{z_k}$ 。对 z_j 求导，用商法则：

$$\frac{\partial p_i}{\partial z_j} = \frac{\mathbf{1}[i=j]e^{z_i}Z - e^{z_i}e^{z_j}}{Z^2} = p_i(\mathbf{1}[i=j] - p_j). \quad (3.18)$$

因此对角项为 $p_i(1 - p_i)$ ，非对角项为 $-p_i p_j$ 。增大一个 logit，会提高自身概率，同时压低其他类别概率。

对一般目标分布 q ，交叉熵为 $\ell = -\sum_i q_i \log p_i$ 。继续使用链式法则：

$$\frac{\partial \ell}{\partial z_j} = -\sum_i \frac{q_i}{p_i} p_i(\mathbf{1}[i=j] - p_j) \quad (3.19)$$

$$= -q_j + p_j \sum_i q_i = p_j - q_j. \quad (3.20)$$

最后一步用到 q 的总和为 1。这解释了为什么 one-hot 标签、软标签和知识蒸馏都能出现相同的“预测减目标”结构。

二阶导数告诉我们什么

对二分类、单个特征 x ，令 $p = \text{sigmoid}(wx + b)$ 。有

$$\frac{\partial \ell}{\partial w} = (p - y)x, \quad \frac{\partial^2 \ell}{\partial w^2} = p(1 - p)x^2. \quad (3.21)$$

p 接近 0.5 时曲率较大，接近 0 或 1 时曲率较小。注意自信错误时一阶梯度仍可能很大；曲率小并不意味着损失小。

多维 Hessian 是 $p(1-p)xx^\top$ 。若所有样本只沿同一个特征方向变化，其他方向可能无法被数据识别。正则化不仅限制权重，也能改善这种病态几何。

从最大熵角度重新理解同一个模型

另一条推导路线是：在满足经验特征期望的所有条件分布中，选择条件熵最大的分布。固定输入经验分布 $\tilde{p}(x)$ ，约束为

$$\sum_x \tilde{p}(x) \sum_y p(y|x) f_j(x, y) = \frac{1}{N} \sum_i f_j(x_i, y_i). \quad (3.22)$$

再最大化 $-\sum_x \tilde{p}(x) \sum_y p(y|x) \log p(y|x)$ ，并加入各输入下的归一化约束。对 p 求导后得到

$$\log p(y|x) = \sum_j \theta_j f_j(x, y) - \log Z(x). \quad (3.23)$$

所以在适当条件下，最大熵约束问题与对数线性最大似然是同一结构的两面。前者强调“除已知特征统计外，不额外作偏好”，后者强调“让已观测标签更可能”。

有限差分检查梯度

手推梯度可以用小维度数实验检查。对第 j 个参数，中心差分为

$$g_j^{\text{num}} = \frac{\ell(\theta + \epsilon e_j) - \ell(\theta - \epsilon e_j)}{2\epsilon}. \quad (3.24)$$

在双精度、平滑目标和合适 ϵ 下，它应接近解析梯度。 ϵ 太大造成截断误差，太小又受浮点舍入影响。通常先在极小模型上检查，而不是对大模型所有参数进行有限差分。

算法实现 | 一个坐标的梯度核验

```
eps = 1e-5
plus, minus = theta.copy(), theta.copy()
plus[j] += eps
minus[j] -= eps
numeric = (loss(plus) - loss(minus)) / (2*eps)
relative_error = abs(numeric-analytic[j]) / max(
    1.0, abs(numeric), abs(analytic[j]))
```

如果误差很大，先检查损失是求和还是平均，以及正则化的系数是否在两处一致。

推演与自检

三类 logits 为 (2, 1, 0)，正确标签为第三类。概率约为 (0.665, 0.245, 0.090)，因此梯度为 (0.665, 0.245, -0.910)。梯度下降会同时压低前两类 logit、抬高第三类。检查梯度分量之和为零，这与“共同平移 logits 不改变概率”的对称性一致。

第 4 章 语言模型、稀疏性与平滑

分类器从有限标签中选答案，语言模型则面对任意长度的字符串。一个句子即使从未在训练集中出现，也可能完全合乎语言规律；因此不能把每句话当作一个独立类别来记忆。我们需要把句子概率分解成可以从有限数据估计的局部项。

计数语言模型看似古典，却把大语言模型仍然要面对的问题暴露得非常清楚：什么算一个单位，如何表示历史，如何处理没见过的事件，以及怎样比较两个模型的预测质量。

4.1 自回归分解与边界符号

将句子表示为 $w_1, \dots, w_T$ ，再加终止符 $\langle /s \rangle$ 。完整的句子概率为

$$p(w_{1:T}, \langle /s \rangle) = \prod_{t=1}^{T+1} p(w_t | w_{<t}), \quad w_{T+1} = \langle /s \rangle. \quad (4.1)$$

结束符使不同长度的句子可放在同一个概率空间中。若只计算固定前缀的概率而不含停止概率，得到的是“生成以这段文字开头的序列”的概率，不是恰好生成这个完整句子的概率。

n -gram 模型近似为

$$p(w_t | w_{<t}) \approx p(w_t | w_{t-n+1:t-1}). \quad (4.2)$$

bigram 只看前一个词，trigram 看前两个词。句首用 $n-1$ 个开始符补齐历史；开始符不应作为普通下一个词输出。

数学直觉 | 限制的是条件，而不是句长

bigram 可以生成任意长句子，只是每一步只依赖上一个词。它的“记忆短”不等于“句子短”。神经模型的最大上下文长度和生成长度限制，也应区分开来。

从计数估计条件概率：

$$\hat{p}(w | h) = \frac{C(h, w)}{C(h)}, \quad C(h) = \sum_{v \in \mathcal{V}} C(h, v). \quad (4.3)$$

同一个历史下所有后继词计数归一化即可。若某个历史从未出现，分母为零，此时不能直接使用最大似然公式。

以两句训练文本“我爱猫”“我爱狗”为例，加入边界后， $p(\text{我} | \langle s \rangle) = 1$ 、 $p(\text{爱} | \text{我}) = 1$ ， $p(\text{猫} | \text{爱}) = p(\text{狗} | \text{爱}) = 1/2$ 。因此训练句“我爱猫”的概率为 $1/2$ ；未出现的“我爱鸟”却被赋予零概率。

归一化不自动保证终止

每个历史下的下一词分布和为 1，是必要条件；若模型永远不产生结束符，所有有限句子的总概率仍可能小于 1。实践中通常通过学习结束符并设置最大长度保证生成过程终止，但最大长度截断本身改变了输出过程。

4.2 困惑度与不可直接比较的数字

对测试序列中的 M 个预测位置，平均负对数似然为

$$\mathcal{H} = -\frac{1}{M} \sum_{t=1}^M \log p_{\theta}(w_t | h_t). \quad (4.4)$$

困惑度为

$$\text{PPL} = \exp(\mathcal{H}) = \left(\prod_{t=1}^M p_{\theta}(w_t | h_t) \right)^{-1/M}. \quad (4.5)$$

它是实际词概率的几何平均的倒数。若模型每一步均匀地在 V 个词中选择，PPL 为 V ；若每一步给实际词概率 $1/4$ ，PPL 为 4。

“平均有几个候选词”是帮助理解的比喻，真实分布不必均匀，PPL 也不必为整数。较低 PPL 表示在给定评测语料及分词约定下更好的概率预测，并不直接等价于更强的事实问答或更高的指令遵循能力。

假设三个真实 token 的概率分别为 $1/2, 1/4, 1/8$ ，则句子概率为 $1/64$ ，PPL 为 $64^{1/3} = 4$ 。不要对三个概率的倒数做算术平均，那会得到 $14/3$ ，并非困惑度。

备注 | 分词改变计量单位

同一段文字用字符、词或子词分词， M 不同，预测事件也不同，所以 token 级 PPL 不能直接跨分词器比较。需要统一分词协议，或改用每字节 / 每字符负对数似然，并清楚说明编码与边界约定。

测试集的未知词也影响难度。把所有罕见词替换为 `<unk>`，会把多个难预测的事件合并，可能人为降低 PPL。只有词表、未知词处理、结束符计数和语料相同，模型之间的数字才有清晰可比性。

4.3 从加性平滑到折扣与插值

加性平滑写为

$$p_{\alpha}(w | h) = \frac{C(h, w) + \alpha}{C(h) + \alpha V}. \quad (4.6)$$

词表大时，加一平滑可能转移过多质量。若某历史只出现 10 次，而 $V = 10000$ ，总伪计数远大于真实计数，几乎把分布推回均匀。这说明“没有零概率”不等于“好的估计”。

插值混合不同长度的上下文：

$$p(w | u, v) = \lambda_3 p_{\text{ML}}(w | u, v) + \lambda_2 p_{\text{ML}}(w | v) + \lambda_1 p_{\text{ML}}(w), \quad (4.7)$$

其中 $\lambda_j \geq 0$ 、 $\sum_j \lambda_j = 1$ 。高阶分布精细但稀疏，低阶分布粗糙却稳定。权重可依赖历史计数：历史越罕见，越应依赖低阶统计。

回退模型只在高阶事件未见时使用低阶分布；插值模型始终结合多个阶数。两者都可以设计成归一化分布，但一个简单的经验回退评分不一定归一化，不能未经处理就拿来计算标准困惑度。

Good-Turing 的频次之频次

令 N_r 表示恰好出现 r 次的事件种类数，总观测数为 $N = \sum_r r N_r$ 。Good-Turing 用

$$r^* = (r + 1) \frac{N_{r+1}}{N_r} \quad (4.8)$$

替代原始计数，未见事件的总概率质量估计为 N_1/N 。如果语料里大量事件只出现一次，下一次遇到新事件就不应令人意外。

这个原始公式在 $N_r = 0$ 或高频区会不稳定，实际需要对频次之频次进行平滑，并与其他估计配合。它不是在不知道未见事件有哪些时，为每个未见事件都凭空分配确定概率；首先得到的是未见集合的总质量。

绝对折扣从每个已见计数减去常数 D ，把节省的质量交给低阶模型：

$$p(w | h) = \frac{\max(C(h, w) - D, 0)}{C(h)} + \lambda(h) p_{\text{lower}}(w), \quad (4.9)$$

当 $0 < D < 1$ 且计数为整数，取

$$\lambda(h) = \frac{D N_{1+}(h, *)}{C(h)} \quad (4.10)$$

即可补回减去的质量， $N_{1+}(h, *)$ 为该历史后出现过的不同词数。若历史未见，则整体回退到低阶分布。

4.4 Kneser-Ney：什么词适合出现在新环境

假设 Francisco 在语料中频率很高，却几乎只跟在 San 后面。普通 unigram 回退会把它当作任意未知上下文中的好候选，这并不合理。真正需要衡量的是：这个词是否曾在很多不同的上下文中出现。

bigram Kneser-Ney 使用续接概率

$$p_{\text{cont}}(w) = \frac{N_{1+}(*, w)}{N_{1+}(*, *)}, \quad (4.11)$$

分子是词 w 的不同前驱数，分母是语料中不同 bigram 的总数。插值形式为

$$p_{\text{KN}}(w | h) = \frac{\max(C(h, w) - D, 0)}{C(h)} + \lambda(h) p_{\text{cont}}(w). \quad (4.12)$$

高阶项仍描述当前历史的具体偏好，低阶项则描述词出现在新环境中的适应性。

数学直觉 | 频率与生产力

一个词出现一千次但都依赖同一个前驱，说明它在某个搭配中很常见；一个词只出现一百次却拥有八十个不同前驱，说明它能适应多种上下文。回退时需要的更接近后者。

一次完整的归一化检查

设某历史 h 后，a 出现 3 次，b 出现 1 次，c 未出现； $D = 0.5$ ，续接分布为 $(0.2, 0.3, 0.5)$ 。折扣后的直接贡献为 $(2.5/4, 0.5/4, 0)$ 。因不同后继数为 2， $\lambda = 0.5 \times 2/4 = 0.25$ ，所以

$$p(\cdot | h) = (0.625, 0.125, 0) + 0.25(0.2, 0.3, 0.5) = (0.675, 0.200, 0.125). \quad (4.13)$$

三项之和恰好为 1；未见 c 得到非零概率，而且其大小取决于续接分布。

算法实现 | bigram Kneser–Ney 的核心

```
from collections import Counter

def kn_prob(h, w, bigram, D=0.75):
    # bigram is Counter({(previous, next): count})
    total = sum(c for (u, v), c in bigram.items() if u == h)
    followers = {v for u, v in bigram if u == h}
    predecessors = {u for u, v in bigram if v == w}
    cont = len(predecessors) / len(bigram)
    if total == 0:
        return cont
    direct = max(bigram[h, w] - D, 0) / total
    return direct + D * len(followers) / total * cont
```

此版本强调定义，实际实现应预计算每个历史的总计数和不同前后驱数，避免每次查询扫描整个表。

从计数模型走向神经模型

n 增大能利用更长上下文，却加重稀疏性。若词表大小为 V ，完整的 n -gram 条件表有约 $V^{n-1}(V-1)$ 个自由参数。稀疏存储减小实际内存，不能让相似但不同的上下文自动共享证据。

神经语言模型将词与历史映射到连续向量。“我喜欢猫”和“我喜欢狗”不再必须拥有完全分离的统计表；相似表示可以让两个上下文共享参数。下一章先研究这种向量表示怎样从共现关系中学习出来。

完整算例：平滑怎样改变未见句子的概率

考虑词表 $\{a, b, \text{EOS}\}$ ，训练句子为 ab 、 ab 、 aa 。每句前加 BOS，后加 EOS。bigram 计数为

$$C(\text{BOS}, a) = 3, \quad C(a, b) = 2, \quad C(a, a) = 1, \quad C(a, \text{EOS}) = 1, \quad C(b, \text{EOS}) = 2. \quad (4.14)$$

注意 a 出现 4 次，不是 3 次，因为句子 aa 中有两个位置。故 $C(a) = 4$ ，MLE 给出 $p(b | a) = 1/2$ 、 $p(a | a) = 1/4$ 、 $p(\text{EOS} | a) = 1/4$ 。

测试句子 ba 需要从 BOS 生成 b ，但训练中从未出现这条转移，因此 MLE 概率为零。加一平滑后：

$$p(b | \text{BOS}) = \frac{1}{3+3} = \frac{1}{6}, \quad (4.15)$$

$$p(a | b) = \frac{1}{2+3} = \frac{1}{5}, \quad (4.16)$$

$$p(\text{EOS} | a) = \frac{1+1}{4+3} = \frac{2}{7}. \quad (4.17)$$

完整句子概率为 $1/105$ 。如果把结束符计入三个预测位置，困惑度为 $105^{1/3} \approx 4.718$ 。

为什么词表大小也改变已见事件

若增加一个从未在训练中出现的词 c ，并重新做同样的加一平滑，所有历史的分母都会增加 1。即使测试句中没 c ，其概率也改变。这是因为模型为新候选分配了质量，已见事件必须让出一部分。

因此比较平滑参数时必须固定词表。把未知词处理和 α 同时改变，会使效果难以解释。

插值权重可以怎样从验证集学习

假设只有两个已归一化模型 p_1, p_2 ，混合为 $p_\lambda = \lambda p_1 + (1 - \lambda)p_2$ 。验证损失为

$$L(\lambda) = - \sum_i \log [\lambda p_1(w_i | h_i) + (1 - \lambda)p_2(w_i | h_i)], \quad 0 \leq \lambda \leq 1. \quad (4.18)$$

其一阶导数为

$$L'(\lambda) = - \sum_i \frac{p_{1i} - p_{2i}}{\lambda p_{1i} + (1 - \lambda)p_{2i}}, \quad (4.19)$$

二阶导数是非负项之和，因此这是关于单一权重的凸问题。可以网格搜索，也可以用有界优化求解，不需要把权重凭经验固定成相等。

如果某些历史计数很多、另一些几乎没见过，一个全局权重可能过于粗糙。按历史频次分桶学习权重，是在估计可靠性与参数数量之间的一种折中。

续接概率的一张小表

假设不同 bigram 类型如下：San Francisco、in Beijing、to Beijing、from Beijing、in Shanghai、to Shanghai。总共 6 种。Francisco 的不同前驱只有 1 个，Beijing 有 3 个，Shanghai 有 2 个。

$$p_{\text{cont}}(\text{Francisco}) = \frac{1}{6}, \quad p_{\text{cont}}(\text{Beijing}) = \frac{3}{6}, \quad p_{\text{cont}}(\text{Shanghai}) = \frac{2}{6}. \quad (4.20)$$

即使 San Francisco 在语料中重复了上千次，只要没有新增前驱类型，它的续接概率仍为 $1/6$ 。高阶条件 $p(\text{Francisco} | \text{San})$ 可以很高，但在新历史下回退时不会过度偏爱它。

这正是 Kneser–Ney 的分工：高阶统计处理固定搭配，低阶统计处理跨上下文的可用性。

推演与自检

如果验证集上的 trigram 比 bigram 困惑度更高，不能直接得出“三个词的上下文没有价值”。应依次检查样本量、未知历史比例、平滑强度和实现是否归一化。更强的条件信息只有在估计足够可靠时，才可能转化为更好的测试预测。

第 5 章 分布语义、矩阵分解与主题

如果从未见过一个词，却发现它经常出现在“喂养”“毛茸茸”“宠物”附近，我们仍可能猜出它与动物有关。这就是分布语义的出发点：一个词的使用环境，提供了关于其意义的可计算证据。

词向量不是给词贴上一组人为定义的属性，而是让上下文规律变成几何关系。本章先用计数矩阵展示这个过程，再解释降维与潜在主题。下一章将把同样的思想写成预测任务。

5.1 从上下文构造共现矩阵

定义词集合 $\mathcal{V}$ 与上下文集合 $\mathcal{C}$ ，矩阵

$$M_{wc} = \#\{\text{目标词 } w \text{ 与上下文 } c \text{ 的共现事件}\} \tag{5.1}$$

的每行描述一个词在不同环境中的分布。上下文可以是窗口里的另一个词、整篇文档，或带有依存关系的邻词，例如“被 eat 支配的宾语”。

Corpus

... and the cute kitten purred and then ...
... the cute furry cat purred and miaowed ...
... that the small kitten miaowed and she ...
... the loud furry dog ran and bit ...

Basis Vocabulary: {..., bit, cute, furry, loud, miaowed, purred, ...}.
kitten's context words: {cute, purred, small, miaowed, ...}.
cat's context words: {furry, purred, ...}.
dog's context words: {furry, ran, ...}.

图 5.1 相似目标词共享部分上下文，因而获得相似的共现行向量。（课件 lec5_dist，第 34 页）

窗口大小改变所强调的关系。小窗口偏向句法和局部搭配，大窗口更容易反映主题关联；依存上下文可以跨过线性距离，把语法作用相似的词放在一起。没有一种上下文定义对所有任务都最合适。

一阶共现表示两个词经常直接一起出现，如 coffee 与 cup；二阶相似表示两个词拥有类似的上下文，如 coffee 与 tea。前者更接近关联，后者更接近可替换性。把二者混为一谈，容易误解词向量的邻居。

数学直觉 | 词向量是一张压缩的使用说明

若把一个词在所有环境中的使用频率都记录下来，就得到高维分布画像。词向量把这张画像压缩成较短的坐标。相近坐标意味着某种使用模式相似，并不保证两个词可以在任意句子里互换。

从计数到经验概率

令所有词与上下文配对的总数为 $N = \sum_{w,c} M_{wc}$ ，则

$$p(w, c) = \frac{M_{wc}}{N}, \quad p(w) = \frac{\sum_c M_{wc}}{N}, \quad p(c) = \frac{\sum_w M_{wc}}{N}. \quad (5.2)$$

这里的 N 是配对事件数，不一定等于语料 token 数；一个 token 可以与窗口内多个位置形成多个配对。推导 PMI 时，分子和边缘概率必须采用同一种样本空间。

5.2 PMI、余弦与低秩几何

原始共现计数偏爱高频词。点互信息衡量实际共现概率相对于独立基线的提升：

$$\text{PMI}(w, c) = \log \frac{p(w, c)}{p(w)p(c)} = \log \frac{p(c | w)}{p(c)}. \quad (5.3)$$

若上下文 c 在 w 周围比平常常见，PMI 为正；接近独立时为零；共现少于独立预期时为负。正点互信息截去负值：

$$\text{PPMI}(w, c) = \max(\text{PMI}(w, c), 0). \quad (5.4)$$

零计数对应的 PMI 在未平滑时为负无穷，计算 PPMI 时可直接把这些位置保留为零。

假设 $N = 1000$ ，目标词总共出现于 100 个配对，某上下文出现于 50 个配对，两者共现 20 次。独立时的预期共现次数为 $100 \times 50 / 1000 = 5$ ，实际是它的四倍，所以 PMI 为 $\log 4 \approx 1.386$ 。

备注 | 罕见事件的高 PMI

只出现一次的词与上下文如果恰好共现，也可能得到很高 PMI。高比值未必意味着可靠关联。最小计数阈值、折扣、上下文分布平滑等方法，可以减少稀有事件噪声。

余弦相似度比较方向：

$$\cos(u, v) = \frac{u^\top v}{\|u\|_2 \|v\|_2}. \quad (5.5)$$

将两个向量先归一化到单位球面，余弦排序与欧氏距离排序等价，因为

$$\|\bar{u} - \bar{v}\|_2^2 = 2 - 2\bar{u}^\top \bar{v}. \quad (5.6)$$

若不归一化，点积同时受到向量长度影响。某些检索模型会利用长度编码置信或流行度，此时随意改成余弦可能改变模型学到的评分含义。

SVD 如何压缩共现结构

对加权矩阵 $M \in \mathbb{R}^{V \times C}$ 做奇异值分解：

$$M = U \Sigma V^\top, \quad M_k = U_k \Sigma_k V_k^\top. \quad (5.7)$$

M_k 在秩不超过 k 的矩阵中最小化 Frobenius 重构误差。词表示可选 $U_k \Sigma_k$ 、 U_k 或 $U_k \Sigma_k^{1/2}$ ，不同选择对应不同的几何加权，不应把这些约定当作同一表示。

若取词向量 $e_w = (U_k \Sigma_k^{1/2})_w$ 、上下文向量 $u_c = (V_k \Sigma_k^{1/2})_c$ ，则 $e_w^\top u_c$ 近似原矩阵的 M_{wc} 。稀疏记录被分解成两组共享坐标，未见共现也可能通过其他相关行列得到非零预测。

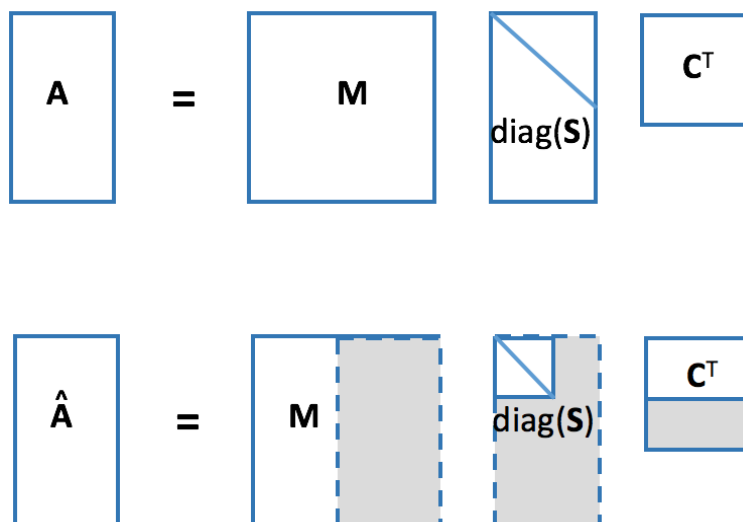


图 5.2 保留较大的奇异值，把高维共现矩阵近似成低秩结构。（课件 lec5_dist，第 61 页）

算法实现 | PPMI 与低秩词表示

```
import numpy as np

def ppmi_svd(C, k):
    total = C.sum()
    expected = C.sum(1, keepdims=True) @ C.sum(0, keepdims=True)
    ratio = np.divide(C * total, expected,
                      out=np.ones_like(C, dtype=float),
                      where=expected > 0)
    M = np.zeros_like(C, dtype=float)
    mask = C > 0
    M[mask] = np.maximum(np.log(ratio[mask]), 0)
    U, s, Vt = np.linalg.svd(M, full_matrices=False)
    return U[:, :k] * np.sqrt(s[:k])
```

大规模稀疏矩阵应使用截断或随机 SVD。不要为了教学公式，把百万词表的矩阵转成稠密数组。

5.3 潜在主题：从几何到概率混合

LSA 用线性代数找潜在方向，却不保证坐标非负，也不自动具有主题比例的意义。概率潜在语义分析引入主题 $z \in \{1, \dots, K\}$ ：

$$p(w | d) = \sum_{z=1}^K p(z | d)p(w | z). \quad (5.8)$$

文档的主题比例 $\theta_{dz} = p(z | d)$ 和主题的词分布 $\phi_{zw} = p(w | z)$ 都在概率单纯形上。一个文档可以同时包含多个主题，而不是被硬分到一个簇。

例如，体育主题给“比赛”概率 0.4，财经主题给它 0.02。如果某文档的主题比例为 (0.75, 0.25)，则

$$p(\text{比赛} | d) = 0.75 \times 0.4 + 0.25 \times 0.02 = 0.305. \quad (5.9)$$

观测到“比赛”后，该词由体育主题生成的后验概率为

$$p(z = \text{体育} | d, \text{比赛}) = \frac{0.75 \times 0.4}{0.305} \approx 0.984. \quad (5.10)$$

文档整体仍可包含财经内容，但这个具体词的责任主要落在体育主题上。

EM 的两步

在 E 步计算每个词计数对应的主题责任：

$$r_{dwz} = \frac{\theta_{dz} \phi_{zw}}{\sum_{z'} \theta_{dz'} \phi_{z'w}}. \quad (5.11)$$

在 M 步使用软计数重新估计：

$$\theta_{dz} \propto \sum_w C_{dw} r_{dwz}, \quad (5.12)$$

$$\phi_{zw} \propto \sum_d C_{dw} r_{dwz}. \quad (5.13)$$

分别对每个文档的主题维度、每个主题的词维度归一化。EM 交替更新隐变量解释与参数；通常只能保证不降低观测似然，不能保证找到全局最优。

LDA 为文档主题比例加先验

潜在狄利克雷分配使用

$$\theta_d \sim \text{Dir}(\alpha), \quad \phi_k \sim \text{Dir}(\eta), \quad z_{dn} \sim \text{Cat}(\theta_d), \quad w_{dn} \sim \text{Cat}(\phi_{z_{dn}}). \quad (5.14)$$

Dirichlet 是定义在概率向量上的分布。对称参数小于 1 时常鼓励靠近单纯形边界的稀疏比例；参数较大时更偏向均匀比例。主题数、先验强度和词表处理共同影响得到的主题。

LDA 与 PLSA 的主要区别并不是“多了一个主题变量”，二者都有主题变量；LDA 为主题比例及词分布提供了概率先验，形成可推广到新文档的生成模型。新文档仍需推断其后验主题比例。

5.4 词义、语境与评价的限制

静态词向量为同一个词形只保存一个向量。bank 的金融义与河岸义混在同一组统计中，邻居可能同时出现两种主题。上下文化表示则写成

$$h_t = f_\theta(w_{1:T}, t), \quad (5.15)$$

同一个词在不同句子中获得不同向量。静态嵌入仍可作为输入查表，但最终表示已包含语境。

词相似度数据集通常要求预测的人类相似度排序与模型排序一致，可使用 Spearman 相关。类比任务通过 $e_b - e_a + e_c$ 搜索候选，检查某些关系是否近似对应平移。它们测到的是表示的局部性质，不能替代下游任务评价。

反义词也可能很相似，因为“热”和“冷”经常出现在相同句法位置与同一语义领域中。关联、同义、蕴含和情感极性不是同一种关系。若任务需要区分它们，必须使用相应监督或结构。

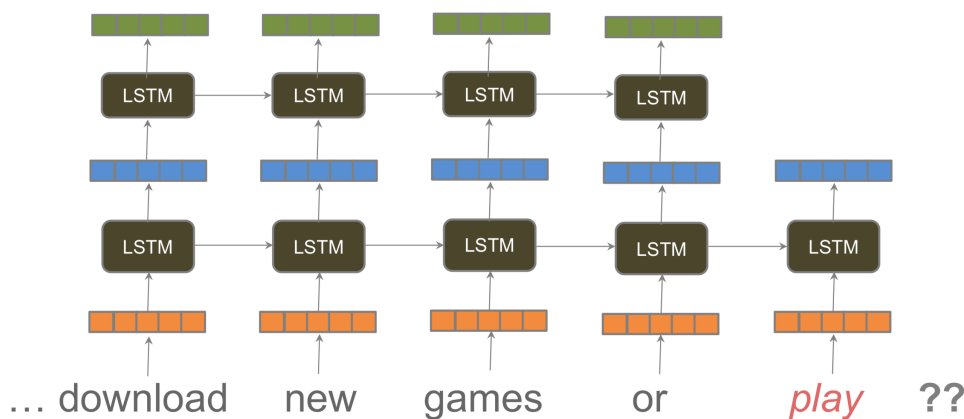


图 5.3 同一个词的表示随上下文变化。层间也可能组织不同层次的信息。（课件 lec5_dist，第 85 页）

数学直觉 | 坐标没有天然名字

对一组词向量共同施加正交变换，点积、长度与余弦均不变。因此第 17 维不必对应某个固定语义概念。更稳定的对象往往是词之间的关系，而不是单个坐标轴的名称。

完整算例：一个二维矩阵怎样压成一维

考虑共现矩阵

$$M = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}. \quad (5.16)$$

它的两个单位特征方向为 $u_1 = (1, 1)^\top / \sqrt{2}$ 与 $u_2 = (1, -1)^\top / \sqrt{2}$ ，对应特征值 4 和 2。因为矩阵对称正定，这里奇异值也为 4 和 2。

秩一近似只保留最大方向：

$$M_1 = 4u_1u_1^\top = \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}. \quad (5.17)$$

原矩阵的两行分别偏爱不同上下文；压成一维后，这个差异被丢掉，两词只保留共享的总体模式。误差为

$$\|M - M_1\|_F^2 = 1^2 + (-1)^2 + (-1)^2 + 1^2 = 4, \quad (5.18)$$

恰好等于被丢弃奇异值的平方 2^2 。低秩分解会去掉变化较小的方向，但这些方向对某个下游任务仍可能重要。

一次主题模型的软计数更新

有一篇文档，两个词的计数为 $C_{d,a} = 3, C_{d,b} = 1$ 。两个主题的词分布为 $\phi_1 = (0.8, 0.2)$ 、 $\phi_2 = (0.3, 0.7)$ ，文档主题比例先设为 $(0.5, 0.5)$ 。

对 a，主题 1 的责任为

$$r_{da1} = \frac{0.5 \times 0.8}{0.5 \times 0.8 + 0.5 \times 0.3} = \frac{8}{11}. \quad (5.19)$$

对 b，主题 1 的责任为 $r_{db1} = 0.2 / (0.2 + 0.7) = 2/9$ 。软主题计数为

$$\tilde{C}_{d,1} = 3 \times \frac{8}{11} + \frac{2}{9} \approx 2.404, \quad \tilde{C}_{d,2} = 4 - \tilde{C}_{d,1} \approx 1.596. \quad (5.20)$$

归一化后新主题比例约为 (0.601, 0.399)。文档中 a 较多，而主题 1 更偏爱 a，所以一次更新就提高了主题 1 的比例。

这一步只更新文档比例。完整 EM 还会汇总所有文档的软计数，更新每个主题的词分布。单篇文档本身不足以稳定识别整个主题系统。

LDA 中的计数与平滑

把主题比例和词分布积分掉后，LDA 的一种 Gibbs 更新为

$$p(z_{dn} = k \mid z_{-dn}, w) \propto (n_{dk}^{-dn} + \alpha_k) \frac{n_{kw}^{-dn} + \eta_w}{n_k^{-dn} + \sum_v \eta_v}. \quad (5.21)$$

第一项看当前文档是否偏爱主题 k，第二项看主题 k 是否偏爱当前词 w。上标 $-dn$ 表示先移除正在重新分配的这个 token，否则会把它自身的旧标签当作支持旧标签的证据。

这个公式连接了两种直觉：文档中已有的主题分配提供局部偏好，主题中已有的词分布提供全局共享。先验伪计数避免某个主题或词因暂时没有计数而永远无法被选择。

语义评价中的一个反例

向量空间可能把“成功”和“失败”放得较近，因为它们都能接“实验”“项目”“原因”。若任务是主题检索，这种相近可能有用；若任务是判断结论极性，就必须进一步区分。

因此，不能仅凭最近邻列表“看起来合理”就确认表示足够好。应构造同主题反义、不同主题同义、实体与属性等多种关系，并检查目标任务究竟需要哪一种区别。

推演与自检

设两个归一化向量的余弦为 0.8，欧氏距离平方为 0.4。如果仅把其中一个向量放大十倍，余弦不变，点积放大十倍。这说明选择相似度度量，实际上是在选择是否把表示长度当作信息。

第 6 章 通过预测学习词向量

计数矩阵先记录所有共现，再进行加权与降维。另一条路线直接问：如果一个向量能帮助预测一个词的上下文，它是否已经捕捉到了有用的语义？word2vec 把这种想法变成非常简单的训练目标。

这里需要分清三件事：嵌入查表只是参数组织形式；上下文预测才规定向量要保存什么；负采样则改变了训练目标和计算代价。理解这三层之后，就不容易把某种具体软件实现误当成词向量的定义。

6.1 查表、共享参数与上下文预测

设词表大小为 V ，嵌入维度为 d 。输入嵌入矩阵 $E \in \mathbb{R}^{V \times d}$ 的第 w 行是词向量 e_w 。若 one-hot 向量为 o_w ，则

$$e_w = E^\top o_w. \quad (6.1)$$

实际实现只查一行，不需要构造长度为 V 的 one-hot。训练过程更新这些行，使它们服务于指定目标。

Skip-gram 使用中心词预测窗口中的上下文词：

$$\mathcal{L}_{\text{SG}} = - \sum_t \sum_{j \in \mathcal{W}_t} \log p(w_j | w_t). \quad (6.2)$$

$\mathcal{W}_t$ 是排除中心位置的上下文索引集合。模型通常为输入词和输出上下文各维护一张表 E, U ：

$$p(c | w) = \frac{\exp(u_c^\top e_w)}{\sum_{v \in \mathcal{V}} \exp(u_v^\top e_w)}. \quad (6.3)$$

同一词拥有两种角色，因此出现两组向量并不矛盾。训练后可使用输入表、输出表或两者组合，但需要在下游验证选择。

CBOW 反过来，聚合上下文预测中心词：

$$h_t = \frac{1}{|\mathcal{W}_t|} \sum_{j \in \mathcal{W}_t} e_{w_j}, \quad p(w_t | \mathcal{W}_t) = \text{softmax}(U h_t)_{w_t}. \quad (6.4)$$

平均操作不保留窗口内部的词序；将位置特定表示拼接，则会得到不同模型。

数学直觉 | 不是“相邻就一定靠近”

Skip-gram 直接提高的是目标词向量与上下文向量的点积。两个目标词因为需要预测相似的上下文，才间接获得相似表示。直接相邻的词与语义可替换的词，其关系不完全相同。

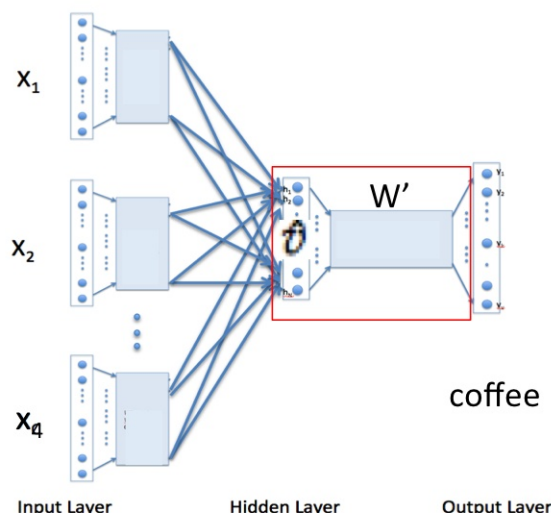


图 6.1 CBOW 汇集上下文表示，再预测中心词。训练误差会回传到参与平均的各个词向量。（课件 lec5_dist, 第 73 页）

6.2 softmax 的梯度与计算瓶颈

对一个正配对 (w, c) ，损失为

$$\ell = -u_c^\top e_w + \log \sum_v e^{u_v^\top e_w}. \quad (6.5)$$

对中心词向量求导：

$$\frac{\partial \ell}{\partial e_w} = \sum_v p(v | w) u_v - u_c. \quad (6.6)$$

梯度下降将 e_w 朝真实上下文向量拉近，并相对远离当前预测分布的平均上下文。对输出向量有

$$\frac{\partial \ell}{\partial u_v} = (p(v | w) - \mathbf{1}[v = c]) e_w. \quad (6.7)$$

一次更新原则上涉及全部输出词，计算量为 $O(Vd)$ 。大词表下，这比查输入嵌入本身昂贵得多。

层次 softmax 把词放在二叉树叶子上，预测一个词等价于沿根到叶的路径进行二元决策。若路径上的分支标签为 b_j 、参数为 a_j ，

$$p(w | h) = \prod_{j \in \text{path}(w)} \text{sigmoid}((2b_j - 1)a_j^\top h). \quad (6.8)$$

平衡树的路径长度为 $O(\log V)$ ；Huffman 树可使高频词路径较短。它仍然定义完整归一化分布，但树结构会影响参数共享方式。

训练样本从哪里来

对句子“我喜欢自然语言处理”，窗口半径为 1 时，中心词“语言”的正配对为（语言，自然）与（语言，处理）。半径为 2 时再加入“喜欢”；不能跨过句子边界随意构造配对，否则会加入并不存在的局部关系。

频繁词可下采样，减少“的”“是”等词占据训练步骤的比例。这样做不仅加速，还改变了有效的共现分布。动态窗口、词频阈值和子采样规则，都会影响最终向量。

6.3 负采样：把多类预测改为二元识别

负采样不对完整词表归一化，而是区分真实共现配对与噪声配对。对正配对 (w, c) ，从噪声分布 q 中抽取 k 个上下文词 $n_1, \dots, n_k$ ：

$$\ell_{\text{NS}} = -\log \text{sigmoid}(u_c^\top e_w) - \sum_{j=1}^k \log \text{sigmoid}(-u_{n_j}^\top e_w). \quad (6.9)$$

正配对希望点积大，负配对希望点积小，计算降为 $O(kd)$ 。**负采样优化的是二元判别目标，并不直接给出归一化的 $p(c | w)$ 。**不能把这些 sigmoid 分数直接当成标准语言模型概率来计算困惑度。

常见噪声分布为 $q(c) \propto C(c)^{3/4}$ ，比原始词频更平坦，使中低频词也有机会成为负例。这里的指数是经验设计，不是概率论要求。

一个配对的更新方向

设 $e_w = (1, 0)^\top$ 、正上下文 $u_c = (0, 1)^\top$ 、负上下文 $u_n = (1, 0)^\top$ 。两个点积分别为 0 和 1，因此

$$\nabla_{e_w} \ell = (\text{sigmoid}(0) - 1)u_c + \text{sigmoid}(1)u_n \approx (0.731, -0.5)^\top. \quad (6.10)$$

梯度下降会减少第一维、增加第二维，即削弱与负例的对齐、增强与正例的对齐。

算法实现 | 负采样的一步核心计算

```
# PyTorch-like pseudocode; IDs have already been sampled.
v = input_embedding(center_ids)          # [B, d]
u_pos = output_embedding(positive_ids) # [B, d]
u_neg = output_embedding(negative_ids) # [B, k, d]
pos = (v * u_pos).sum(-1)
neg = (v[:, None, :] * u_neg).sum(-1)
loss = -logsigmoid(pos).mean()
loss -= logsigmoid(-neg).sum(-1).mean()
loss.backward()
optimizer.step()
optimizer.zero_grad()
```

logsigmoid 应使用稳定实现。教学代码省略了批量采样、下采样和词表构建，它们属于数据管线。

6.4 负采样为什么连接 PMI

把一个词与上下文的点积记为 s_{wc} ，暂时假设每个点积都能独立调整，不受低维分解约束。设正配对计数为 C_{wc} ，中心词配对总数为 C_w 。在期望意义下，该配对的负例数为 $kC_w q(c)$ 。最大化局部目标

$$J(s) = C_{wc} \log \text{sigmoid}(s) + kC_w q(c) \log \text{sigmoid}(-s). \quad (6.11)$$

令导数为零：

$$0 = C_{wc}(1 - \text{sigmoid}(s)) - kC_w q(c) \text{sigmoid}(s), \quad (6.12)$$

$$\frac{\text{sigmoid}(s)}{1 - \text{sigmoid}(s)} = \frac{C_{wc}}{kC_w q(c)}. \quad (6.13)$$

由于左边等于 e^s ，得到

$$s_{wc}^* = \log \frac{p(c|w)}{kq(c)}. \quad (6.14)$$

若恰取 $q(c) = p(c)$ ，则

$$s_{wc}^* = \text{PMI}(w, c) - \log k. \quad (6.15)$$

真实模型使用低维点积 $u_c^\top e_w$ ，所有配对的理想值未必能同时满足，所以它近似学习一个移位 PMI 结构。若噪声分布使用 $3/4$ 次幂，右边要按实际 q 计算，不能原样套用最简单的 PMI 等式。

数学直觉 | 计数与预测是两种实现路径

矩阵分解从显式共现统计出发，负采样从随机抽取的配对出发。它们可能在相近的统计结构上工作；“预测式向量”并不意味着完全摆脱了共现信息。

子词与未登录词

只存整词向量时，新词没有参数。子词模型把一个词表示为字符 n -gram 向量之和或组合：

$$e_w = \sum_{g \in \mathcal{G}(w)} z_g. \quad (6.16)$$

这样，未出现过的词仍可借助共享片段获得表示，适合形态变化丰富的语言。但相同字形片段不一定共享语义，子词组合也可能把表面相似、意义不同的词拉近。

类比与不变性

“国王减男人加女人接近女王”是某些表示中的经验现象，而非每个语义关系都严格线性。训练目标只约束输入与输出表的配对分数；某些可逆坐标变换会保留这些分数，因此单个坐标的解释并不唯一。

负采样与完整 softmax 的不同概率问题

完整 softmax 在固定中心词下回答“上下文是哪个词”，候选词概率和为 1。负采样的 sigmoid 回答“这个配对更像真实数据还是噪声”，它的两类是数据与噪声。

设真实条件概率为 $p(c|w) = 0.2$ ，噪声概率为 $q(c) = 0.01$ ，每正例配 5 个负例。理想点积为 $s = \log 4$ ，因此 $\text{sigmoid}(s) = 0.8$ 。这个 0.8 是数据对噪声的判别概率，并不是原本的上下文概率 0.2。

要从理想分数恢复条件概率，还必须乘回噪声分布与负例比例：

$$p(c|w) = kq(c)e^{s_{wc}}. \quad (6.17)$$

真实低维模型未必精确达到理想分数，因此恢复后还可能需归一化。训练目标的概率含义必须与输出解释一致。

输入表与输出表的几何不唯一

将所有输入向量排成矩阵 E ，输出向量排成 U ，配对分数矩阵为 $EU^\top$ 。对任意可逆矩阵 R ，令

$$E' = ER, \quad U' = UR^{-\top}. \quad (6.18)$$

则 $E'U'^\top = EU^\top$ ，所有配对分数不变。但若 R 不是正交矩阵，输入向量之间的余弦可能改变。

这说明只拟合输入输出点积，并不唯一确定输入向量内部的全部几何。正则化、初始化、优化过程和最终选用哪张表，都会影响下游相似度。不能认为训练损失相同就必然得到完全相同的最近邻列表。

GloVe 与共现概率比

GloVe 也是从全局共现出发学习向量。一个常见目标是

$$J = \sum_{i,j} f(C_{ij}) \left(e_i^\top u_j + b_i + \tilde{b}_j - \log C_{ij} \right)^2, \quad (6.19)$$

通常只遍历非零共现。权重函数 f 抑制极低频噪声，并控制高频项的影响。偏置可以吸收词与上下文的总体频率。

其动机之一，是比较两个目标词对不同上下文的相对偏好。例如 `ice` 与 `steam` 对 `solid` 和 `gas` 的条件概率比具有区分意义。若两者都同样容易出现在 `water` 周围，这个上下文的比值接近 1，区分力较小。

这与朴素贝叶斯的似然比、PMI 的独立基线比较，以及负采样的数据噪声比较相呼应：**很多表示学习目标真正利用的是相对关系，而不是孤立的高频计数。**

字符片段怎样帮助新词

设词向量由若干子词向量求和。某个未见词含三个已知片段，向量分别为 $(1, 0)$ 、 $(0, 2)$ 、 $(-0.5, 0.5)$ ，则组合向量为 $(0.5, 2.5)$ 。即使整词没有专属参数，模型仍能得到一个可用于下游的表示。

不过，如果新词的含义无法由字面片段推断，组合也可能错误。例如专名、缩写和历史形成的固定表达，不一定能通过局部字形恢复意义。子词主要改善覆盖和共享，并没有消除歧义。

如何设计一次公平的词向量比较

固定语料、分词、词表、维度和训练 token 预算，再比较计数分解与预测模型。若一个模型使用更大窗口、更多语料或额外词典，其提升不能只归于目标函数。

最近邻和类比适合快速检查，但最终应把向量放回目标任务。还应单独查看低频词与未登录词，因为总体指标容易被高频词主导。一个方法在罕见实体上更稳健，可能比平均相似度多几个点更有实际价值。

推演与自检

若正配对概率为 $p(c | w) = 0.2$ 、噪声概率 $q(c) = 0.01$ 、每个正例配 $k = 5$ 个负例，理想点积为 $\log 4$ 。若把负例数加倍为 10，理想点积降为 $\log 2$ 。这不是关系变弱，而是判别任务中的正负先验改变了。

第 7 章 神经语言模型与循环序列网络

词向量提供了连续表示，但还没有回答如何表示一整段历史。固定窗口把最近几个词拼起来，卷积提取局部模式，循环网络则不断更新记忆状态。这些选择代表了不同的信息压缩方式。

本章从前馈语言模型走到 LSTM。重点不是背出门的名称，而是看清每个状态承载什么信息、梯度怎样通过时间传播，以及训练和生成时能看到哪些词。

7.1 前馈语言模型与参数共享

设窗口包含前 m 个 token，嵌入维度为 d ，将它们拼接：

$$x_t = [e_{w_{t-m}}; \cdots; e_{w_{t-1}}] \in \mathbb{R}^{md}. \quad (7.1)$$

经过隐藏层与输出层：

$$h_t = \tanh(Wx_t + b), \quad p(w_t | w_{t-m:t-1}) = \text{softmax}(Uh_t + a). \quad (7.2)$$

$W \in \mathbb{R}^{H \times md}$ 、 $U \in \mathbb{R}^{V \times H}$ 。与计数表相比，同一个嵌入和隐藏层参数被多个上下文共享；相似历史因而可以产生相似的预测。

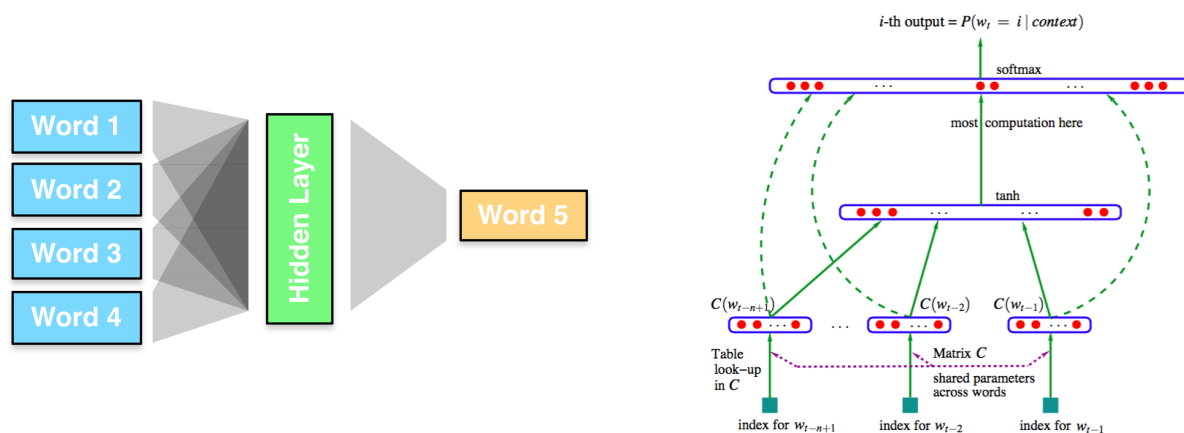


图 7.1 左：固定窗口的条件输入；右：嵌入、隐藏层与词表输出。（课件 lec4_nlm，第 22、19 页）

词向量不必预先训练。随机初始化 E 后，语言模型的交叉熵就能同时更新嵌入与其他参数。使用预训练向量只是另一种初始化或迁移策略。

如果 $V = 10000, d = 64, m = 3, H = 128$ ，输入嵌入有 64 万参数，隐藏层权重约 2.46 万，输出层约 128 万。输出词表层常占相当大比例。输入与输出维度兼容时，可以共享嵌入与输出权重，减少参数并约束表示。

非线性做了什么

若去掉隐藏层非线性， $Uh = UWx + Ub$ 仍是输入的线性函数，多层线性变换可合并成一层。非线性让模型表达特征之间的条件组合，例如“not”对不同评价词的影响不能总用固定相加解释。

前馈模型仍只看固定窗口。窗口外的信息完全不可见，增加隐藏层宽度无法改变这个事实。为了让远处证据参与预测，需要更合适的序列结构。

7.2 卷积与循环：两种组织上下文的方法

一维因果卷积在位置 t 使用当前位置及过去输入：

$$h_t = \rho \left(b + \sum_{j=0}^{k-1} W_j x_{t-j} \right). \quad (7.3)$$

用于下一词预测时，输入与目标还需整体错开一位。卷积核在所有位置共享，能识别位置无关的局部模式。堆叠 L 层、核宽 k 、步幅 1 且无扩张时，感受野长度为 $1 + L(k - 1)$ ；扩张卷积可以更快扩大感受野。

文本分类可以对整句卷积特征做最大或平均池化；自回归语言建模则不能把未来位置池化进当前预测。同一种层是否泄漏未来信息，取决于它的连接、掩码与目标对齐方式。

RNN 使用隐藏状态递推：

$$h_t = \tanh(W_x e_{w_t} + W_h h_{t-1} + b), \quad p(w_{t+1} | w_{1:t}) = \text{softmax}(U h_t + a). \quad (7.4)$$

状态维度固定，但可以承接任意长度的历史。注意当前 h_t 已读入 w_t ，所以它预测的是 w_{t+1} ；若用它预测 w_t ，可能不小心把答案作为输入。

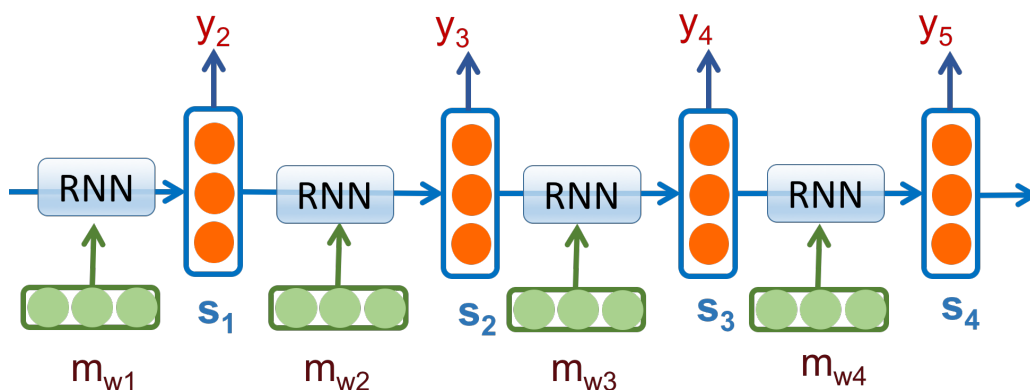


图 7.2 沿时间展开的循环网络在每一步复用同一组参数。（课件 lec4_nlm，第 45 页）

数学直觉 | 固定状态是可学习的摘要

RNN 并不把所有历史原样储存下来，而是不断决定哪些信息进入一个有限维摘要。长距离依赖困难，不只因为句子长，还因为重要信息必须在多次状态更新中存活。

双向网络与条件范围

双向 RNN 分别计算 $\vec{h}_t$ 与 $\overleftarrow{h}_t$ ，再拼接为 $h_t = [\vec{h}_t; \overleftarrow{h}_t]$ 。它适合已知完整输入的标注、分类和编码任务；如果直接用它预测同一序列的下一个词，反向分支可能看到未来答案。双向性不是好坏标签，而是是否符合任务允许的信息条件。

7.3 梯度消失、爆炸与门控记忆

展开 RNN 后，早期状态对后期损失的影响包含 Jacobian 连乘：

$$\frac{\partial h_t}{\partial h_s} = \prod_{j=s+1}^t \frac{\partial h_j}{\partial h_{j-1}}, \quad \frac{\partial h_j}{\partial h_{j-1}} = \text{diag}(1 - h_j^2)W_h. \quad (7.5)$$

若沿某方向的有效增益长期小于 1，梯度指数衰减；长期大于 1 则可能爆炸。非线性饱和还会让导数接近零。因此，依赖链越长，不代表可学习的有效记忆就越长。

梯度裁剪可限制爆炸。例如整体范数裁剪

$$g \leftarrow g \min\left(1, \frac{c}{\|g\|_2}\right) \quad (7.6)$$

保留梯度方向并限制步幅。它不能根本解决梯度消失。截断反向传播限制每次向过去回传的长度，节省内存，但也改变长程依赖的训练信号。

LSTM 加入单独的细胞状态 c_t 。给定拼接向量 $z_t = [h_{t-1}; x_t]$ ：

$$f_t = \text{sigmoid}(W_f z_t + b_f), \quad i_t = \text{sigmoid}(W_i z_t + b_i), \quad (7.7)$$

$$o_t = \text{sigmoid}(W_o z_t + b_o), \quad g_t = \tanh(W_g z_t + b_g), \quad (7.8)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \quad h_t = o_t \odot \tanh(c_t). \quad (7.9)$$

遗忘门 f_t 控制保留多少旧状态，输入门 i_t 控制写入多少候选内容，输出门 o_t 控制展示多少内部记忆。

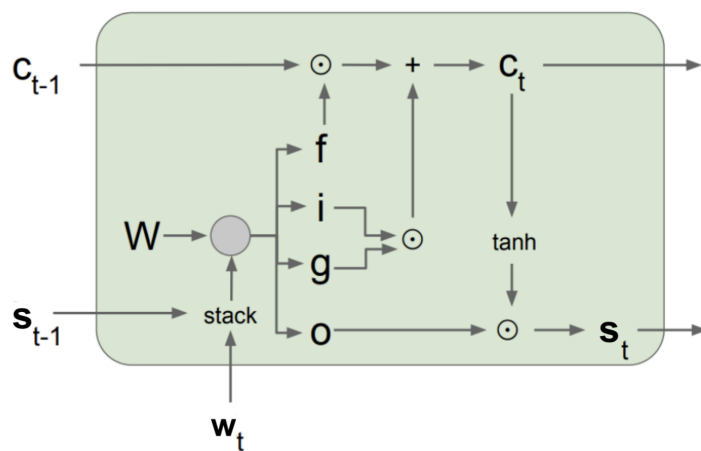


图 7.3 LSTM 通过加性更新保留细胞状态，并用门控制写入、遗忘与读出。（课件 lec6_seq，第 10 页）

只看直接的细胞状态通路、暂时固定门值，有

$$\frac{\partial c_t}{\partial c_{t-1}} = \text{diag}(f_t). \quad (7.10)$$

当 f_t 接近 1，旧信息有近似恒等的传播路径。完整总导数还包含门对先前状态的依赖，所以不能把这条局部通路误写成完整梯度的全部。

手算一个记忆单元

取旧状态 $c_{t-1} = 2$ ，遗忘门 $f_t = 0.9$ ，输入门 $i_t = 0.2$ ，候选内容 $g_t = -0.5$ ，则

$$c_t = 0.9 \times 2 + 0.2 \times (-0.5) = 1.7. \quad (7.11)$$

若输出 $o_t = 0.8$ ，隐藏输出为 $h_t = 0.8 \tanh(1.7) \approx 0.748$ 。内部记忆与对外输出不同：网络可以保留某条信息，却暂时不把它暴露给预测层。

GRU 将记忆与隐藏状态合并。一种常用约定为

$$z_t = \text{sigmoid}(W_z x_t + U_z h_{t-1}), \quad (7.12)$$

$$r_t = \text{sigmoid}(W_r x_t + U_r h_{t-1}), \quad (7.13)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1})), \quad (7.14)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \quad (7.15)$$

有些资料交换 z_t 与 $1 - z_t$ 的角色；阅读时应看公式，不能只凭“更新门”名称判断其数值意义。

7.4 训练、生成与序列标注

训练自回归模型时，通常把真实历史送入每一步，称为 teacher forcing。整句损失为

$$\mathcal{L} = - \sum_{t=1}^T \log p_{\theta}(w_t \mid w_{<t}). \quad (7.16)$$

真实历史已知，因此能批量准备输入与目标；RNN 的状态计算仍沿时间递归。生成时，模型从自己先前采样的 token 继续，输入历史的分布会发生变化。

算法实现 | RNN 语言模型训练的最小骨架

```
# ids: [B, T+1], with boundaries and padding already handled
inp, target = ids[:, :-1], ids[:, 1:]
emb = embedding(inp) # [B, T, d]
hidden, state = rnn(emb)
logits = output_layer(hidden) # [B, T, V]
loss = cross_entropy(logits.reshape(-1, V),
                      target.reshape(-1),
                      ignore_index=pad_id)

loss.backward()
clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
optimizer.zero_grad()
```

跨小段延续隐藏状态时，应明确什么时候对状态 `detach`，否则计算图可能无限增长。不同独立文档之间通常应重置状态。

对序列标注，输入整句 $x_{1:T}$ ，每个位置输出标签：

$$p(y_t \mid x) = \text{softmax}(W h_t + b). \quad (7.17)$$

若每个位置独立解码，可能产生非法标签转移。例如 BIO 标注中，在 O 后直接出现 I-PER。神经编码器改善每个位置的特征，并不会自动保证输出结构合法。下一章用 HMM 和 CRF 显式描述标签之间的依赖。

批处理中的长度问题

padding 使不同长度序列能够堆叠，但填充位置不应进入损失。双向 RNN 尤其需要使用真实长度或打包序列，否则反向分支会先读入一串填充。平均池化也应除以有效 token 数，而不是统一的最大长度。

手算梯度传播：为什么远处信号会变小

先去掉非线性，考虑标量递推 $h_t = ah_{t-1} + x_t$ 。连续展开三步：

$$h_1 = ah_0 + x_1, \quad (7.18)$$

$$h_2 = a^2h_0 + ax_1 + x_2, \quad (7.19)$$

$$h_3 = a^3h_0 + a^2x_1 + ax_2 + x_3. \quad (7.20)$$

所以 $\partial h_t / \partial h_0 = a^t$ 。若 $a = 0.5$ ，经过 10 步后只剩 $1/1024$ ；若 $a = 1.5$ ，经过 10 步约为 57.7。这已经展示消失与爆炸，不需要复杂网络才能出现。

加入 tanh 后，每一步还要乘以不大于 1 的局部导数。状态接近饱和时，导数更小。矩阵 RNN 则在不同方向上有不同增益，不能只用一个标量完整描述，但连乘问题仍然存在。

遗忘门对应多长的记忆

暂时令输入门为零、遗忘门固定为 f ，则 LSTM 细胞状态满足 $c_t = f^t c_0$ 。如果希望记忆保留到初始值的一半，半衰期 H 满足

$$f^H = \frac{1}{2}, \quad H = \frac{\log(1/2)}{\log f}. \quad (7.21)$$

$f = 0.9$ 时 H 约为 6.58 步； $f = 0.99$ 时约为 68.97 步。门值只是从 0.9 变成 0.99，记忆时间却相差约十倍。

这说明门的作用不能只用“大一点保留多一点”概括。接近 1 的区域对长期保留很敏感。实际门值随输入变化，因此模型可以在需要时保留，在句子转折或新事件到来时主动更新。

卷积感受野的逐层推导

核宽为 3、步幅为 1 的因果卷积，第一层位置 t 读取 $t, t-1, t-2$ 。第二层又读取上一层三个位置，因此最远到达原输入 $t-4$ ，感受野为 5。第三层扩到 7。

若第 ℓ 层扩张率为 r_ℓ ，感受野长度为

$$R = 1 + \sum_{\ell=1}^L (k_\ell - 1)r_\ell \quad (7.22)$$

(这里各层步幅为 1)。核宽都为 3、扩张率为 1, 2, 4 时， $R = 1 + 2(1 + 2 + 4) = 15$ 。扩大感受野可以减少层数，但不同位置之间的信息路径仍与注意力不同。

teacher forcing 的分布差异

训练时第三步条件是正确前缀 y_1^*, y_2^* 。生成时若第二步出错，第三步读到的是 $\hat{y}_1, \hat{y}_2$ 。模型可能进入训练中较少出现的前缀，从而继续犯错。

这个差异常被称为暴露偏差。简单把真实前缀随机换成模型预测，并不自动得到一个统计上无偏或更好的目标；还要考虑训练目标是否一致、错误样本怎样标注，以及任务指标怎样变化。

对某些任务，改进训练数据、使用更合适的搜索或引入序列级目标会有帮助。判断方法时，应观察错误是否确实来自前缀偏移，而不是把所有生成失败都归于暴露偏差。

序列中的状态重置

假设一个批次里的两段文本互不相关。如果第二段继承第一段的隐藏状态，模型便把第一段当作上下文。训练和测试若使用不同重置规则，损失可能不可比较。

长文档切成多个训练段时则相反：可以保留隐藏状态，让信息跨段传递，但在计算图上截断反向传播。这表示“保留数值状态，但不把梯度无限传回所有旧段”。数值依赖与梯度依赖是两件事。

推演与自检

如果将输入目标都设为同一序列，而网络在每个位置已经读入当前 token，它可以通过复制输入获得很低损失。检查训练代码时，先用极短序列手工列出每个位置能看到什么、要预测什么，通常比直接观察总损失更容易发现泄漏。

第 8 章 序列标注、HMM 与条件随机场

“南京市长江大桥”可以对应不同的切分假设，实体识别又需要同时确定名称的边界与类型。逐位置分类虽然简单，但标签之间通常存在约束：实体内部标签应延续同一实体，词性组合也有规律。

本章的关键，是把指数多个标签序列压缩进一个动态规划表。HMM 与 CRF 使用不同的概率模型，却共享前向求和与 Viterbi 最大化的算法骨架。

8.1 标签体系与隐马尔可夫假设

给定观测 $x_{1:T}$ ，寻找标签 $y_{1:T}$ 。词性标注的标签是词类；BIO 实体标注用 B 表示实体开始、I 表示实体内部、O 表示实体之外；中文分词可用 B、M、E、S 分别表示词首、词中、词尾和单字词。

例如“北京大学位于北京”的字符级实体标注，可以把前四字标为 B-ORG、I-ORG、I-ORG、I-ORG，后两字标为 B-LOC、I-LOC，中间为 O。词表和标注规范必须同时说明，不能只说“使用 BIO”而不定义非法转移怎样处理。

一阶 HMM 包含初始分布 π_j 、转移概率 A_{ij} 与发射概率 $B_j(x)$ ：

$$p(x_{1:T}, y_{1:T}) = \pi_{y_1} B_{y_1}(x_1) \prod_{t=2}^T A_{y_{t-1}, y_t} B_{y_t}(x_t). \quad (8.1)$$

给定长度的写法暂时省略结束状态；需要显式建模终止时，可在最后乘 $A_{y_T, \text{END}}$ ，同时统一转移归一化约定。

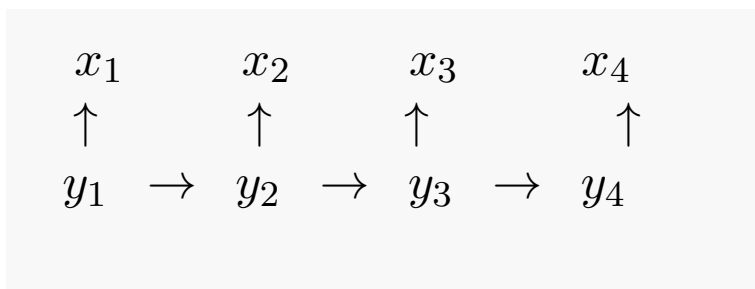


图 8.1 隐标签沿链转移，每个观测由当前位置的隐状态发射。（课件 lec6_hmm，第 24 页）

两个主要假设是 $p(y_t | y_{<t}) = p(y_t | y_{t-1})$ ，以及 $p(x_t | y_{1:T}, x_{\neq t}) = p(x_t | y_t)$ 。后一假设较强：给定当前位置标签后，当前词的分布不再依赖其他词。

标注数据可直接估计参数：

$$\hat{A}_{ij} = \frac{C(i, j)}{\sum_k C(i, k)}, \quad \hat{B}_j(w) = \frac{C(j, w)}{\sum_v C(j, v)}. \quad (8.2)$$

实际要平滑，并为未知词设计字形、后缀或 <unk> 处理。二阶 HMM 用 $p(y_t | y_{t-2}, y_{t-1})$ ，其动态规划状态也要记住两个标签。

8.2 Viterbi: 保留对未来有用的历史

直接枚举共有 K^T 个标签序列。观察一阶模型：两条路径如果都在时刻 t 结束于标签 j ，它们未来的延续分布相同。因此，对于找最佳路径，只需保留其中分数较高的一条。

定义

$$\delta_t(j) = \max_{y_{1:t-1}} \log p(x_{1:t}, y_{1:t-1}, y_t = j). \quad (8.3)$$

初始化与递推为

$$\delta_1(j) = \log \pi_j + \log B_j(x_1), \quad (8.4)$$

$$\delta_t(j) = \log B_j(x_t) + \max_i [\delta_{t-1}(i) + \log A_{ij}], \quad (8.5)$$

$$\psi_t(j) = \arg \max_i [\delta_{t-1}(i) + \log A_{ij}]. \quad (8.6)$$

ψ 保存最优前驱。最后选 $y_T^* = \arg \max_j \delta_T(j)$ ，再按 $y_{t-1}^* = \psi_t(y_t^*)$ 回溯。时间复杂度为 $O(TK^2)$ ；只算最佳分数可滚动使用 $O(K)$ 内存，恢复路径通常还要 $O(TK)$ 回溯表。

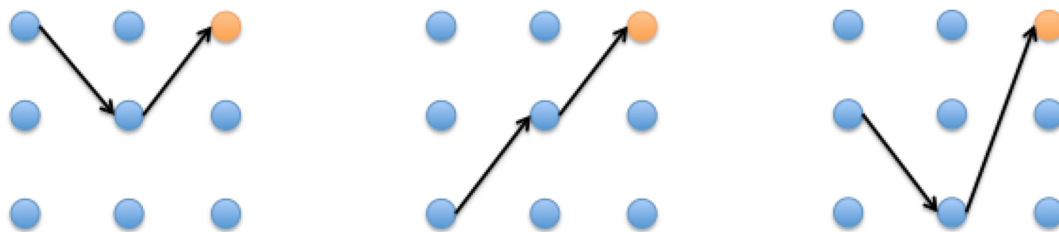


图 8.2 同一终点的多条前缀只保留最佳者，但不同终点的候选必须同时保留。（课件 lec6_hmm，第 30 页）

数学直觉 | 动态规划为什么敢丢掉路径

不是因为低分前缀永远不可能反超，而是因为比较的两条前缀具有相同的未来接口。任何后缀给它们增加的分数都一样，较差者才可以安全丢弃。状态设计必须保留影响未来的全部信息。

两状态、两观测的手算

状态为 N、V，初始分布 (0.6, 0.4)，转移矩阵为

$$A = \begin{pmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{pmatrix}. \quad (8.7)$$

观测为 fish、swim。发射概率分别满足 $B_N(\text{fish}) = 0.5$ 、 $B_V(\text{fish}) = 0.1$ ， $B_N(\text{swim}) = 0.1$ 、 $B_V(\text{swim}) = 0.6$ 。其余发射概率质量留给其他词。

第一步的概率域分数是 (0.3, 0.04)。第二步结束于 N：

$$\max(0.3 \times 0.7, 0.04 \times 0.4) \times 0.1 = 0.021. \quad (8.8)$$

结束于 V：

$$\max(0.3 \times 0.3, 0.04 \times 0.6) \times 0.6 = 0.054. \quad (8.9)$$

最佳终点为 V，其前驱为 N，因此最佳序列为 N,V。逐位置只选最大发射概率在这个例子里碰巧相同，但改变转移概率就可能不一致。

算法实现 | 对数域 Viterbi

```
import numpy as np

def viterbi(log_start, log_trans, emit):
    # emit[t, j] = log B_j(x_t)
    T, K = emit.shape
    dp = log_start + emit[0]
    back = np.zeros((T, K), dtype=int)
    for t in range(1, T):
        scores = dp[:, None] + log_trans # previous x next
        back[t] = scores.argmax(axis=0)
        dp = scores.max(axis=0) + emit[t]
    path = [int(dp.argmax())]
    for t in range(T-1, 0, -1):
        path.append(int(back[t, path[-1]]))
    return path[::-1], float(dp.max())
```

非法转移用 $-\infty$ 屏蔽；若所有路径都非法，应显式报告无解，不能把 `argmax` 返回的零号标签当作有效答案。

8.3 前向后向：对所有解释求和

Viterbi 得到最佳路径，却没有给出观测序列概率或每个标签的后验。定义前向量

$$\alpha_t(j) = p(x_{1:t}, y_t = j). \quad (8.10)$$

把 Viterbi 的最大值换成求和：

$$\alpha_1(j) = \pi_j B_j(x_1), \quad (8.11)$$

$$\alpha_t(j) = B_j(x_t) \sum_i \alpha_{t-1}(i) A_{ij}, \quad (8.12)$$

$$p(x_{1:T}) = \sum_j \alpha_T(j). \quad (8.13)$$

在上例中， $\alpha_2(N) = 0.0226$ 、 $\alpha_2(V) = 0.0684$ ，总概率为 0.091，大于最佳路径概率 0.054，因为它包含四条路径的概率之和。

后向量定义为 $\beta_t(i) = p(x_{t+1:T} | y_t = i)$ ，边界 $\beta_T(i) = 1$ ，递推为

$$\beta_t(i) = \sum_j A_{ij} B_j(x_{t+1}) \beta_{t+1}(j). \quad (8.14)$$

由此前后信息相乘得到标签后验：

$$\gamma_t(j) = p(y_t = j | x) = \frac{\alpha_t(j) \beta_t(j)}{p(x)}. \quad (8.15)$$

相邻状态后验为

$$\xi_t(i, j) = \frac{\alpha_t(i) A_{ij} B_j(x_{t+1}) \beta_{t+1}(j)}{p(x)}. \quad (8.16)$$

它们也是无监督 HMM 训练中 EM 的软计数。更新时，用 $\sum_t \xi_t(i, j)$ 代替转移计数，用 $\sum_t \gamma_t(j) \mathbf{1}[x_t = w]$ 代替发射计数，再归一化。

长序列的概率会下溢。可以每一步缩放 α 并记录缩放因子，或在数域用 log-sum-exp。前向后向算法的数学复杂度不变，数值计算却会可靠得多。

备注 | 最佳路径不等于边缘最优标签拼接

逐位置选择 $\arg \max_j \gamma_t(j)$ 最小化期望逐位置零一损失，Viterbi 最大化整条路径概率。独立选择的标签甚至可能违反结构约束。两种决策优化的目标不同，不能互相替代。

8.4 CRF：保留链结构，放宽观测假设

线性链 CRF 不再建模输入如何生成，而是直接给标签序列打分。令 $e_t(j)$ 为位置 t 对标签 j 的发射分数， A_{ij} 为转移分数：

$$S_\theta(x, y) = b_{y_1}^{\text{start}} + \sum_{t=1}^T e_t(y_t) + \sum_{t=2}^T A_{y_{t-1}, y_t} + b_{y_T}^{\text{end}}. \quad (8.17)$$

这些都是任意实数分数，不要求每行归一化。条件分布为

$$p_\theta(y | x) = \frac{e^{S_\theta(x, y)}}{\sum_{y' \in \mathcal{Y}(x)} e^{S_\theta(x, y')}}. \quad (8.18)$$

e_t 可以来自手工特征、BiLSTM 或 Transformer，因此能依赖整句输入。CRF 只对输出的局部因子作结构假设，不要求观测词条件独立。

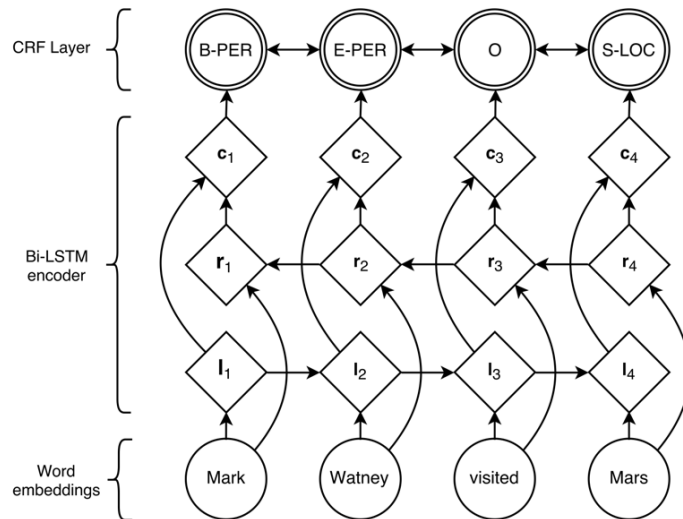


图 8.3 编码器负责上下文表示，CRF 负责标签间的联合评分。两层解决不同问题。（课件 lec6_seq，第 21 页）

负对数似然是 $\mathcal{L} = \log Z(x) - S(x, y^*)$ 。对某个转移分数求导，有

$$\frac{\partial \mathcal{L}}{\partial A_{ij}} = \mathbb{E}_{p_\theta(y|x)}[C_{ij}(y)] - C_{ij}(y^*). \quad (8.19)$$

这与对数线性模型的“模型期望减真实计数”完全一致。前向后向提供期望计数，Viterbi 提供最佳路径。

全局归一化与标签偏置

局部归一化模型在每个前一状态下分别给后继标签分配概率。若一个状态只有一个可行后继，它的后继概率自动为 1，无法通过本地归一化充分表达“整条路径其实很差”。全局归一化比较完整序列的总分，允许不同前缀之间竞争。这个区别常称为标签偏置问题的背景，但并不意味着任意 CRF 都必然胜过任意局部模型。

实体级评价

实体识别通常按完整跨度与类型匹配计 TP，而不是按每个字符标签是否正确计分。假设真实实体为“北京大学”，预测为“北京”，即使前两个字符标签正确，也不能算一个完整实体正确。实体级精确率、召回率和 F_1 更接近最终抽取需求。

完整算例：把前向、后向与路径概率放在一起

继续使用 fish、swim 的两状态 HMM。四条路径的联合概率为 四项相加为 0.091，最大项为

路径	因子乘积	概率
N,N	$0.6 \times 0.5 \times 0.7 \times 0.1$	0.0210
N,V	$0.6 \times 0.5 \times 0.3 \times 0.6$	0.0540
V,N	$0.4 \times 0.1 \times 0.4 \times 0.1$	0.0016
V,V	$0.4 \times 0.1 \times 0.6 \times 0.6$	0.0144

表 8.1 枚举结果用于核对动态规划。真实长序列不能这样枚举。

0.054。因此最佳路径的后验概率为 $0.054/0.091 \approx 0.593$ ，并没有接近 1。最佳分析与“十分确定”不是同义词。

后向计算从 $\beta_2(N) = \beta_2(V) = 1$ 开始：

$$\beta_1(N) = 0.7 \times 0.1 + 0.3 \times 0.6 = 0.25, \quad (8.20)$$

$$\beta_1(V) = 0.4 \times 0.1 + 0.6 \times 0.6 = 0.40. \quad (8.21)$$

于是第一个位置为 N 的后验为

$$\gamma_1(N) = \frac{0.3 \times 0.25}{0.091} \approx 0.8242. \quad (8.22)$$

第二个位置为 V 的后验为 $0.0684/0.091 \approx 0.7516$ 。两个位置的边缘置信度都高于整条路径置信度，因为它们分别汇总了多条路径。

后验边缘为什么可能拼出非法序列

考虑两个位置和三个标签 A、B、C，允许的五条路径及概率为 AB:0.21、AC:0.21、BA:0.21、CA:0.21、BC:0.16。AA 被结构约束禁止。

第一个位置的边缘概率为 A:0.42、B:0.37、C:0.21；第二个位置为 A:0.42、B:0.21、C:0.37。逐位置选最大值会得到 AA，恰好是非法路径。

并不是边缘概率计算错了。它们分别汇总不同路径，两个局部最优标签不一定曾在同一条合法路径中同时出现。若必须输出合法序列，应在合法集合中优化相应决策目标。

CRF 的配分函数怎样反向提供边缘概率

考虑一个标签位置的发射分数 $e_t(j)$ 。对数配分函数为

$$\log Z(x) = \log \sum_y e^{S(x,y)}. \quad (8.23)$$

由于 $\partial S(x,y)/\partial e_t(j) = \mathbf{1}[y_t = j]$ ，有

$$\frac{\partial \log Z(x)}{\partial e_t(j)} = \sum_y p(y | x) \mathbf{1}[y_t = j] = p(y_t = j | x). \quad (8.24)$$

因此，前向算法不仅算一个归一化常数；对其计算图求导还能得到边缘量。这是自动微分与动态规划相结合的一个重要例子。

二阶标签依赖需要什么状态

若转移分数依赖 (y_{t-2}, y_{t-1}, y_t) ，只记最后一个标签已不够。定义状态 $(u, v) = (y_{t-1}, y_t)$ ，递推为

$$D_t(u, v) = e_t(v) + \max_z [D_{t-1}(z, u) + A_{zuv}]. \quad (8.25)$$

状态数为 K^2 ，每个状态枚举 K 个前驱，所以时间增为 $O(TK^3)$ 。模型表达能力增加，推断成本也增加。

这说明动态规划不是固定的一段循环。它由模型的依赖结构决定：改变条件依赖，就要重新确认状态是否保留了足够信息。

推演与自检

把一个 CRF 的所有标签发射分数在同一位置都加常数 c ，每条合法路径总分都增加 c ，归一化后的概率不变。若只给 B-ORG 加 c ，则改变了路径之间的相对分数。这与分类 softmax 的平移不变性是同一条规律。

第 9 章 成分句法与 CKY 动态规划

序列标签只描述每个位置属于哪一类，不能直接表达“这个短语包含那个短语”。“我用望远镜看见了那个男孩”中，望远镜可能是观察工具，也可能属于男孩；两种解释使用同样的词，却拥有不同的附着关系。

成分句法把句子表示为嵌套的连续片段。本章介绍如何用文法描述合法结构，以及如何在所有候选树中高效找到最佳分析。CKY 是从序列动态规划走向层级结构动态规划的一次自然推广。

9.1 成分、递归与上下文无关文法

一个成分是能作为结构单位参与更大组合的片段，例如名词短语 NP、动词短语 VP。句法树的叶子为词，内部节点为类别标签，树的层级表示组合顺序。

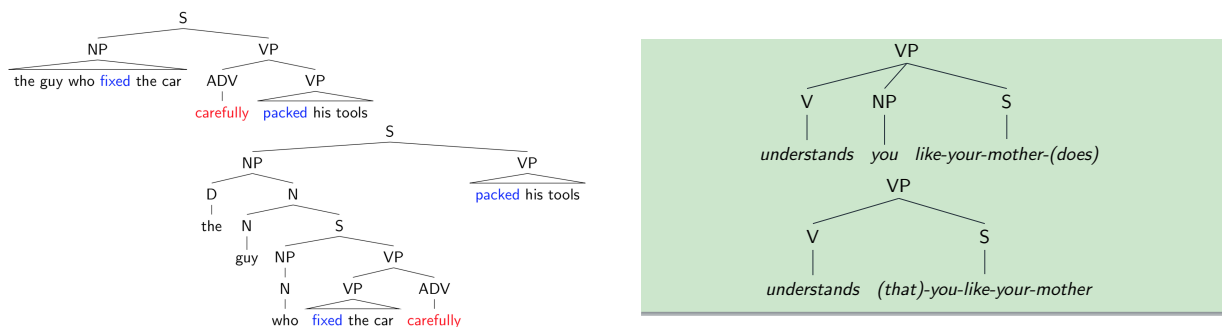


图 9.1 左：嵌套的成分结构；右：同一表达的不同附着解释。（课件 lec7_phrase / lec1_introduction，第 16 / 34 页）

上下文无关文法为四元组 $G = (N, \Sigma, R, S)$ ： N 是非终结符集合， Σ 是终结符集合， R 是规则集合， S 是起始符号。规则形如 $A \rightarrow \beta$ ，其中 $A \in N$ ， $\beta \in (N \cup \Sigma)^*$ 。

“上下文无关”表示规则左侧只有一个非终结符，是否可展开不显式依赖其左右邻居；它不表示句子没有上下文，也不表示模型忽略所有语言环境。

一个极小文法可以写成

$$S \rightarrow NP VP, \quad NP \rightarrow \text{我} \mid \text{鱼}, \quad (9.1)$$

$$VP \rightarrow V NP, \quad V \rightarrow \text{吃}. \quad (9.2)$$

通过 $S \Rightarrow NP VP \Rightarrow \text{我} VP \Rightarrow \text{我} V NP \Rightarrow \text{我吃鱼}$ ，可得到一棵句法树。若加入 $NP \rightarrow NP PP$ 与 $VP \rightarrow VP PP$ ，介词短语便可附着在不同层级，引入结构歧义。

乔姆斯基范式

标准 CKY 假定二叉规则 $A \rightarrow BC$ 与词汇规则 $A \rightarrow w$ 。一般 CFG 可以经过二叉化、单元规则处理等变换转为适当形式；空串需要特殊处理，不能直接说所有细节都被两类规则无条件覆盖。

例如将 $S \rightarrow A B C$ 改为 $S \rightarrow A X$ 、 $X \rightarrow B C$ 。 X 是人工引入的中间符号，最终输出树通常要去掉它。若原规则有概率 p ，一种保留该展开概率的方法是令第一条规则概率为 p 、专用辅助规则概率为 1。

备注 | 语法允许与使用概率

CFG 规定哪些结构可以生成；它本身不会区分两个都合法的分析哪个更合理。文法规则覆盖能力与统计评分能力，是两个独立的问题。

9.2 概率文法与树的评分

PCFG 为每条规则设置条件概率 $p(A \rightarrow \beta)$ ，对每个固定左侧 A ，所有候选右侧概率和为 1。树的概率为各次规则展开概率之积：

$$p(T) = \prod_{r \in \text{rules}(T)} p(r), \quad \log p(T) = \sum_r C_r(T) \log p(r). \quad (9.3)$$

同一条规则在树中重复使用时，应重复计入。给定带标注树的树库，最大似然估计为

$$\hat{p}(A \rightarrow \beta) = \frac{C(A \rightarrow \beta)}{\sum_{\beta'} C(A \rightarrow \beta')}. \quad (9.4)$$

对于句子 x ， $p(x) = \sum_{T: \text{yield}(T)=x} p(T)$ ；最佳解析为 $\arg \max_T p(T)$ ，因为 $p(x)$ 对同一句子的候选树是共同分母。

PCFG 假设某个非终结符的展开只依赖其类别，忽略了父节点、词汇头及更远语境。相同的 NP 在主语和宾语位置可能偏好不同结构，基本 PCFG 不能直接表达这些差异。父节点注释、词汇化与神经评分都是放宽此假设的方式。

局部归一化与终止性

每个非终结符的规则概率和为 1，不自动保证最终生成有限终结字符串的总概率为 1。递归文法可能以正概率无限展开。解析一个固定有限句子时仍可计算其有限树的分数，但作为完整生成模型还需关注一致性条件。

9.3 CKY：按跨度长度填表

用半开区间 $[i, j)$ 表示 $x_i, \dots, x_{j-1}$ ，记

$$D[i, j, A] = \max_{T: A \Rightarrow x_{i:j}} \log p(T). \quad (9.5)$$

此处 $x_{i:j}$ 采用程序式半开切片记法。长度为 1 时初始化

$$D[i, i+1, A] = \log p(A \rightarrow x_i), \quad (9.6)$$

没有对应规则则为 $-\infty$ 。对更长跨度，枚举切分点 k 与二叉规则 $A \rightarrow BC$ ：

$$D[i, j, A] = \max_{\substack{i < k < j \\ A \rightarrow BC}} [\log p(A \rightarrow BC) + D[i, k, B] + D[k, j, C]]. \quad (9.7)$$

先填短跨度，再填长跨度，保证每个子问题已经求解。为恢复树，还需保存实现最大值的规则与切分点。

$S \rightarrow NP VP$ 1.0	$VP \rightarrow VP PP$ 0.5	$VP \rightarrow V NP$ 0.5
$NP \rightarrow Det N$ 0.6	$NP \rightarrow NP PP$ 0.3	$PP \rightarrow P NP$ 1.0
$V \rightarrow \text{saw}$ 1.0	$Det \rightarrow \text{the}$ 0.4	$Det \rightarrow \text{a}$ 0.6
$N \rightarrow \text{boy}$ 0.5	$N \rightarrow \text{telescope}$ 0.5	$P \rightarrow \text{with}$ 1.0
$NP \rightarrow I$ 0.1		

0	1	2	3	4	5	6	7
1							
2							
3							
4							
5							
6							

0	1	2	3	4	5	6	7
(NP, 0.1)				(S, 0.006)			(S)
	(V, 1.0)			(VP, 0.06)			(VP, 0.0054)
			(Det, 0.4)	(NP, 0.12)			(NP, 0.00648)
				(N, 0.5)			
					(P, 1.0)		(PP, 0.18)
						(Det, 0.6)	(NP, 0.18)
							(N, 0.5)

图 9.2 左：初始化后的跨度表；右：合并更长跨度后的结果。同一单元存储某个跨度上不同非终结符的最佳分数。（课件 lec7_phrase, 第 44、73 页）

数学直觉 | 状态是一个可替换的子树接口

两棵子树如果具有相同的根类别 A 、覆盖同一段输入 $[i, j]$ ，上层 PCFG 只看见这个接口。因此在寻找最佳树时，可丢弃其中较差的一棵。词汇化文法若让上层依赖子树的中心词，就必须把中心词也放进状态。

三个词的完整演算

考虑“我吃鱼”，规则为 $S \rightarrow NP VP$ 概率 1， $VP \rightarrow V NP$ 概率 1， $NP \rightarrow \text{我}$ 概率 0.6， $NP \rightarrow \text{鱼}$ 概率 0.4， $V \rightarrow \text{吃}$ 概率 1。

第一层词汇单元为 $D[0, 1, NP] = \log 0.6$ 、 $D[1, 2, V] = 0$ 、 $D[2, 3, NP] = \log 0.4$ 。长度为 2 的 $[1, 3]$ 可用 $VP \rightarrow V NP$ 在 $k = 2$ 合并，得 $D[1, 3, VP] = \log 0.4$ 。

长度为 3 的 $[0, 3]$ 用 $S \rightarrow NP VP$ 在 $k = 1$ 合并，得

$$D[0, 3, S] = \log 1 + \log 0.6 + \log 0.4 = \log 0.24. \quad (9.8)$$

这个例子只有一棵合法树。复杂文法中同一单元会有多个候选，递推才体现出共享子问题的价值。

算法实现 | CKY 的核心循环

```
# score[i,j,A] defaults to -inf; lexical rules initialized.
for width in range(2, n + 1):
    for i in range(n - width + 1):
        j = i + width
        for k in range(i + 1, j):
            for A, B, C, logp in binary_rules:
                value = logp + score[i,k,B] + score[k,j,C]
                if value > score[i,j,A]:
                    score[i,j,A] = value
                    back[i,j,A] = (k, B, C)
# Recursively follow back[(0,n,S)] to recover the tree.
```

按二叉规则枚举时，时间为 $O(n^3|R_2|)$ ，空间约为 $O(n^2|N|)$ 。规则索引、稀疏单元和剪枝可降低实际开销。

9.4 Inside、神经评分与解析评价

把 CKY 的最大化换成求和，得到 inside 概率：

$$I[i, j, A] = \sum_{\substack{i < k < j \\ A \rightarrow BC}} p(A \rightarrow BC) I[i, k, B] I[k, j, C]. \quad (9.9)$$

$I[0, n, S]$ 是句子在文法下的总概率。对数域用 log-sum-exp。结合 outside 值可得到规则使用的后验期望，并用于无树标注时的 EM；其思想与 HMM 前向后向一致。

更一般地，同一张计算图可以在不同运算规则下工作：布尔“或 / 且”判断可解析性，“加 / 乘”计算总概率，“最大 / 加”寻找最佳对数分数。这种统一常用半环语言表述，重要的是看清我们究竟在合并概率质量还是选择一条路径。

神经成分解析器可以直接为跨度和类别评分：

$$S(x, T) = \sum_{(i, j, A) \in T} s_{\theta}(i, j, A; x). \quad (9.10)$$

只要评分仍能按动态规划状态分解，就能利用 CKY 风格解码。编码器可以看整句，动态规划负责保证输出是一棵合法树。复杂的表示学习并没有使结构算法失去价值。

跨度级精确率与召回率

从预测树与真实树中抽取有标签跨度集合，交集为正确成分。设交集大小为 m ，预测跨度数为 p ，真实跨度数为 g ，则

$$P = \frac{m}{p}, \quad R = \frac{m}{g}, \quad F_1 = \frac{2m}{p + g}. \quad (9.11)$$

是否计入根、单词级成分、标点及某些标签，需要固定协议。树的整体完全匹配率更严格：只要一个附着错误，整句就不算正确。

完整算例：望远镜属于谁

考虑句子 I saw boy with telescope。为了只研究附着歧义，使用下列简化 PCFG：

$$S \rightarrow NP VP \quad [1], \quad (9.12)$$

$$VP \rightarrow V NP \quad [0.6], \quad VP \rightarrow VP PP \quad [0.4], \quad (9.13)$$

$$NP \rightarrow I \quad [0.2], \quad NP \rightarrow boy \quad [0.4], \quad (9.14)$$

$$NP \rightarrow telescope \quad [0.1], \quad NP \rightarrow NP PP \quad [0.3], \quad (9.15)$$

$$PP \rightarrow P NP \quad [1], \quad V \rightarrow saw \quad [1], \quad P \rightarrow with \quad [1]. \quad (9.16)$$

每个非终结符的规则概率和都为 1。这不是完整英语文法，只用于展示两种树的竞争。

第一种把 with telescope 接到 VP 上，表示用望远镜看见男孩。第二种把它接到 boy 所在 NP 上，表示看见拿着望远镜的男孩。两种树共享全部词汇概率，只在附着规则上不同。

$$p(T_{VP}) = 0.2 \times 0.4 \times 0.6 \times 0.4 \times 0.1 = 0.00192, \quad (9.17)$$

$$p(T_{NP}) = 0.2 \times 0.6 \times 0.3 \times 0.4 \times 0.1 = 0.00144. \quad (9.18)$$

句子总概率为 0.00336。VP 附着的后验为 4/7，NP 附着为 3/7。模型偏好前者，但两种解释都占有明显概率。

把两种树放回 CKY 表

按零起始半开区间, with telescope 覆盖 $[3, 5)$, 其 PP 概率为 0.1。saw boy 覆盖 $[1, 3)$, 其 VP 概率为 $0.6 \times 1 \times 0.4 = 0.24$ 。

boy with telescope 的 NP 概率为

$$I[2, 5, NP] = 0.3 \times 0.4 \times 0.1 = 0.012. \quad (9.19)$$

对 $[1, 5)$ 的 VP, 有两个候选:

$$VP \rightarrow VP PP: \quad 0.4 \times 0.24 \times 0.1 = 0.0096, \quad (9.20)$$

$$VP \rightarrow V NP: \quad 0.6 \times 1 \times 0.012 = 0.0072. \quad (9.21)$$

Viterbi CKY 取最大值得 0.0096; inside 求和得到 0.0168。再乘主语 NP 的 0.2, 分别恢复前面的最佳树概率与句子总概率。

这张表没有为两棵完整树重复计算 telescope 的 NP 和 with telescope 的 PP。句子更长、歧义更多时, 这种共享会带来巨大节省。

一条规则的后验使用次数

在这两棵树中, $NP \rightarrow NP PP$ 只在 NP 附着树里出现一次, 因此给定句子的期望使用次数为

$$\mathbb{E}[C_{NP \rightarrow NP PP} | x] = \frac{3}{7}. \quad (9.22)$$

$VP \rightarrow VP PP$ 的期望次数为 $4/7$ 。而 $VP \rightarrow V NP$ 在两棵树中都出现一次, 期望次数就是 1。

无监督文法学习需要的正是这种软计数。若只保留最佳树, 前两条规则的计数会被硬设成 0 和 1, 丢掉另一种解释的概率质量。

为什么基本 PCFG 容易选错附着

这个模型比较两种附着时, 主要看规则概率 0.4 与 0.3。它没有直接读取“望远镜常作为观察工具”这样的词汇知识。把 telescope 换成另一种物体, 规则层面的偏好也许仍然一样。

词汇化或神经跨度评分可以让规则选择依赖具体词义。代价是评分的分解方式可能更复杂。设计模型时要同时考虑表达力与是否仍能使用原来的精确推断。

推演与自检

假设两个候选子树都覆盖 $[2, 5)$ 且根为 NP, 分数分别为 -3 与 -4 。基本 PCFG 中可安全删除第二棵; 如果父规则会根据子树是否包含某个词改变分数, 而状态没有记录该信息, 则这种删除不再有保证。动态规划的正确性永远依赖评分的分解假设。

第 10 章 依存句法：从动作序列到最优树

成分句法关注哪些词构成一个短语，依存句法关注哪个词支配哪个词。例如在“我喜欢红色苹果”中，“喜欢”连接主语“我”和宾语“苹果”，“红色”再修饰“苹果”。这种表示直接暴露词与词的关系，适合信息抽取和语义分析。

本章比较两条解析路线：转移式方法逐步执行动作构建树，图式方法为所有候选弧评分后做全局搜索。二者的差别主要在于怎样组织推断，而非是否使用神经网络。

10.1 有根依存树与投射性

给句子增加虚拟根节点 0，其余词的位置为 $1, \dots, n$ 。依存弧 (h, m, ℓ) 表示中心词 h 支配修饰词 m ，关系类型为 ℓ 。常见类型有主语、宾语、修饰语等。

基本依存树要求每个实际词有且只有一个入弧，虚拟根没有入弧，图无环且所有词从根可达。某些规范还要求恰有一个词直接依赖虚拟根。增强依存图可能允许多重父节点或额外关系，应与基本树任务区分。

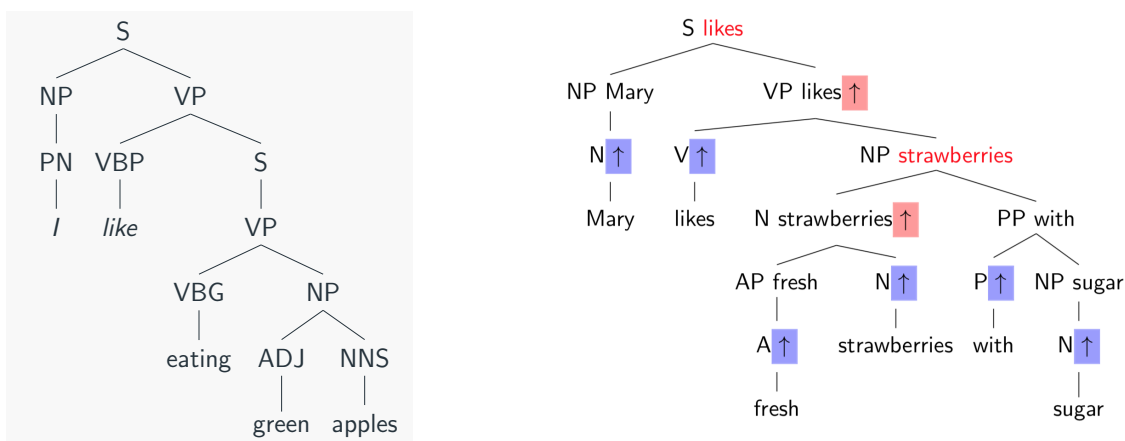


图 10.1 左：成分句法树；右：从成分结构中选择中心词。（课件 lec8_dp，第 4、25 页）

若一条依存弧 $h \rightarrow m$ 之间的所有词都是 h 的后代，该弧满足投射性；整棵树所有弧满足此条件则为投射树。在适当的有根线性排列约定下，它对应弧不交叉的结构。自由语序与某些长距离结构可能产生非投射依存。

数学直觉 | 树结构是一个全局约束

每个词独立选择最高分的父节点，不保证结果是一棵树：两个词可能互相选择，形成环；也可能有多个互不相连的部分。局部弧分数再准确，仍需有约束的解码。

有标签与无标签任务

无标签解析只确定 $h \rightarrow m$ ；有标签解析还确定关系 ℓ 。可以先预测头再预测标签，也可以联合评分 $s(h, m, \ell)$ 。前一种实现简单，但第二阶段不能纠正已经选错的头。

10.2 Arc-standard 转移系统

一个配置记为 $c = (\sigma, \beta, A)$ ，其中 σ 是栈、 β 是尚未处理的缓冲区、 A 是已建立的弧集合。初始配置为 $([0], [1, \dots, n], \emptyset)$ ，目标是缓冲区为空、栈只剩根。

记栈顶为 j ，次栈顶为 i 。三种基本动作是：

$$\text{SHIFT} : (\sigma, i \mid \beta, A) \rightarrow (\sigma \mid i, \beta, A), \quad (10.1)$$

$$\text{LEFT}_\ell : (\sigma \mid i \mid j, \beta, A) \rightarrow (\sigma \mid j, \beta, A \cup \{j \rightarrow_\ell i\}), \quad (10.2)$$

$$\text{RIGHT}_\ell : (\sigma \mid i \mid j, \beta, A) \rightarrow (\sigma \mid i, \beta, A \cup \{i \rightarrow_\ell j\}). \quad (10.3)$$

左弧删除次栈顶，右弧删除栈顶。动作名按线性词序的方向解释，容易与“从哪个栈元素出发”混淆，所以实现时应直接核对被删除的节点和弧方向。

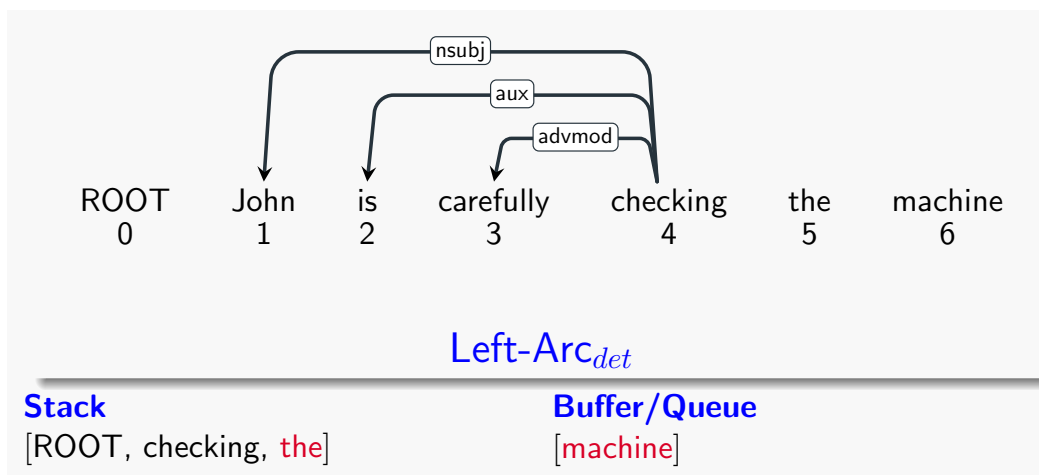


图 10.2 转移系统的状态包含栈、缓冲区和已建弧。每一步改变结构而不重新读取整棵树。(课件 lec8_dp, 第 67 页)

根不能作为被依赖的修饰词删除。左右弧至少要求两个栈元素，SHIFT 要求缓冲区非空。满足基本动作合法性仍不足以定义训练时的最优动作，还需要根据金标准树构造 oracle。

“我爱你” 的动作轨迹

目标弧为 $0 \rightarrow \text{爱}$ 、 $\text{爱} \rightarrow \text{我}$ 、 $\text{爱} \rightarrow \text{你}$ 。

标准投射树有 n 次 SHIFT 和 n 次建弧动作，动作数为 $O(n)$ 。总运行时间还取决于特征提取和动作评分成本；“线性动作数”不意味着所有实现开销都与常数无关。

训练与错误传播

动作分类器可以写为 $p(a \mid c) = \text{softmax}(W\phi(c))_a$ ，也可以使用神经编码器表示栈顶、缓冲区首词及其依存子节点。训练时 oracle 提供正确动作，测试时系统使用自身预测产生后续配置。

贪心解析一个动作出错后，后续状态可能离开训练时见过的分布。beam search 保留多个动作前缀，动态 oracle 则为非理想配置提供仍可达到的最小损失动作。它们解决的是搜索或训练分布问题，不是简单“把分类器做大”。

动作后	栈	缓冲区	新增弧
初始	ROOT	我爱你	无
SHIFT	ROOT 我	爱你	无
SHIFT	ROOT 我爱	你	无
LEFT	ROOT 爱	你	爱 → 我
SHIFT	ROOT 爱你	空	无
RIGHT	ROOT 爱	空	爱 → 你
RIGHT	ROOT	空	ROOT → 爱

表 10.1 每次规约都会删除已经完成其子树的一个节点。

10.3 图式解析与最大生成树

图式解析为候选弧打分，寻找满足树约束的最高分结构：

$$\hat{T} = \arg \max_{T \in \mathcal{T}(x)} \sum_{(h,m) \in T} s_{\theta}(h, m; x). \quad (10.4)$$

若允许非投射树，一阶弧分解模型可使用有向最大生成树算法。若要求单根，需确保解码器额外满足恰好一个根子节点的约束。

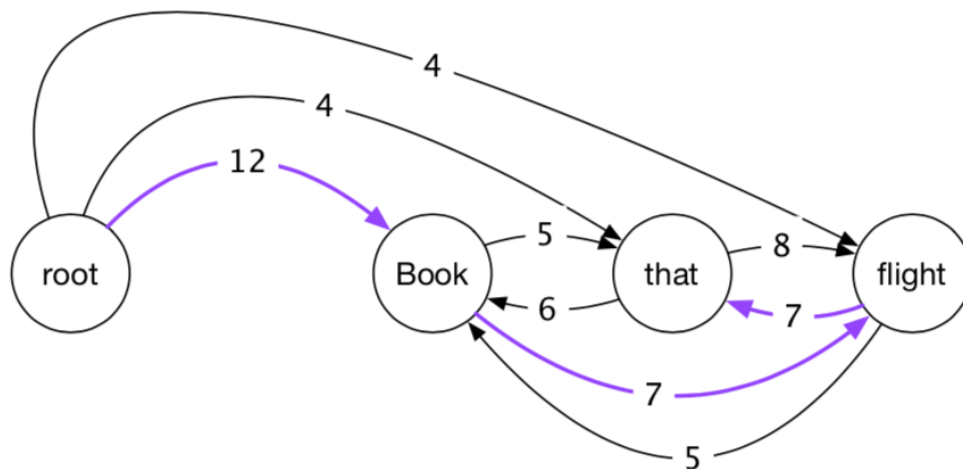


图 10.3 先为候选依存弧评分，再选择满足有根树约束的弧集合。（课件 lec8_dp，第 76 页）

Chu-Liu/Edmonds 的思路是先给每个非根节点选最高分入弧。如果没有环，已经得到最优有根树；如果出现环，把它收缩成一个超级节点，调整进入环的边分数，在缩小的图上继续求解，再展开恢复。

调整进入环的分数时，要考虑替换哪一条环内入弧。若外部节点 u 连接到环内 v ，保留其余已选环内边的收益，因此比较量可写为

$$s(u, v) - s(\text{parent}_{\text{cycle}}(v), v) \quad (10.5)$$

再加所有候选共享的环内总分。这个减法正是“打破环需要牺牲哪条边”的代价。

一个环的例子

三个词各自最高分入弧形成 $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$ ，边分数都为 5；根到 1、2、3 的分数分别为 4、2、1。要得到一棵树，需要替换一条分数为 5 的环内边。接根到 1 损失最小，为 $5 - 4 = 1$ ，最终保留两条环内边加根边，总分为 14。

这个例子解释了为什么不能随意删掉环上的最低编号边：打破环应比较总分损失，而非采用任意规则。

10.4 投射动态规划、神经评分与评价

要求投射性时，可以使用 Eisner 算法。它把跨度分成两类：不完整项已建立两端之间的一条依存弧，但还需接上另一侧子树；完整项表示某个端点为中心词、内部结构已经闭合的跨度。

每类再区分中心词在左端还是右端。四类状态的作用，是让合并时同时保证树结构与不交叉性。按跨度长度递推，时间为 $O(n^3)$ ，空间为 $O(n^2)$ 。仅记“某跨度的最好分数”不够，因为上层还需要知道中心词在哪一端、哪些边界仍可继续连接。

备注 | 算法与假设匹配

MST 可以处理非投射的一阶弧分解评分；Eisner 为投射树提供精确动态规划。若评分还依赖兄弟节点、祖父节点等高阶结构，原来的状态或算法可能不再足够，需要扩充状态或采用近似推断。

神经依存解析器可为每个词计算两个表示，分别对应作为中心词与作为修饰词的角色。双仿射评分为

$$s(h, m) = r_h^\top U r_m + w^\top [r_h; r_m] + b. \quad (10.6)$$

双线性项建模两者的交互，线性项提供单边偏好。表示可来自 BiLSTM 或 Transformer，解码仍执行全局树约束。

算法实现 | 转移式解析的推断骨架

```
stack, buffer, arcs = [ROOT], list(range(1, n+1)), []
while buffer or len(stack) > 1:
    state = encode(stack, buffer, arcs)
    actions = legal_actions(stack, buffer, arcs)
    action = max(actions, key=lambda a: score(state, a))
    stack, buffer, arcs = apply(action, stack, buffer, arcs)
return arcs
```

实际实现需要避免反复从列表头删除造成额外复杂度，可使用索引或队列。legal_actions 必须包含根约束与结构约束。

无标签附着准确率 UAS 衡量中心词位置是否正确，有标签附着准确率 LAS 同时要求中心词和关系类型正确。标点是否计入、根如何处理、分词是否给定，都应固定。若分词不同，直接按位置比较依存弧会失去意义。

训练 oracle 为什么要等子节点收齐

在 arc-standard 中，一个节点被规约出栈后，通常不能再作为栈中的中心词接收新的依存子节点。因此，即使次栈顶与栈顶之间存在正确依存弧，也不能总是立刻执行对应规约。

例如栈顶词“喜欢”后面还要接宾语“苹果”。若过早把“喜欢”连接到 ROOT 并弹出，它就无法再接收宾语。静态 oracle 会检查即将弹出的节点是否已经收齐金标准中的相关子节点。

这个检查体现了动作系统的时间约束。金标准树只描述最终结构，oracle 还要把它转换成能被当前转移系统实现的动作顺序。

一个非投射结构的例子

设四个词按 1、2、3、4 排列，依存弧包含 $1 \rightarrow 3$ 与 $2 \rightarrow 4$ 。两条弧的端点交错为 $1 < 2 < 3 < 4$ ，在同一侧绘制时会交叉。如果再有 $1 \rightarrow 2$ 和 $0 \rightarrow 1$ ，整个结构可以是有根树，但不是投射树。

基本 arc-standard 不覆盖所有非投射树。可以加入 SWAP 等动作处理更一般结构，也可以使用允许非投射性的图式解码。不同语言与树库中的非投射比例，会影响这个选择。

树约束为什么不能只靠行归一化

假设每个词分别通过 softmax 选择父节点：

$$p(h \mid m, x) = \frac{e^{s(h,m)}}{\sum_{h' \neq m} e^{s(h',m)}}. \quad (10.7)$$

每个词的父节点概率和为 1，但独立最大化仍可能让 1 选 2、2 选 1，构成环。局部概率合法不等于联合结构合法。

可以用这些局部分数训练，再用 MST 解码；也可以在所有合法树上定义全局分布：

$$p(T \mid x) = \frac{\exp \sum_{(h,m) \in T} s(h,m)}{\sum_{T' \in \mathcal{T}(x)} \exp \sum_{(h,m) \in T'} s(h,m)}. \quad (10.8)$$

后者需要计算树集合的配分函数。对某些一阶有向树模型，可利用矩阵树定理完成求和。这里的求和算法与最大生成树解码再次是两个不同问题。

UAS 与 LAS 的计数示例

一个句子有 5 个计分词，其中 4 个中心词预测正确。正确中心词的 4 个词中，只有 3 个关系标签也正确。则 UAS 为 $4/5 = 0.8$ ，LAS 为 $3/5 = 0.6$ 。

不能把 LAS 算成 $3/4$ ，因为其分母仍是所有计分词，而不是中心词已经正确的子集。后者可作为条件标签准确率，但要用不同名称。

如果系统 A 把一个长距离依存错连到邻词，而系统 B 把它连到根，两者在 UAS 上同样记一次错。实际下游影响可能不同，因此需要结合关系类型和距离分析，而不只看单个指标。

神经评分与约束解码的分工

双仿射评分可对每个候选中心词、修饰词组合产生分数。若输入有 n 个词，候选弧矩阵大致有 n^2 项。训练编码器使正确弧分数提高，解码器则在这些分数上执行树约束。

这一设计把统计学习与组合优化分开：评分器不必通过参数隐式学会每条树约束，解码器也不必理解词义。两者各自处理擅长的部分，是经典 NLP 与神经方法结合的重要方式。

第 11 章 组合语义与可执行的意义表示

句法树告诉我们词怎样组合，但还没有直接告诉机器一句话在什么条件下为真，也没有告诉数据库应该执行什么查询。“每个学生读了一本书”看似简单，却可能表示大家读同一本书，也可能每个人读不同的书。

语义分析的目标不是把自然语言逐词翻译成符号，而是构造可以组合、比较和执行的表示。本章用一阶逻辑、类型与 λ 演算解释这一过程，并把它连接到语义解析与文本到 SQL。

11.1 真值条件与一阶逻辑

一阶逻辑包含常量、变量、谓词、函数、逻辑连接词与量词。若 $\text{Student}(x)$ 表示 x 是学生， $\text{Read}(x, y)$ 表示 x 读了 y ，则“某个学生读了这本书”可写为

$$\exists x [\text{Student}(x) \wedge \text{Read}(x, b)]. \quad (11.1)$$

常量 b 指某个特定对象。解释结构为符号指定实体、集合或关系；只有在这样的结构中，公式才获得真值。

“所有学生都读了这本书”应写为

$$\forall x [\text{Student}(x) \Rightarrow \text{Read}(x, b)]. \quad (11.2)$$

这里用蕴含而不是合取。若写 $\forall x [\text{Student}(x) \wedge \text{Read}(x, b)]$ ，就声称论域中的所有对象都是学生且都读过书，含义过强。

数学直觉 | 量词的范围就是意义的一部分

符号化时最重要的常常不是谓词名字，而是变量由哪个量词约束、否定作用于哪一部分、两个事件是否共享同一个参与者。词相同而作用域不同，真值条件就可能不同。

否定与量词满足

$$\neg \forall x P(x) \equiv \exists x \neg P(x), \quad \neg \exists x P(x) \equiv \forall x \neg P(x). \quad (11.3)$$

“并非所有学生都通过”表示至少一人未通过，不等同于“所有学生都未通过”。这样的区别是逻辑推理评价中常见的错误来源。

量词作用域的两个读法

“每个学生读了一本书”至少有两种表示：

$$\forall x [\text{Student}(x) \Rightarrow \exists y (\text{Book}(y) \wedge \text{Read}(x, y))], \quad (11.4)$$

$$\exists y [\text{Book}(y) \wedge \forall x (\text{Student}(x) \Rightarrow \text{Read}(x, y))]. \quad (11.5)$$

第一式允许不同学生选择不同书；第二式要求存在一本被所有学生读过的书。第二式在通常语义下蕴含第一式，反向不成立。若有两名学生分别只读不同的书，就构成反例。

11.2 类型与函数应用如何组成句子

令实体类型为 e ，真值类型为 t 。“小王”表示一个 e 类型对象，“睡觉”表示 $e \rightarrow t$ 的函数。于是

$$\text{sleep} : e \rightarrow t, \quad \text{wang} : e, \quad \text{sleep}(\text{wang}) : t. \quad (11.6)$$

类型规定组合接口，像程序设计语言一样，可以在执行之前排除一部分不合法组合。

$\lambda x.P(x)$ 表示一个以 x 为参数的函数。函数应用通过 β 归约完成：

$$(\lambda x.\text{Sleep}(x))(\text{wang}) \Rightarrow \text{Sleep}(\text{wang}). \quad (11.7)$$

二元关系可以柯里化。例如令“喜欢”的词义为

$$\lambda y.\lambda x.\text{Like}(x, y), \quad (11.8)$$

先接宾语，再接主语。对“Bill likes Mary”：

$$(\lambda y.\lambda x.\text{Like}(x, y))(\text{mary}) \Rightarrow \lambda x.\text{Like}(x, \text{mary}), \quad (11.9)$$

$$(\lambda x.\text{Like}(x, \text{mary}))(\text{bill}) \Rightarrow \text{Like}(\text{bill}, \text{mary}). \quad (11.10)$$

这把句法规则 $VP \rightarrow V NP$ 对应到函数应用，再把 $S \rightarrow NP VP$ 对应到第二次应用。

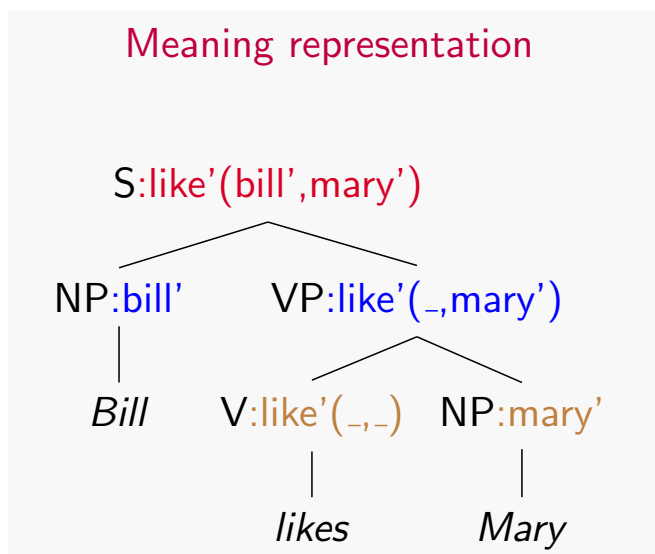


图 11.1 语义随句法树自底向上组合，短语的意义成为上层组合的输入。（课件 lec9_sp，第 51 页）

为什么普通实体类型还不够

“每个学生”不是一个特定实体，也不只是学生集合；它要接收一个性质并判断所有学生是否具有该性质。因此可以把它表示为广义量词：

$$\llbracket \text{每个学生} \rrbracket = \lambda Q.\forall x[\text{Student}(x) \Rightarrow Q(x)]. \quad (11.11)$$

其类型是 $(e \rightarrow t) \rightarrow t$ 。与谓词“睡觉”组合，得到

$$(\lambda Q.\forall x[\text{Student}(x) \Rightarrow Q(x)])(\lambda z.\text{Sleep}(z)) \Rightarrow \forall x[\text{Student}(x) \Rightarrow \text{Sleep}(x)]. \quad (11.12)$$

专名也可提升到同样类型： $[[小王]] = \lambda Q.Q(wang)$ 。这样不同类型的名词短语可以进入统一的组合规则。

限定词“每个”的表示为

$$\lambda P.\lambda Q.\forall x[P(x) \Rightarrow Q(x)], \quad (11.13)$$

先接名词性质 P ，再接动词短语性质 Q 。这比为每个完整短语单独写逻辑式更能体现组合性。

备注 | 变量改名与捕获

$\lambda x.P(x)$ 与 $\lambda z.P(z)$ 只改变绑定变量名称，意义相同。代入时必须避免把自由变量意外变成被绑定变量；必要时先做变量改名。一个可靠的语义解析器通常操作抽象语法树，而非直接进行字符串替换。

11.3 事件、关系图与推理

把事件作为变量，可以自然添加时间、地点、方式等修饰。“小王昨天在北京吃了苹果”可表示为

$$\exists e \exists a [Eat(e) \wedge Agent(e, wang) \wedge Apple(a) \wedge Theme(e, a) \wedge Time(e, yesterday) \wedge Location(e, beijing)]. \quad (11.14)$$

增加一个状语，只需增加有关同一事件变量的关系，不必为每一种动词参数数量定义全新的谓词。

关系图把实体、事件及属性作为节点或边，也能表示共享参与者、重入和省略。某些语义图并不是树：同一实体可能同时参与多个关系，因此不能直接照搬句法树的所有约束。

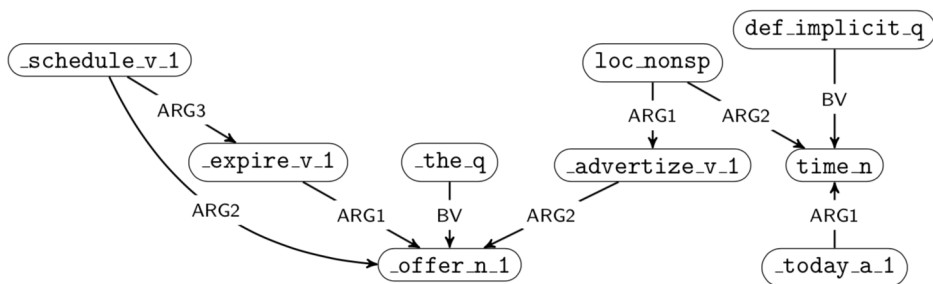


图 11.2 图式语义表示显式连接事件、实体与关系，可包含超出树结构的共享节点。（课件 lec9_sp，第 78 页）

逻辑蕴含 $\Gamma \models \phi$ 表示：所有满足前提集合 Γ 的解释结构，也满足结论 ϕ 。若知道“所有学生都是人”和“小王是学生”，就能推出“小王是人”。这是对所有可能模型的语义约束，不是单纯比较两个句子的词向量距离。

自然语言推断通常标为蕴含、矛盾或中立。中立表示前提不足以决定假设，而不是假设为假。数据中没有出现某个事实，也不自动等于该事实为假：知识库的开放世界假设与数据库查询中常见的封闭世界约定需要区分。

规则推理的小例子

设有规则 $Parent(x, y) \wedge Parent(y, z) \Rightarrow Grandparent(x, z)$ ，事实为 $Parent(A, B)$ 、 $Parent(B, C)$ 。变量统一给出 $x = A, y = B, z = C$ ，推出 $Grandparent(A, C)$ 。系统需要保证两处 y 绑定同一个对象，不能分别任取不同的人。

算法实现 | 一个二跳关系推理

```

parents = {("A", "B"), ("B", "C"), ("D", "E")}
grandparents = {
    (x, z)
    for x, y in parents
    for y2, z in parents
    if y == y2
}
# {("A", "C")}

```

集合连接是逻辑变量共享的一个具体实现。复杂规则系统需要索引和更系统的统一算法。

11.4 从自然语言到程序

语义解析学习从文本 x 到逻辑式或程序 z 的映射：

$$\hat{z} = \arg \max_{z \in \mathcal{Z}(x)} p_{\theta}(z | x). \quad (11.15)$$

如果有成对的 (x, z) 标注，可直接做序列到序列训练，或用语法约束生成抽象语法树。若只有问题与答案 (x, a) ，可将程序作为隐变量：

$$p(a | x) = \sum_z p_{\theta}(z | x) \mathbf{1}[\text{Exec}(z) = a]. \quad (11.16)$$

求和只包含执行结果为正确答案的程序，但其中可能有“碰巧答对”的伪程序。训练集执行正确，不代表语义正确；换一个数据库后它可能立即失败。

文本到 SQL 的完整映射

设表 `students(name, score, city)`，问题为“北京学生中，分数超过 90 的有哪些人？”一种查询是

```

SELECT name
FROM students
WHERE city = 'Beijing' AND score > 90;

```

这里至少需要四种对齐：问题中的实体值对应数据库值，属性对应列名，比较词对应运算符，提问目标对应 SELECT 列。只生成语法正确的 SQL 远远不够。

如果问题改为“每个城市最高分学生的姓名”，简单的 `GROUP BY city` 加 `MAX(score)` 并不自动返回与最高分对应的姓名，还涉及分组、并列最高分和连接 / 窗口函数。自然语言中的“最高”“每个”“至少”往往指向具体的逻辑结构。

受约束解码在生成过程中屏蔽语法或类型不合法的扩展。例如 SELECT 后只能输出有效列或表达式，数值比较不应作用于不兼容的类型。约束缩小搜索空间，但语法正确仍不保证含义正确。

评价可比较逻辑式完全匹配，也可比较执行结果。不同程序可能语义等价，因此完全匹配有时过严；不同程序又可能只在某个数据库上碰巧得到相同结果，因此单一执行测试有时过松。多种测试数据库和中间结构检查，可以减少这两种偏差。

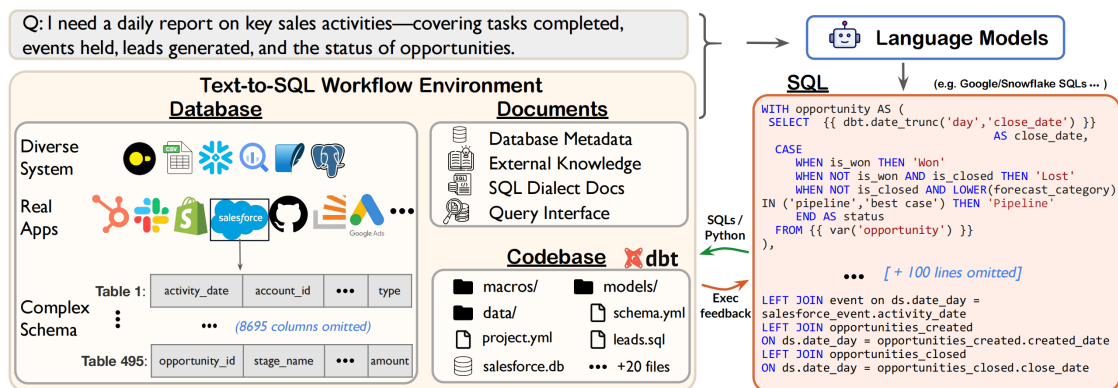


图 11.3 文本到 SQL 需要把语言与具体数据库结构对齐，并在执行环境中检验结果。（课件 lec9_sp，第 82 页）

用一个小世界检查量词公式

设论域中有两名学生 a, b 和两本书 u, v 。已知 a 只读过 u ， b 只读过 v 。

公式“每个学生读过某本书”为真，因为给 a 选择 u 、给 b 选择 v 即可。公式“存在一本书，所有学生都读过”为假，因为 u 只有 a 读过， v 只有 b 读过。

可以把读书关系写成矩阵

$$R = \begin{array}{c|cc} & u & v \\ \hline a & 1 & 0 \\ b & 0 & 1 \end{array} \quad (11.17)$$

$\forall x \exists y R(x, y)$ 检查每行是否至少有一个 1； $\exists y \forall x R(x, y)$ 检查是否存在全为 1 的一列。两个量词顺序对应不同的矩阵条件，这比只背符号更直观。

“只有”与“所有”的方向

“只有学生可以进入”表示可以进入的人必定是学生：

$$\forall x [\text{Enter}(x) \Rightarrow \text{Student}(x)]. \quad (11.18)$$

“所有学生都可以进入”则是

$$\forall x [\text{Student}(x) \Rightarrow \text{Enter}(x)]. \quad (11.19)$$

前者把学生身份作为必要条件，后者把学生身份作为充分条件。自然语言中的一个词就能改变蕴含方向，所以简单的词袋相似度不足以处理这类推理。

把及物动词与量化宾语组合

普通及物动词的类型为 $e \rightarrow (e \rightarrow t)$ ，但“每本书”的类型是 $(e \rightarrow t) \rightarrow t$ ，不能直接当作实体代入。可以把组合改写为让量词接收一个性质。

对固定主语 a ，“ a 读每本书”写为

$$(\lambda Q. \forall y [\text{Book}(y) \Rightarrow Q(y)])(\lambda z. \text{Read}(a, z)). \quad (11.20)$$

归约后得到 $\forall y [\text{Book}(y) \Rightarrow \text{Read}(a, y)]$ 。这里先把带空缺宾语的动词短语变成性质，再交给宾语量词。

这种类型调整解释了为什么组合语义不能只规定“左边函数吃掉右边参数”一种固定动作。不同语法位置与量词范围可能需要不同的函数组合。

变量捕获的具体例子

考虑将自由变量 y 代入 $(\lambda x.\lambda y.x)$ 的 x 。若直接替换成 $\lambda y.y$ ，原本自由的 y 被内部量词绑定，含义改变了。

正确做法先把内部绑定变量改名：

$$(\lambda x.\lambda y.x) y \equiv (\lambda x.\lambda z.x) y \Rightarrow \lambda z.y. \quad (11.21)$$

最终函数无论输入什么，都返回原来的自由变量 y 。语义解析器中的变量名看似只是字符串，但绑定关系必须由结构管理。

弱监督中的伪程序

数据库当前只有一名叫“小王”的北京学生。问题问“北京学生有哪些”，程序一是按 city 查询，程序二是直接返回小王。二者在当前数据库上都答对，但只有第一种表达了问题含义。

如果训练只根据当前执行结果奖励正确答案，两种程序都会得到奖励。加入包含多名北京学生的新数据库，或设计针对城市条件的反事实测试，才能区分它们。

这个问题与大模型奖励投机相通：系统可能找到满足评价器的捷径，而没有学到预期规则。评价数据要能区分正确机制与偶然正确结果。

推演与自检

将“没有学生读过所有书”写为 $\neg \exists x[\text{Student}(x) \wedge \forall y(\text{Book}(y) \Rightarrow \text{Read}(x, y))]$ 。再比较“所有学生都没有读过任何书”。后者更强：前者允许每个人读过部分书，只要求没人读遍所有书。

第 12 章 信息检索：从词项匹配到稠密表示

搜索与分类有一个重要区别：分类通常从少量标签里选答案，搜索要从大量文档中找出最相关的几个结果。一个模型即使判断得准确，如果必须逐篇阅读整个文档库，也很难用于大规模检索。

因此，检索同时关心相关性与效率。本章先解释倒排索引、TF-IDF 和 BM25，再介绍双编码器、对比学习与近似搜索。这些内容也是后面 RAG 系统的基础。

12.1 索引、匹配与排序

给定查询 q 和文档集合 $\mathcal{D}$ ，检索系统寻找

$$\text{TopK}_{d \in \mathcal{D}} s(q, d). \quad (12.1)$$

相关性不一定是简单的主题相似。查询“如何重置路由器”需要操作说明，一篇讲路由器市场规模的文章虽然主题接近，却未必有用。

倒排索引为每个词存储包含它的文档列表，通常还保存词频和位置。查询“自然语言”时，布尔 AND 检索求两个列表的交集，OR 求并集。列表按文档编号排序后，可以用双指针线性合并，不必扫描所有原文。

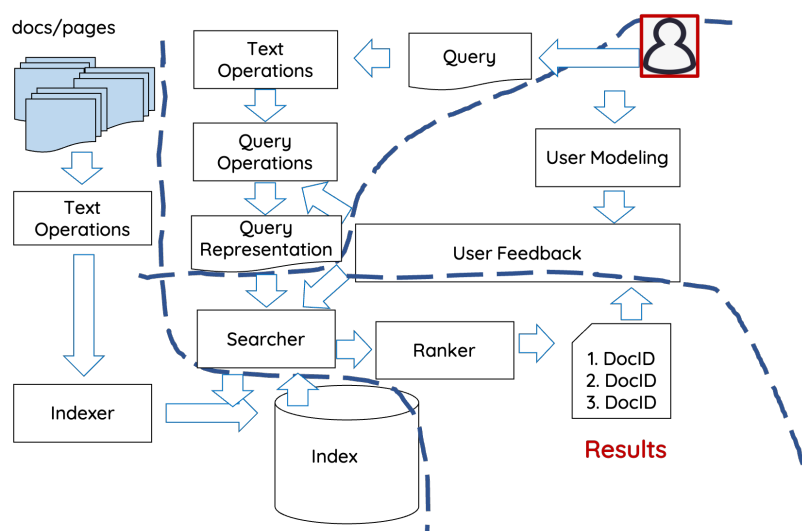


图 12.1 索引在离线阶段整理文档，在线阶段利用索引完成匹配与排序。（课件 lec10_ir，第 9 页）

分词、大小写、词形归一、停用词和字段划分都会影响索引。中文检索中的词边界尤其重要。检索单元也可以是整篇文档、段落或固定窗口；单元越大，上下文越完整，但无关内容也越多。

两阶段检索

第一阶段从海量候选中快速召回几百或几千篇，第二阶段用较昂贵的模型重排。第一阶段的主要目标是不要漏掉好答案，第二阶段再仔细比较。若正确文档根本没被召回，重排模型再强也无从补救。

12.2 TF-IDF 与 BM25

向量空间模型把查询和文档变成词项向量，使用余弦或点积评分。IDF 降低常见词的影响：如果一个词几乎存在于每篇文档中，它对区分文档贡献很小。

仅使用线性词频也有问题。一个词出现 20 次的文档不应总比出现 10 次的文档相关两倍；长文档还天然包含更多查询词。BM25 对词频做饱和处理，并校正文档长度：

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}. \quad (12.2)$$

这里 $f(t, d)$ 是词频， $|d|$ 是文档长度， $k_1 > 0$ 控制饱和速度， $b \in [0, 1]$ 控制长度校正。可使用非负 IDF 变体

$$\text{IDF}(t) = \log \left(1 + \frac{N - \text{df}(t) + 0.5}{\text{df}(t) + 0.5} \right). \quad (12.3)$$

不同系统的 IDF、重复查询词处理和字段权重可能不同，报告实验时应固定实现约定。

当 $b = 0$ ，长度校正消失；当 $b = 1$ ，完全按相对长度校正。在平均长度文档中，词频分式为 $f(k_1 + 1)/(f + k_1)$ 。随着 f 增大，该项趋于 $k_1 + 1$ ，因此高频的边际收益越来越小。

数学直觉 | 出现一次与出现十一次

某篇文档第一次出现一个罕见查询词，通常是强证据；已经出现十次后再多一次，新增证据较少。BM25 的饱和项把这种直觉写进公式。

一个词的评分例子

令 $N = 1000$ 、 $\text{df}(t) = 10$ ，则 IDF 约为 $\log(1 + 990.5/10.5) \approx 4.557$ 。取 $k_1 = 1.2$ ， $b = 0.75$ ，文档长度等于平均长度。词频为 1 时，分式为 1；词频为 3 时，分式为 $6.6/4.2 \approx 1.571$ 。该词贡献从约 4.557 增至 7.161，没有变为三倍。

如果文档长度增至平均长度的两倍，词频仍为 3，分母变为 $3 + 1.2(0.25 + 1.5) = 5.1$ ，分式降为约 1.294。长文档中相同词频提供的证据被适度削弱。

算法实现 | BM25 单词项贡献

```
import math

def bm25_term(tf, df, n_docs, length, avg_length,
              k1=1.2, b=0.75):
    idf = math.log1p((n_docs-df+0.5)/(df+0.5))
    norm = k1 * (1-b+b*length/avg_length)
    return idf * tf*(k1+1)/(tf+norm)
```

多个查询词的贡献再相加。实际搜索应通过倒排列表访问候选文档，而不是对全部文档逐一调用这个函数。

12.3 双编码器与对比学习

词项匹配难以处理“怎样降低发烧”与“退热方法”这类词面差异。稠密检索分别编码查询与文档：

$$u = f_{\theta}(q), \quad v = g_{\phi}(d), \quad s(q, d) = u^{\top} v. \quad (12.4)$$

文档向量可提前计算并存入索引。查询到来时只计算一次 u ，再搜索高分文档。两个编码器可以共享参数，也可以分别训练。

交叉编码器把 $[q; d]$ 一起送入网络，允许细粒度交互。它通常适合重排，因为每个查询文档配对都要重新计算。晚交互模型则保留多个 token 向量，在预编码和细粒度匹配之间折中。

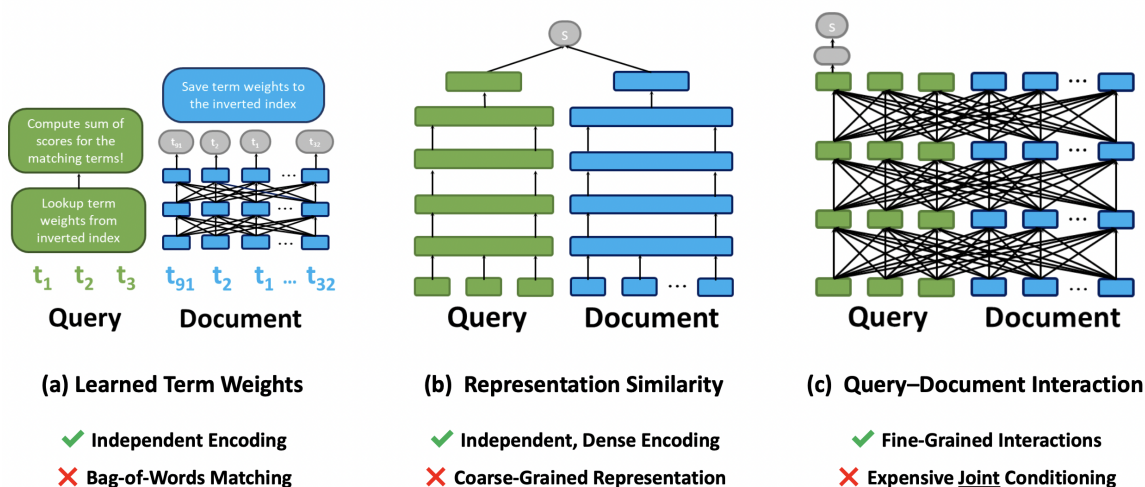


图 12.2 独立编码方便建立索引；更充分的查询文档交互通常需要更多在线计算。（课件 lec10_ir，第 27 页）

对一批 B 个正配对 (q_i, d_i) ，使用其他文档作为批内负例：

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(s(q_i, d_i)/\tau)}{\sum_{j=1}^B \exp(s(q_i, d_j)/\tau)}. \quad (12.5)$$

这是一个 B 类分类目标。温度 τ 控制分数差异对概率的影响。梯度对高分负例更敏感，因为它们更容易与正确文档混淆。

随机负例常常太容易。对于医学查询，随机抽到体育文章几乎不提供有用的区分信号；检索出的主题相似但答案不正确的文档更有训练价值。这类难负例可以来自 BM25，也可以来自当前稠密检索模型。

备注 | 难负例可能其实是正例

一个问题可能有多篇正确文档。如果把其他正确文档当作负例，训练目标会错误地推远它们。应检查多正例标注、近重复内容和自动挖掘的噪声。

负例数量不只是算力参数

更大的批量提供更多竞争文档，也改变了目标难度。跨设备汇集负例时，要确认梯度是否按预期传播、重复样本是否进入负例集合。检索模型更新后，旧的文档向量也会过时，需要约定索引刷新频率。

12.4 近似搜索与检索评价

精确搜索 M 个 d 维向量，需要约 $O(Md)$ 的评分工作。近似最近邻搜索牺牲少量召回换取速度。聚类索引先定位可能的区域，图索引沿相似邻居移动，量化方法则用较短编码近似距离。

近似算法的效果取决于数据分布、索引参数与检索预算，不能笼统保证所有查询都获得严格的次线性最坏情况复杂度。系统评价至少要同时报告相关性、延迟、内存和近似搜索召回。

Recall、MRR 与 NDCG

若问题的相关文档集合为 R_q ，前 k 个结果为 $D_q^{(k)}$ ：

$$\text{Recall}@k = \frac{|R_q \cap D_q^{(k)}|}{|R_q|}. \quad (12.6)$$

只有一个标注正例时，它退化为前 k 中是否命中。MRR 对第一个相关结果的排名 r_q 取倒数再平均，没命中时按协议记为零。

若相关程度有等级，DCG 为

$$\text{DCG}@k = \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)}, \quad \text{NDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k}. \quad (12.7)$$

IDCG 是理想排序的 DCG。折扣使较前位置的相关结果更重要，增益函数则表示不同相关等级的价值。

稀疏与稠密检索常有互补性。前者擅长罕见词、编号与精确短语，后者擅长语义改写。可以先合并召回结果，再统一重排；也可以用倒数排名融合

$$\text{RRF}(d) = \sum_m \frac{1}{k_0 + \text{rank}_m(d)}. \quad (12.8)$$

它不要求不同模型的原始分数具有相同量纲，但融合参数仍需验证。

完整算例：词频与长度的竞争

假设只有一个查询词 t ，三篇文档都包含它。文档 A 的词频为 1、长度为 50；B 的词频为 3、长度为 100；C 的词频为 3、长度为 400。设平均长度为 100， $k_1 = 1.2, b = 0.75$ 。

由于 IDF 对三篇文档相同，只比较词频分式。对 A，长度校正为 $1.2(0.25 + 0.75 \times 0.5) = 0.75$ ，得分因子为 $2.2/1.75 \approx 1.257$ 。对 B 为 $6.6/4.2 \approx 1.571$ 。对 C，长度校正为 $1.2(0.25 + 3) = 3.9$ ，得分因子为 $6.6/6.9 \approx 0.957$ 。

所以排序为 B、A、C。C 虽然与 B 有相同词频，但词在更长文档中没有那么集中。A 只出现一次，却胜过很长的 C。这正是长度归一化试图表达的区别。

双编码器损失的一次手算

某查询对三个文档的缩放后分数为 $(2, 1, 0)$ ，第一个为正例。其 softmax 概率为

$$p \approx (0.665, 0.245, 0.090). \quad (12.9)$$

正例损失约为 $-\log 0.665 = 0.408$ 。

对三个分数的梯度分别为 $(-0.335, 0.245, 0.090)$ 。第二个负例得分更高，对梯度的影响也更大。如果把第三个负例换成与查询非常接近的难负例，分数升到 1.8，它会提供更强的区分信号。

但如果这个“难负例”其实也包含正确答案，梯度就会错误地降低有用文档分数。因此难负例挖掘需要比随机负例更仔细的质量控制。

多个正例应怎样进入目标

若查询有多个相关文档集合 P_i ，可以把它们的概率质量合在分子：

$$\ell_i = -\log \frac{\sum_{j \in P_i} \exp(s_{ij}/\tau)}{\sum_j \exp(s_{ij}/\tau)}. \quad (12.10)$$

这个目标只要求至少部分正例获得较高总质量，不保证每个正例分数都同样提高。另一种做法对每个正例分别计算目标，再平均；两者强调的学习行为不同。

选择应与应用一致。若任意一篇文档都足以回答问题，聚合正例质量可能合理；若需要覆盖多个互补事实，则应鼓励召回多种证据。

排名指标的完整例子

前五个结果的二元相关性为 $(0, 1, 0, 1, 1)$ ，相关文档总数为 4。Recall@5 为 $3/4$ ，第一个相关结果在第 2 位，所以该查询的倒数排名为 $1/2$ 。

若只计算这五个位置上的 average precision，并以全部 4 个相关文档为分母，则

$$AP@5 = \frac{1}{4} \left(\frac{1}{2} + \frac{2}{4} + \frac{3}{5} \right) = 0.4. \quad (12.11)$$

遗漏的第 4 个相关文档贡献零。不同 AP@k 协议可能对分母有不同约定，因此最好明确公式。

索引召回与任务召回不同

近似搜索的索引召回，比较的是近似结果是否包含精确向量搜索的 Top-k。任务召回，比较的是否找到人工标注的相关文档。

一个索引可以完美复现向量 Top-k，却因为编码器表示不好而漏掉相关证据；另一个索引可能漏掉少数精确近邻，却仍保留所有真正相关文档。前者测搜索近似误差，后者测端到端相关性，不能混用。

第二部分

Transformer 与大语言模型

第 13 章 从注意力到 Transformer

循环网络把历史压进一个状态。句子很长时，模型必须一边保存旧信息，一边加入新信息。注意力提供了另一种办法：需要预测时，直接回到已有的表示中寻找相关内容。它不要求每条信息都经过同一条狭窄的记忆通路。

本章从翻译中的软对齐开始，逐步建立自注意力、位置表示和 Transformer。每一步都先说明输入输出，再写公式。这样，长公式中的矩阵就能对应到具体的计算动作。

13.1 序列到序列与软对齐

翻译、摘要和问答都可以写成条件生成：

$$p_{\theta}(y_{1:M} \mid x_{1:T}) = \prod_{t=1}^M p_{\theta}(y_t \mid y_{<t}, x_{1:T}). \quad (13.1)$$

编码器读取输入，解码器根据输入表示和已生成的目标前缀预测下一个 token。早期编码器解码器模型常用一个向量概括整句。

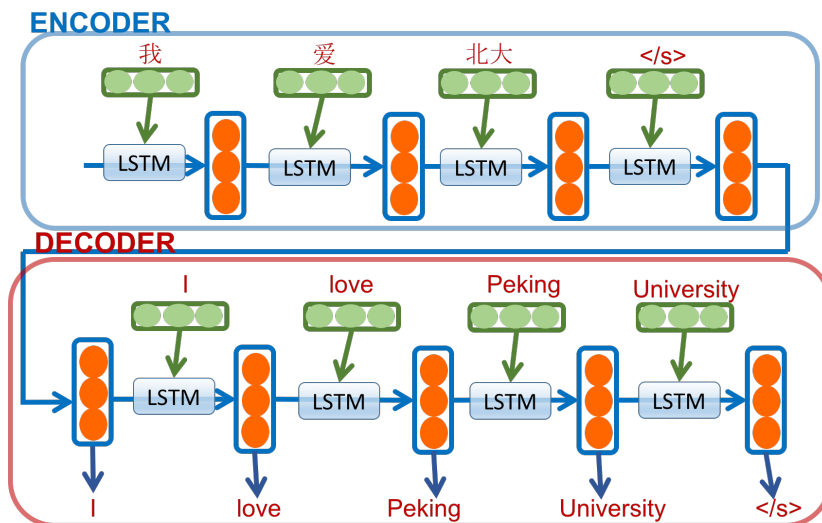


图 13.1 编码器读取源序列，解码器逐步产生目标序列。固定向量传递全部信息时，长输入更难压缩。（课件 lec6_seq，第 37 页）

注意力保留所有编码器状态 $h_1, \dots, h_T$ 。在目标时刻 t ，解码器根据当前状态 s_{t-1} 给每个源位置打分：

$$e_{ti} = v^{\top} \tanh(W_s s_{t-1} + W_h h_i), \quad (13.2)$$

$$\alpha_{ti} = \frac{e^{e_{ti}}}{\sum_{j=1}^T e^{e_{tj}}}, \quad c_t = \sum_{i=1}^T \alpha_{ti} h_i. \quad (13.3)$$

c_t 是当前预测所需的输入摘要。不同目标词可以使用不同权重，所以它不再是全句共享的固定向量。

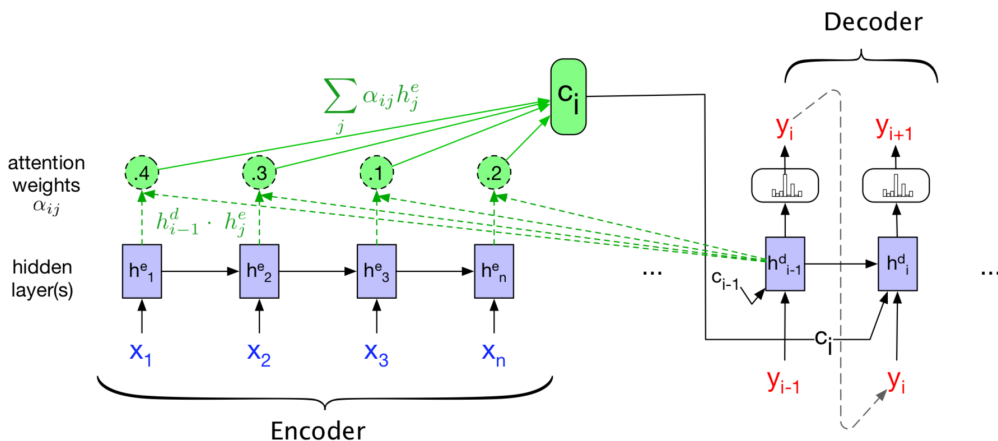


图 13.2 解码器用当前状态选择源序列中的相关位置。权重形成软对齐，不要求一一对应。（课件 lec6_seq, 第 46 页）

如果正在把“红色汽车”译成英文，在输出 red 时更关注“红色”，输出 car 时更关注“汽车”。软对齐可以分散在多个位置，允许信息混合，也可以通过梯度端到端学习。

数学直觉 | 按问题取信息

编码器状态像一组已经整理过的资料。解码器每一步提出不同的问题，注意力再据此取出相关内容。资料没有改变，读取方式却可以改变。

13.2 Q、K、V 与缩放点积注意力

把“提出问题”“用于匹配”“实际读取的内容”分别记为查询 Q、键 K、值 V。对行向量形式的输入 $X \in \mathbb{R}^{T \times d}$ ：

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V. \quad (13.4)$$

若 $W_Q, W_K \in \mathbb{R}^{d \times d_k}$ 、 $W_V \in \mathbb{R}^{d \times d_v}$ ，则 $Q, K \in \mathbb{R}^{T \times d_k}$ 、 $V \in \mathbb{R}^{T \times d_v}$ 。注意力计算为

$$A = \text{softmax}_{\text{row}} \left(\frac{QK^T}{\sqrt{d_k}} + M \right), \quad O = AV. \quad (13.5)$$

QK^T 的形状为 $T \times T$ 。第 i 行比较查询 q_i 与所有键。softmax 沿键的位置归一化，因此每行权重和为 1。输出 O 的形状为 $T \times d_v$ 。

交叉注意力中，Q 来自目标序列，K 与 V 来自源序列；若两边长度为 M, T ，注意力矩阵是 $M \times T$ 。自注意力中三者来自同一组输入，但投影参数通常不同。

为什么除以平方根

假设查询与键各维近似独立、均值为零、方差为 1，则

$$\text{Var}(q^T k) = \text{Var} \left(\sum_{j=1}^{d_k} q_j k_j \right) \approx d_k. \quad (13.6)$$

维度越大，点积分数尺度越大，softmax 可能过早变得接近 one-hot。除以 $\sqrt{d_k}$ 让这个粗略模型下的方差保持在常数量级。实际表示不严格满足这些假设，但它说明了缩放的动机。

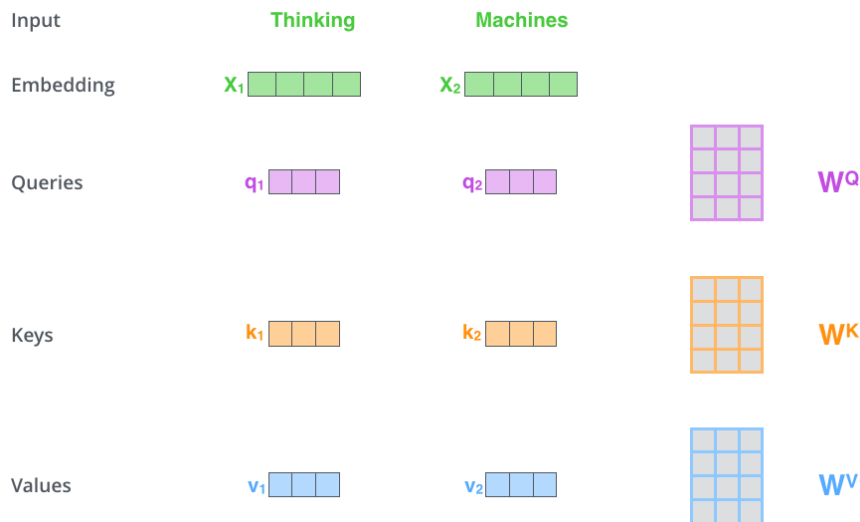


图 13.3 每个输入位置分别投影成 query、key 与 value。同一个输入可以在三种角色中使用不同特征。
(课件 lec6_seq, 第 56 页)

一个可以手算的注意力

为简化计算，令 $d_k = 1$ ，查询 $q = 1$ ，两个键 $k_1 = 0, k_2 = \log 3$ 。分数为 $(0, \log 3)$ ，归一化后权重为 $(1/4, 3/4)$ 。若值为 $v_1 = (2, 0)$ 、 $v_2 = (0, 4)$ ，输出为

$$o = \frac{1}{4}(2, 0) + \frac{3}{4}(0, 4) = (0.5, 3). \quad (13.7)$$

键决定读多少，值决定读出什么。如果只改 v_2 ，权重不变而输出改变；如果只改 k_2 ，读取比例改变。这是区分 K 与 V 最直接的方法。

因果掩码与填充掩码

因果语言模型在位置 i 不能读取未来位置 $j > i$ 。定义

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases} \quad (13.8)$$

加到分数后，未来位置的 softmax 权重为零。padding 掩码则屏蔽无效的键位置。掩码要在 softmax 前加入，而不是在归一化后简单清零，否则剩余权重和不再为 1。

备注 | 注意力权重的解释边界

权重说明某层某头如何混合 value，并不自动说明某个词对最终预测的因果贡献。残差路径、其他头和后续层都会改变输出。可视化注意力有助于理解计算，但不能直接等同于完整解释。

13.3 多头、残差与前馈网络

单个注意力头只有一套匹配空间。多头注意力为每个头学习独立投影：

$$H_r = \text{Attention}(XW_Q^{(r)}, XW_K^{(r)}, XW_V^{(r)}), \quad (13.9)$$

$$\text{MHA}(X) = [H_1; \dots; H_h]W_O. \quad (13.10)$$

通常令每头维度约为 d/h ，拼接后回到 d 维。头数增加不等于总表示维度一定增加；要结合每头维度一起看。

一个常见的 pre-norm Transformer 层为

$$\tilde{X} = X + \text{MHA}(\text{LN}(X)), \quad (13.11)$$

$$X' = \tilde{X} + \text{FFN}(\text{LN}(\tilde{X})). \quad (13.12)$$

注意力让不同位置交换信息；前馈网络在每个位置独立地变换特征。所有位置共享同一组 FFN 参数。

$$\text{FFN}(x) = W_2 \rho(W_1 x + b_1) + b_2. \quad (13.13)$$

残差连接保留原表示，并让子层学习修正量。LayerNorm 沿单个 token 的特征维度计算均值和方差：

$$\text{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta. \quad (13.14)$$

RMSNorm 则不减均值，使用均方根缩放。二者都不是沿时间位置归一化，不能把整句均值错误地代入。

门控前馈层

一种常见的门控形式是

$$\text{SwiGLU}(x) = W_o [\text{SiLU}(W_g x) \odot (W_u x)]. \quad (13.15)$$

一条分支产生内容，另一条分支调节各维通过量。这里的乘法允许输入相关的特征选择。参数量比较时，要计算三组投影矩阵，不能沿用两层 FFN 的宽度而声称成本不变。

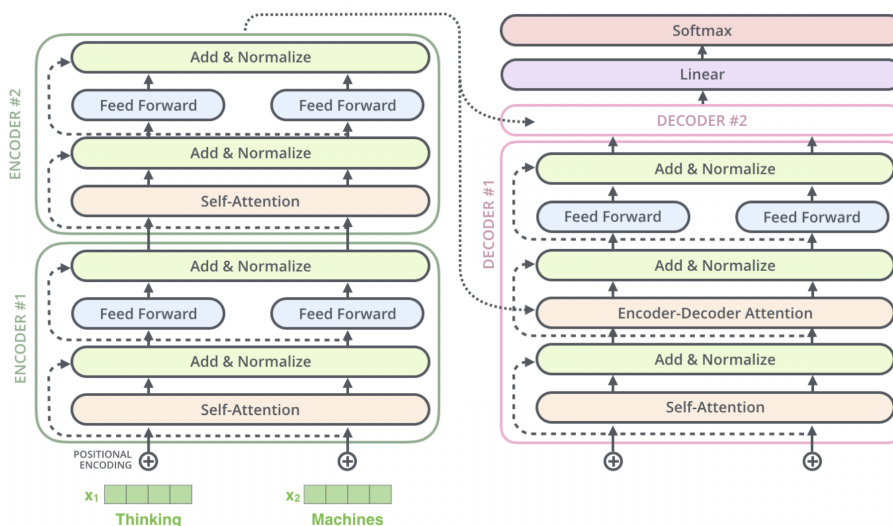


图 13.4 编码器和解码器的层次结构。解码器自注意力需要因果掩码；交叉注意力读取编码器输出。该图展示早期架构，归一化位置可随实现变化。（课件 lec6_seq，第 66 页）

13.4 位置表示：模型怎样知道词序

没有位置特征和掩码时，自注意力对输入排列具有置换等变性。设 P 为排列矩阵， $X' = PX$ ，则 Q 、 K 、 V 同样被排列，最终有

$$\text{Attention}(PX) = P \text{Attention}(X). \quad (13.16)$$

模型会跟着词一起移动表示，却没有得到额外的先后顺序信息。因果掩码提供方向约束，位置表示进一步提供距离与位置特征。

绝对位置方法将词嵌入与位置向量相加： $x_t = e_{w_t} + p_t$ 。正弦位置编码使用不同频率：

$$p_{t,2i} = \sin(t/10000^{2i/d}), \quad p_{t,2i+1} = \cos(t/10000^{2i/d}). \quad (13.17)$$

不同频率让模型同时表达较短与较长的变化尺度。能够计算任意位置的编码，不等于模型一定能在训练长度以外可靠泛化。

RoPE 对查询与键的二维坐标对作旋转。二维旋转矩阵为

$$R_\theta = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}. \quad (13.18)$$

对位置 m, n 使用 $q'_m = R_m q_m$ 、 $k'_n = R_n k_n$ 。由于旋转矩阵的性质，

$$(q'_m)^\top k'_n = q_m^\top R_m^\top R_n k_n = q_m^\top R_{n-m} k_n. \quad (13.19)$$

分数中自然出现相对位置 $n-m$ 。多维情形对不同坐标对采用不同频率。RoPE 的基本构造见 [11]。

数学直觉 | 位置通过角度差进入匹配

两个向量都旋转相同角度时，夹角不变；旋转不同角度时，点积会随角度差变化。RoPE 利用这一点，把位置差融入内容匹配，而不是另外输出一个距离标签。

13.5 计算代价与实现检查

标准稠密自注意力的分数矩阵有 T^2 个元素，计算量约为 $O(T^2 d)$ 。线性投影和 FFN 通常有 $O(T d^2)$ 级别成本。哪部分更贵，取决于 T 、 d 、层宽和具体实现，不能只看到平方项就忽略其他计算。

FlashAttention 通过分块与融合计算，减少大注意力矩阵在显存中的存取。它计算的仍是精确注意力，主要改善 IO 与中间存储，并没有把完整稠密注意力的算术复杂度普遍变成线性 [12]。

算法实现 | 单头因果注意力

```
# X: [B, T, d]; projections are learned matrices.
Q, K, V = X @ Wq, X @ Wk, X @ Wv
scores = Q @ K.transpose(-1, -2) / math.sqrt(dk)
future = torch.triu(torch.ones(T, T, dtype=torch.bool,
                                device=X.device), diagonal=1)
scores = scores.masked_fill(future, float('-inf'))
weights = torch.softmax(scores, dim=-1)
output = weights @ V
```

这是便于理解的实现。大模型应使用经过优化的注意力算子，并核对其掩码约定、数据类型和 dropout 行为。

训练时整段真实前缀已知，因果掩码让所有位置并行计算损失。生成时，下一个输入还没有产生，所以 token 之间仍需按顺序生成。**训练可并行不等于自回归生成可一次完成。**

完整算例：三个位置的因果注意力

令输入矩阵为

$$X = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}, \quad (13.20)$$

并取 $W_Q = W_K = W_V = I$ 。于是 Q、K、V 都等于 X， $d_k = 2$ 。缩放前的点积矩阵为

$$QK^\top = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{pmatrix}. \quad (13.21)$$

除以 $\sqrt{2}$ 并加因果掩码后，第一行只有第一个位置可见，所以输出 $o_1 = (1, 0)$ 。

第二行可见两个位置，分数为 $(0, 1/\sqrt{2})$ 。设 $a = e^{1/\sqrt{2}} \approx 2.028$ ，权重为 $(1/(1+a), a/(1+a)) \approx (0.330, 0.670)$ ，输出为

$$o_2 \approx 0.330(1, 0) + 0.670(0, 1) = (0.330, 0.670). \quad (13.22)$$

第三行分数为 $(1/\sqrt{2}, 1/\sqrt{2}, 2/\sqrt{2})$ 。减去共同的 $1/\sqrt{2}$ 后，等价分数为 $(0, 0, 1/\sqrt{2})$ ，权重约为 $(0.248, 0.248, 0.504)$ 。因此

$$o_3 \approx 0.248(1, 0) + 0.248(0, 1) + 0.504(1, 1) = (0.752, 0.752). \quad (13.23)$$

每个输出都位于当前可见 value 的凸包内。但完整 Transformer 还会经过输出投影、残差与 FFN，最终层表示不受同一个简单凸包限制。

注意力怎样反向传播到匹配分数

对一个查询，令 $o = \sum_i a_i v_i$ ，上游梯度为 $g = \partial \ell / \partial o$ 。首先

$$\frac{\partial \ell}{\partial v_i} = a_i g, \quad \frac{\partial \ell}{\partial a_i} = g^\top v_i. \quad (13.24)$$

记分数为 s_i ， $a = \text{softmax}(s)$ 。利用 softmax Jacobian，可得

$$\frac{\partial \ell}{\partial s_i} = a_i g^\top (v_i - o). \quad (13.25)$$

如果某个 value 沿着有利于降低损失的方向不同于当前平均输出，模型就会调整相应权重。梯度不是单纯把所有权重都增大，而是在候选之间重新分配质量。

对 $s_i = q^\top k_i / \sqrt{d_k}$ ，继续得到

$$\frac{\partial \ell}{\partial q} = \frac{1}{\sqrt{d_k}} \sum_i \frac{\partial \ell}{\partial s_i} k_i, \quad \frac{\partial \ell}{\partial k_i} = \frac{1}{\sqrt{d_k}} \frac{\partial \ell}{\partial s_i} q. \quad (13.26)$$

至此，注意力如何学习“该匹配谁”就变成了完整的链式求导。

残差连接为何方便优化

对 $y = x + f(x)$, 有

$$\frac{\partial y}{\partial x} = I + \frac{\partial f}{\partial x}. \quad (13.27)$$

梯度可以沿恒等项传播, 也可以经过子层。它不保证深层训练永远稳定, 但提供了一条不必连续乘过所有非线性变换的直接路径。

LayerNorm 的位置影响这条路径。pre-norm 先归一化再进入子层, 残差主路保持较直接; post-norm 在相加后归一化, 梯度还要经过归一化变换。两者各有训练与表示上的特点, 不应只凭名称判断。

旋转位置的二维检查

令 $q = k = (1, 0)^\top$ 。位置 m 、 n 的旋转角分别为 θ_m, θ_n , 旋转后点积为 $\cos(\theta_n - \theta_m)$ 。同时给两个角度加常数, 点积不变; 增大角度差则改变匹配。

多频率 RoPE 把这种二维性质放在多个坐标对上。长上下文扩展要改变位置或频率尺度时, 需要检查模型是否仍能使用训练中学到的相位关系。公式可计算更长位置, 不代表长距离行为已经被训练好。

推演与自检

对长度为 3 的序列写出因果掩码。第一行只有第一个键可见, 第二行有两个, 第三行有三个。将最后一个输入 token 改成任意值, 前两个位置的输出应保持不变; 这是检查因果掩码的一项有意义的测试。

第 14 章 预训练：把文本变成监督信号

人工标签昂贵，原始文本却包含大量可利用的结构。一个句子中，前面的词可以帮助预测后面的词；被遮住的片段也可以由剩余内容推断。预训练把这些关系变成学习目标，让模型在进入具体任务前就获得通用表示。

这里的“自监督”并不是没有目标。目标由数据自身构造，而不是由人逐条标注。本章比较几种预训练方式，重点看模型能读取哪些位置、要预测哪些位置，以及训练后怎样使用。

14.1 上下文化表示与迁移学习

静态词向量给每个词一种固定表示。ELMo 则先训练正向和反向语言模型，再组合不同层的状态：

$$\text{ELMo}_t = \gamma \sum_{\ell=0}^L s_{\ell} h_{t,\ell}, \quad s_{\ell} = \frac{e^{a_{\ell}}}{\sum_j e^{a_j}}. \quad (14.1)$$

$h_{t,\ell}$ 可由该层正反向状态拼接而成， s_{ℓ} 与缩放参数 γ 由下游任务学习。不同任务可以偏好不同层的信息。

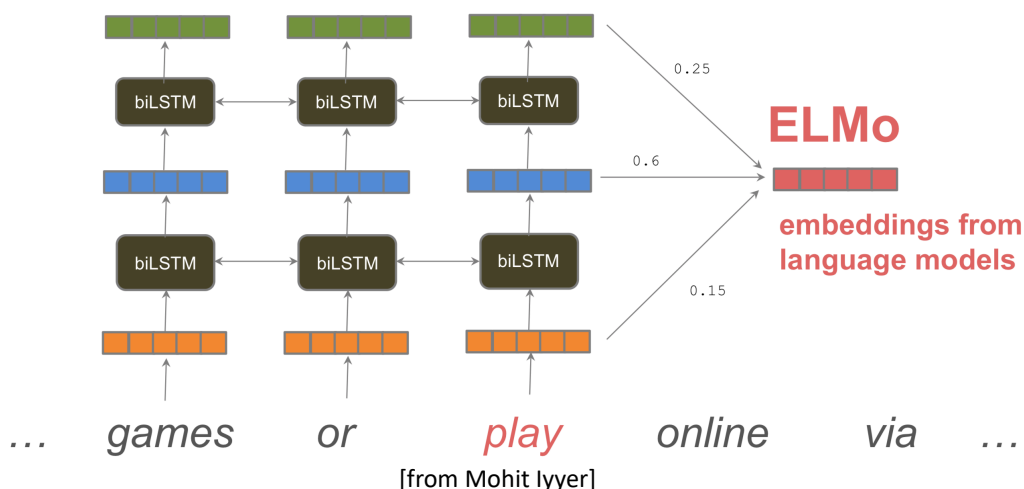


图 14.1 预训练产生多层上下文表示，下游任务学习如何组合这些层。（课件 lec11_pretraining, 第 5 页）

正向语言模型只看过去，反向模型只看未来。它们是两个方向分别训练的模型，不等同于在同一层中让一个目标 token 同时读取两侧内容的深度双向注意力。

预训练模型可作为冻结的特征提取器，也可以整体微调。冻结时主要训练任务头，成本较低，底层知识不容易被小数据改坏；微调时表示也随任务改变，通常更灵活，但要控制学习率、训练步数与过拟合。

迁移究竟迁移什么

预训练并不只记住词频。为了预测文本，模型可能需要表示句法、指代、语义关系与领域知识。但训练目标只奖励预测能力，并没有直接保证模型具有某一种人类式概念结构。

例如，模型可能根据句法正确预测动词形式，也可能利用常见搭配猜对。同样的正确答案可以来自不同机制，因此需要有针对性的测试才能判断学到了什么。

14.2 BERT：从双侧语境恢复被选中的词

BERT 使用编码器式 Transformer，允许输入位置彼此读取。若直接让位置 t 预测原本就在输入中的 token，就会出现复制捷径。因此要破坏部分输入，再训练模型恢复目标。

设被选中的位置集合为 $\mathcal{M}$ ，损坏后的输入为 $\tilde{x}$ ，MLM 目标为

$$\mathcal{L}_{\text{MLM}} = - \sum_{t \in \mathcal{M}} \log p_{\theta}(x_t | \tilde{x}). \quad (14.2)$$

原始 BERT 选择约 15% 的位置参与预测；这些位置中约 80% 换成 [MASK]，10% 换成随机 token，10% 保持原词。保持原词的位置仍参与目标损失。这是一个具体的损坏方案，不能把它当作所有 MLM 的固定定义。

数学直觉 | 把容易复制的任务变成补全任务

模型看到“巴黎是法国的 [MASK]”，需要利用左右语境恢复“首都”。为了完成大量这样的任务，它必须学会使用上下文。不过，恢复语料中的词并不等于验证该句是否符合现实。

BERT 的输入还可包含句段标记与特殊 token。[CLS] 常用于汇总分类，[SEP] 标出片段边界。原始 BERT 同时使用下一句预测 NSP；后续 RoBERTa 的训练方案移除了 NSP，并调整数据、批量和训练方式。不能把两者的整体差异全部归因于删除一个目标。

微调到不同输出空间

句子分类在 [CLS] 表示上接分类头；序列标注在每个 token 上接标签头；抽取式问答预测答案起点和终点。以起止位置为例：

$$p_{\text{start}}(i | x) = \text{softmax}_i(w_s^{\top} h_i), \quad p_{\text{end}}(j | x) = \text{softmax}_j(w_e^{\top} h_j). \quad (14.3)$$

解码时通常要求 $j \geq i$ ，并限制最大跨度长度。起点与终点各自最高概率的位置，不一定构成合法答案。

MLM 的概率含义

MLM 学习的是不同损坏条件下的恢复分布。将每个位置在其他词已知时的概率相乘，可构造伪似然评分；它一般不是一个已归一化的完整句子联合分布。因此不能直接把 MLM 的预测概率按自回归方式拼成标准句子概率。

sentence-level tasks

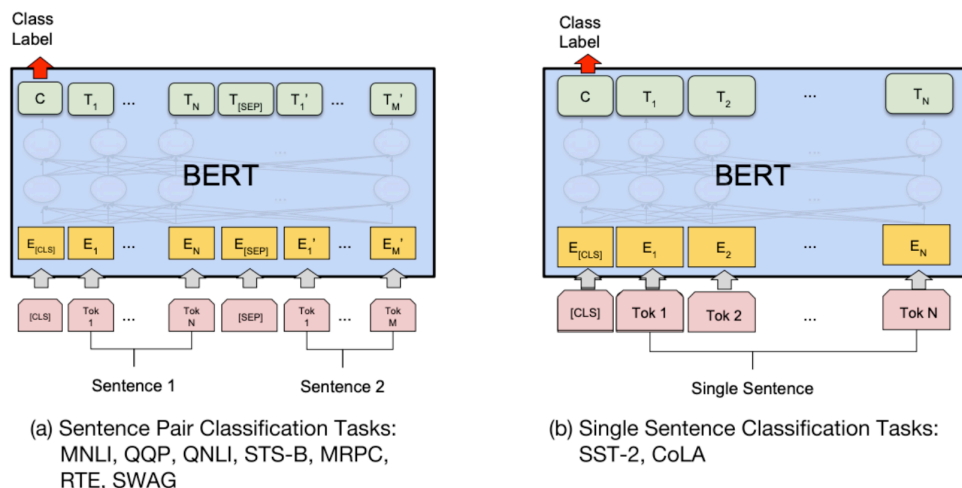


图 14.2 相同的预训练编码器可连接不同任务头。输入格式和输出损失随任务改变。（课件 lec11_pretraining, 第 31 页）

14.3 GPT 与 T5：两条生成式路线

解码器式语言模型使用因果掩码，训练目标为

$$\mathcal{L}_{\text{AR}} = - \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t}). \quad (14.4)$$

输入位置 t 的表示预测下一 token，所有位置的训练标签来自右移后的同一段文本。每个有效位置都可贡献损失，但“预测位置多”并不直接证明它在所有任务上比 MLM 更有效。

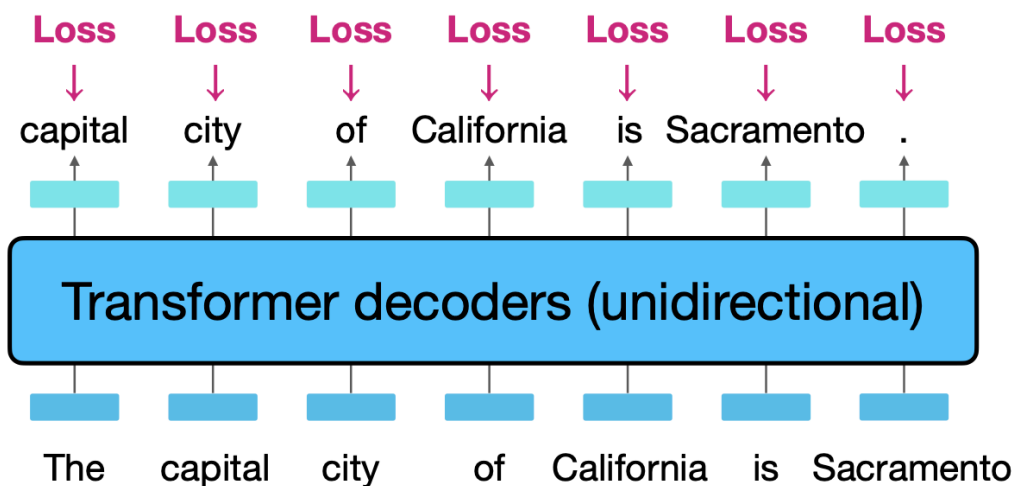


图 14.3 自回归预训练用前缀预测下一 token。训练时目标已知，因果掩码防止答案泄漏。（课件 lec11_pretraining, 第 52 页）

GPT 一类模型可以通过微调适应任务，也可以把任务描述和示例放进上下文直接生成答案。后者不更新参数，属于上下文学习。预训练目标没有直接要求模型“礼貌回答问题”，因此基础模型可能续写问题、模仿论坛讨论，或生成与用户意图不符的文本。

T5 使用编码器解码器架构，并把任务统一成文本到文本。分类标签也可以写成短字符串，翻译与摘要则直接输出目标文本。

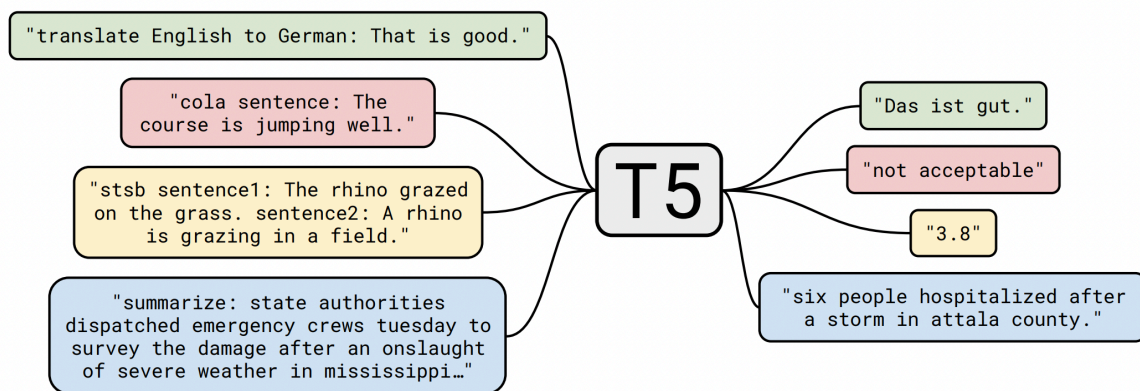


图 14.4 不同任务用文本前缀指定，输出统一为文本。模型结构共享，任务语义仍由数据和目标决定。（课件 lec11_pretraining，第 45 页）

其预训练可使用跨度损坏：将若干连续片段替换成不同哨兵 token，解码器依次生成缺失片段。例如

输入：北京是 <extra_id_0> 的首都，位于 <extra_id_1>。

目标：<extra_id_0> 中国 <extra_id_1> 华北 <extra_id_2>

哨兵标记区分缺失片段，解码器不必复制全部未损坏文本。具体片段长度和损坏比例会改变任务难度。

架构	可见条件	常见预训练目标	典型用途
编码器	完整损坏输入	遮蔽词恢复	表示、分类、抽取
解码器	当前目标的左侧	下一 token 预测	开放式生成、续写
编码器解码器	完整源输入与目标前缀	去噪、跨度恢复	条件生成、翻译

表 14.1 架构、注意力可见范围和训练目标要一起比较。

14.4 训练目标与下游能力之间的距离

语言模型的交叉熵是清晰的数学目标，实际用户任务却可能要求事实正确、遵循格式、完成操作或在信息不足时拒绝猜测。这些要求并不都能由“模仿语料中的下一词”充分表达。

预训练、监督微调 and 偏好训练分别改变模型的不同方面，但边界不是绝对的。预训练可包含指令式文本，微调也可能引入新知识。说“SFT 只教格式，永远不学知识”过于绝对；更准确的说法是，小规模 SFT 常主要调整模型已有能力的使用方式，但结果取决于数据与训练强度。

负迁移与遗忘

如果微调语料很窄，模型可能提高该领域表现，却降低其他任务表现。这既可能来自参数更新覆盖旧能力，也可能来自输出分布变得过于专一。较小学习率、混合通用数据、参数高效微调和保留独立能力集，都是可检查的缓解方式。

评价迁移效果时，应同时保留目标任务和通用能力的测试。只在新领域上报告提升，会看不到代价；只看平均分，也可能掩盖少数关键能力的下降。

算法实现 | 三类目标的标签位置

```
# Autoregressive LM
inputs = tokens[:-1]
labels = tokens[1:]

# Masked LM
inputs = corrupt(tokens, selected_positions)
labels = ignore_everywhere_except(tokens, selected_positions)

# Span denoising
encoder_input, decoder_target = split_by_sentinels(tokens)
decoder_input = shift_right(decoder_target)
```

这三种目标都可能使用交叉熵，但它们的条件信息和监督位置不同。检查数据构造往往比检查损失函数名称更重要。

同一句话怎样形成三种训练样本

考虑五个 token：甲、喜欢、学习、语言、模型。自回归训练可以写为

$$(\text{BOS}, \text{甲}, \text{喜欢}, \text{学习}, \text{语言}, \text{模型}) \rightarrow (\text{甲}, \text{喜欢}, \text{学习}, \text{语言}, \text{模型}, \text{EOS}). \quad (14.5)$$

每一列输入预测右侧对应目标，位置越靠后可见的前缀越长。预测“学习”时不能看到“语言”和“模型”。

MLM 若选中“学习”，输入可以是“甲喜欢 [MASK] 语言模型”，只在选中位置监督“学习”。它允许读取右侧“语言模型”，任务难度和自回归预测不同。

跨度损坏若遮住“学习语言”，输入是“甲喜欢 <s0> 模型”，目标是“<s0> 学习语言 <s1>”。编码器读取剩余源文本，解码器按顺序恢复缺失跨度。

这些例子说明，三种训练方式都可使用同一条原始文本，却产生不同条件和不同目标。比较收敛速度时，仅比较优化步数并不公平，还应看每步处理的 token、监督位置及计算量。

MLM 损失为什么不能直接当句子概率

设只有两个二元变量 x_1, x_2 ，某系统分别估计 $p(x_1 | x_2)$ 和 $p(x_2 | x_1)$ 。将二者相乘得到的数一般不在全部四种输入上归一化。

例如两个条件分布都强烈偏好两变量相同：相同概率为 0.9，不同为 0.1。乘积对 00、11 各为 0.81，对 01、10 各为 0.01，总和为 1.64，不是 1。

这些条件分布可以用于恢复任务，也可以构造伪似然分数，但不能未经归一化就声称得到联合分布。自回归链式分解则按一个方向排列条件，具有不同的概率结构。

预训练损失的加权方式

一个批次含两篇文本，第一篇有 10 个目标 token，平均损失为 1；第二篇有 90 个，平均损失为 3。token 平均损失为

$$\frac{10 \times 1 + 90 \times 3}{100} = 2.8. \quad (14.6)$$

逐文档平均再平均则是 $(1 + 3)/2 = 2$ 。如果训练框架与日志使用不同分母，学习率调节和模型比较就会被误导。

MLM 中各样本被选中的位置数也可能不同。应明确分母是被监督 token 数、原序列长度，还是样本数。目标名称相同，不代表权重口径相同。

选择题概率与生成答案不完全相同

模型可能给正确选项内容较高概率，却不善于输出指定字母；也可能知道字母映射，却偏爱某个位置。评价时可以直接比较 A、B、C、D 的条件概率，也可以让模型生成完整答案再解析，但两者测到的流程不同。

如果选项长度差异较大，用完整选项序列概率比较会引入长度影响。通过标签字母评分减少了这种差异，却又依赖提示中的字母映射。公平比较要求把这些细节固定下来。

冻结编码器与全量微调的对照

用同一预训练模型，先冻结编码器只训练线性头，再允许更新最后几层，最后全量微调。若冻结表示已经很好，说明预训练特征可直接分离任务；若只有全量微调显著改善，任务可能需要改变表示。

这个对照也能帮助发现数据不足的问题。全量微调训练分数很高、验证分数较低，而冻结特征较稳定时，可考虑更强正则化、更少更新层或更多数据，而不是单纯继续训练。

推演与自检

如果在抽取式问答中把完整答案直接追加到编码器输入，再预测答案位置，模型可能得到很高分，却没有完成原任务。判断是否泄漏，应从任务定义出发：测试时真正允许提供的信息是什么？预训练与微调都需要同样的检查。

第 15 章 分词与训练数据：模型实际读到了什么

模型不会直接读取网页或人类眼中的句子。网页先被抽取成文本，文本再被切成 token，最后变成整数序列。这个过程如果损坏了段落、公式或代码结构，再大的模型也只能学习损坏后的材料。

本章从分词讲到语料清洗。一个共同问题贯穿始终：我们怎样把有限的训练预算分配给真正有用的信息，而不是重复、噪声和格式残片？

15.1 字符、词与子词

整词词表直观，却容易遇到新词和形态变化。字符词表覆盖强，但序列长。子词分词在两者之间折中：常见片段合成较长 token，罕见词拆成较短片段。

词表大小 V 与平均序列长度 T 相互影响。增大词表常能缩短序列，却增加嵌入和输出层参数；减小词表可以共享更多片段，却增加注意力和生成步数。没有一个对所有语言和领域都最优的词表大小。

BPE 的学习过程

BPE 从基本符号开始，反复合并语料中最常见的相邻符号对。一个简化流程是：统计词频；将每个词拆为字符及边界标记；统计相邻对的加权频次；合并最高频的一对；重复到词表达到目标规模。

假设语料包含 low、lower、lowest。最初它们共享 l、o、w，合并 l 与 o 后得到 lo，再与 w 合并成 low。这样，lower 可以表示为 low 与 er，而无需把所有变化形式分别保存为整词。

训练分词器得到的是一个有顺序的合并规则表。推断时应遵循合并优先级，不能每遇到一个新文档就重新统计最常见相邻对。否则同一段文字会得到不稳定的编码。

WordPiece 与 BPE 都产生子词，但训练准则和匹配规则并不完全相同。Unigram 分词模型则为候选子词赋概率，并在多种切分之间比较概率。它们共享“用有限词表覆盖大量字符串”的目标，不能只凭输出看起来像子词就当成同一种算法。

字节覆盖与可逆性

字节级方案可以覆盖任意 UTF-8 文本，但一个汉字或罕见字符可能被拆成多个 token。可逆分词要求解码后恢复原始字节或约定的标准化文本；如果预处理做了小写化、空白折叠或 Unicode 归一化，就要明确这些变化是否可逆。

算法实现 | BPE 的教学骨架

```
while len(vocab) < target_size:
    pair_count = Counter()
    for symbols, frequency in word_types.items():
        for pair in zip(symbols, symbols[1:]):
            pair_count[pair] += frequency
    if not pair_count:
        break
    pair = max(pair_count, key=pair_count.get)
    word_types = merge_pair_everywhere(word_types, pair)
    merge_rules.append(pair)
    vocab.add(pair[0] + pair[1])
```

真实实现还要处理空白、Unicode、字节编码和特殊 token。合并后的相邻对计数也必须正确更新。

数学直觉 | token 是编码单位，不是固定的语义单位

一个 token 可能是完整词、半个词、空格加词片段，甚至一个字节片段。模型说“预测下一个 token”，不等于每一步都在预测一个自然语言词。

15.2 文本抽取与线性化

训练材料可以来自网页、书籍、代码、论文与其他文档。不同格式要先转成模型可读的序列。网页的正文、菜单、广告和脚本并不具有相同价值；PDF 的双栏排版和表格，也不能随意按屏幕坐标拼接。



图 15.1 数据处理通常包含采集、线性化和筛选。每一步都会改变最终的训练分布。（课件 lec13_data，第 23 页）

抽取过程中常见的错误包括：把页眉插进句子，打乱表格行列，把公式拆成无意义字符，丢失代码缩进，或把导航栏当作正文反复保留。这些不是表面排版问题，而是直接改变了模型接收到的语言规律。

应按内容类型保留结构。代码需要缩进与边界，问答需要角色和轮次，表格需要明确的行列关系。统一成纯文本并不意味着把所有结构都删除，而是选择一种稳定的序列化方式。

过滤顺序为什么影响成本

简单规则通常很便宜，可先移除空文档、明显乱码和极短噪声。去重减少后续需要评分的文本量。较昂贵的模型质量评分再作用于剩余材料。这种顺序通常节省计算，但具体流程仍要看任务：过早使用粗糙过滤器，也可能删掉稀有但有价值的数据。

网页归档中，WARC 保留抓取的原始响应与元数据，WET 提供抽取后的文本。使用哪种格式会影响可控制的清洗步骤。课件中的数据工作说明，抽取器本身就可能带来可测的训练差异，而不只是换一个文件读取库。

15.3 去重、质量筛选与数据混合

精确去重可以对标准化文本计算哈希。近重复去重则需要比较局部片段。把文档表示为 n -gram 集合 A, B ，Jaccard 相似度为

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}. \quad (15.1)$$

它衡量共享片段占全部不同片段的比例。网页模板可能导致很高重叠，因此是否先去模板、按整篇还是按段落比较，会明显影响结果。

MinHash 用随机置换下的最小哈希值构造签名。理想随机置换下，两个集合最小元素相同的概率等于 Jaccard 相似度。使用 m 个独立签名时，可用相同位置比例估计相似度：

$$\hat{J} = \frac{1}{m} \sum_{r=1}^m \mathbf{1}[h_r(A) = h_r(B)]. \quad (15.2)$$

局部敏感哈希可先找近重复候选，再用更精确的方法确认。哈希碰撞、签名长度和阈值都会影响误删与漏删。

备注 | 重复并非只有一种含义

同一本书的多个副本通常没有提供同等数量的新信息；但关键事实在不同文体中的自然重复，可能帮助学习。去重的目标是减少冗余和泄漏，不能简单等同于删除所有相似表达。

质量筛选可以使用启发式规则，也可以训练分类器。模型评分器往往学习“像高质量参考语料的程度”，这个分数不等于客观质量。若正例主要来自标准书面语，方言、少数语言和特殊领域文本可能被误判为低质量。

混合比例就是训练分布

设数据源 s 的采样权重为 α_s ，则训练分布为

$$p_{\text{train}}(x) = \sum_s \alpha_s p_s(x), \quad \sum_s \alpha_s = 1. \quad (15.3)$$

参数更新主要服务于这个混合分布，而不是抽象的“所有语言”。提高代码数据比例可能改善代码任务，也可能挤占其他内容的训练预算，需要实测权衡。

为了避免大语种完全支配训练，可以按 n_s^τ 设采样权重，其中 n_s 为各源规模、 $0 < \tau < 1$ 。较小的 τ 使分布更平坦，但也会更频繁重复小数据源。

重复轮数与有效信息

若某源只有 D_s 个 token，却在总训练中被采样 M_s 次 token，它的平均重复轮数约为 M_s/D_s 。增加训练 token 数既可能意味着读到更多新文本，也可能只是反复读旧文本。两种情况的收益不能混为一谈。

15.4 数据泄漏、合成数据与样本打包

测试污染指测试内容或其近似版本在预训练、微调或其他拟合阶段出现。它不只包括逐字复制，还包括答案解析、翻译和改写。模型在污染测试上表现好，不能直接证明能解决新的同类问题。

应在数据管线中保留来源与处理记录，并按文档、时间或来源组划分。去重时不仅比较训练内部，也要检查训练与验证 / 测试之间的交叉重复。对于无法完全掌握来源的基础模型，要在结果解释中保留这种不确定性。

合成数据可以补充特定任务，但生成数量不是唯一指标。若教师反复输出同一种模板，学生可能学到格式而非能力。有效的合成过程通常包括任务多样化、答案核验、去重和对困难样本的筛选。

数学直觉 | 用模型生成数据仍需要外部约束

模型可以批量给出候选题目和答案，但不能仅凭自己流畅的解释确认答案正确。数学可用符号或数值检查，代码可运行测试，事实任务可对照可靠资料。验证器决定哪些生成内容值得学习。

训练时常把短文档打包进固定长度块，以减少 padding。这里有两个独立选择：是否插入文档结束符，是否允许跨文档注意力。若任务要求独立文档训练，应使用相应的分段掩码；仅有结束符并不自动阻止模型读取上一文档。

损失掩码也不同于注意力掩码。某个 token 可以不计入损失，却仍作为上下文被读取。SFT 中的用户指令常是这种情况。二者混淆，会导致模型看到答案或忽略必要条件。

完整算例：带词频的 BPE 合并

语料只有 banana 出现 2 次、band 出现 1 次。暂不考虑词尾标记，只在词内合并。初始相邻对计数为

$$C(ba) = 3, \quad C(an) = 5, \quad C(na) = 4, \quad C(nd) = 1. \quad (15.4)$$

最高频 an 合并后，banana 变为 $[b, an, an, a]$ ，band 变为 $[b, an, d]$ 。下一轮相邻对计数为

$$C(b, an) = 3, \quad C(an, an) = 2, \quad C(an, a) = 2, \quad C(an, d) = 1. \quad (15.5)$$

再合并 b 与 an，得到 $[ban, an, a]$ 与 $[ban, d]$ 。相同字符片段因为共享统计而进入词表。

若忽略词频、把每个不同词都只算一次，第一轮计数就会改变。BPE 统计的是语料中的出现次数，不只是词典中存在多少个词形。

切分不同，token 级困惑度怎样变化

假设某段文本的总负对数似然为 20。分词器 A 把它切成 10 个 token，则 token 级 PPL 为 e^2 ；分词器 B 切成 20 个 token，在假设总负对数似然仍为 20 的比较下，PPL 为 e。

后者数字更小，却没有减少这段文本的总编码代价。这个简化例子强调分母单位的作用。真实模型和分词器改变时，总负对数似然本身也会改变，更需要统一按字符或字节比较。

近重复阈值的具体含义

文档 A 的片段集合为 $\{a, b, c, d\}$ ，B 为 $\{a, b, c, e\}$ 。交集大小为 3，并集为 5，Jaccard 为 0.6。若使用“共享片段占较短文档的比例”，结果却为 $3/4 = 0.75$ 。

两个指标都可用于去重，但阈值 0.7 的意义不同。数据管线中只写“重叠超过 70% 就删除”并不完整，还必须说明分母、片段长度和比较粒度。

如果 B 是 A 的完整子集，Jaccard 可能因 A 很长而较低，包含率则为 1。网页转载、节选与模板重复需要不同检测策略。

采样比例怎样改变小语种的重复次数

两种语言分别有 9000 万和 1000 万 token。按数据量采样时比例为 0.9 : 0.1。若采用平方根温度采样，即 $\tau = 1/2$ ，权重比为 $\sqrt{9} : \sqrt{1} = 3 : 1$ ，归一化后为 0.75 : 0.25。

总训练读取 1 亿 token 时，小语种平均被读 2.5 轮，大语种约 0.833 轮。小语种获得更多更新机会，但重复也更明显。语言平衡与新信息覆盖需要同时考虑。

合成数据的覆盖与验证

假设需要训练一个算术问答模型。只让教师生成“两个小整数相加”的题，即使生成百万条，也没有覆盖减法、进位、多步运算和单位转换。

更好的构造先定义题型与难度，再生成候选，最后独立计算答案。还可以记录解题所需步骤数、数值范围与错误类型，用来检查训练与测试分布。合成数据的优势在于可控制，而不是只在于生成速度快。

推演与自检

一个语料库含 100 万 token，其中 90 万来自同一模板重复。另一个只有 50 万 token，但内容较丰富。仅凭规模不能断定前者更好。可以固定训练算力，对比验证损失、重复率和分领域任务表现，判断新增文本是否提供了新的学习信号。

第 16 章 规模、优化与训练成本

把模型扩大并不只是增加几层网络。更多参数需要更多数据和计算，训练还要容纳梯度、优化器状态与中间激活。预算固定时，扩大模型可能迫使我们减少训练 token，结果未必更好。

本章把规模问题写成资源分配问题，再介绍优化与系统实现中的关键环节。目的不是记住某个硬件配置，而是学会估计成本，并判断训练曲线背后的原因。

16.1 缩放规律与计算预算

一种常见的经验形式把验证损失写成

$$L(N, D) \approx L_{\infty} + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}, \quad (16.1)$$

其中 N 是参数量， D 是训练 token 数， L_{∞} 是拟合中的不可约项。 A, B, α, β 从特定实验范围估计，不是对所有模型和数据都相同的常数。

对稠密 Transformer，训练计算量常粗略估为

$$C \approx 6ND. \quad (16.2)$$

它把前向与反向的主要矩阵计算合并估计。长序列注意力、嵌入、重计算和系统开销可能让实际成本偏离。这个式子适合做第一轮预算判断，不适合作为精确的训练账单。

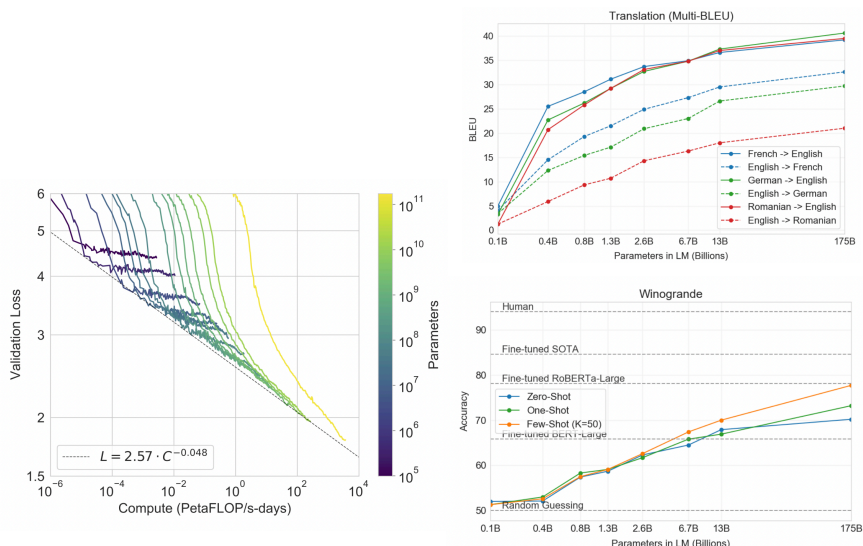


图 16.1 增加参数、训练数据与计算后，验证损失和下游任务表现通常呈现不同形状的变化。具体曲线依赖实验设置。（课件 lec11_after-gpt2，第 15 页）

固定预算 C 后， $D = C/(6N)$ ，于是

$$L(N) = L_{\infty} + AN^{-\alpha} + B(6N/C)^{\beta}. \quad (16.3)$$

第一项随模型增大下降，第二项因为可读数据减少而上升。对 N 求导并令其为零：

$$\alpha AN^{-\alpha} = \beta B(6N/C)^{\beta}. \quad (16.4)$$

解得尺度关系

$$N^* \propto C^{\beta/(\alpha+\beta)}, \quad D^* \propto C^{\alpha/(\alpha+\beta)}. \quad (16.5)$$

若两个指数接近，最优参数与数据规模都大致按预算的平方根增长。计算最优训练的实证研究见 [13]；具体比例需要根据任务和数据重新验证。

数学直觉 | 更大与练得更充分之间的选择

固定时间里，既可以训练一个较大的模型较少步，也可以训练一个较小的模型较多步。前者容量更大，后者看过更多样本。最优点取决于两种收益的下降速度。

训练最优不等于部署最优

如果模型只训练一次，却要调用很多次，总成本还包括推理：

$$C_{\text{total}} = C_{\text{train}} + Q C_{\text{infer}}, \quad (16.6)$$

其中 Q 为预计调用次数。训练更久的小模型可能在部署中更划算。选择架构和训练 token 数时，应明确优化的是一次训练预算，还是完整使用周期。

16.2 梯度、批量与 AdamW

最小批量梯度是全数据梯度的随机估计。批量较小，噪声通常较大；批量很大时，每步估计更稳定，但相同 token 预算下更新次数更少。学习率与批量大小需要一起考虑。

Adam 维护梯度的一阶和二阶滑动平均：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (16.7)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (16.8)$$

$$\hat{m}_t = m_t / (1 - \beta_1^t), \quad \hat{v}_t = v_t / (1 - \beta_2^t). \quad (16.9)$$

随后按坐标缩放更新：

$$\theta_{t+1} = \theta_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \quad (16.10)$$

平方和除法均逐元素进行。二阶平均不是 Hessian，它是梯度平方的统计量。

AdamW 将权重衰减与自适应梯度分开：

$$\theta_{t+1} = (1 - \eta_t \lambda) \theta_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \quad (16.11)$$

在自适应优化器下，把 L_2 惩罚直接加进梯度与这种衰减并不一般等价，因为前者也会经过各坐标的自适应缩放。

预热与学习率下降

训练初期，优化器统计量和激活尺度尚未稳定，常用预热让学习率逐步上升。随后可按余弦曲线下降：

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left[1 + \cos \left(\pi \frac{t - t_w}{T - t_w} \right) \right]. \quad (16.12)$$

此式适用于预热结束后的 $t \in [t_w, T]$ 。调度器应与实际优化器更新次数一致，而不是误按梯度累积中的每个微批次推进。

梯度累积与有效批量

若每个设备微批量为 b ，累积 a 次，有 G 个设备，则全局样本批量为 $B_{\text{global}} = b \times a \times G$ 。语言模型更适合按有效 token 数统计批量，因为不同样本长度可能不同。

为了使累积等价于一个大批量，损失应按总有效 token 数归一化。对不同长度微批次分别取平均，再平均各微批次，会给短批次中的 token 更大权重。只有有效长度相同等条件下，两种写法才等价。

16.3 显存由哪些部分构成

训练显存主要包含参数、梯度、优化器状态和激活。以一种常见混合精度配置作粗估：低精度参数 2 字节，低精度梯度 2 字节，FP32 主参数 4 字节，Adam 两组 FP32 状态 8 字节，总计约 16 字节 / 参数，尚未包括激活、临时缓冲和碎片。

不同框架可能省略主参数、采用不同梯度精度或分片状态，因此这个数不是固定规范。对 10 亿参数，上述配置仅这些持久状态就约 16 GB（十进制），训练还需要更多空间。

混合精度用较低精度做大量矩阵乘法，同时让敏感累积保持较高精度。FP16 与 BF16 的指数范围不同，是否需要损失缩放也取决于数据类型和实现。不能只看位数相同就把两者当成完全等价。

激活检查点只保留部分中间结果，反向时重新计算其他结果。它以额外计算换显存。序列更长时激活成本上升，这个交换常常有用。

方法	分担的对象	主要代价
数据并行	不同样本的计算	同步梯度
张量并行	单层矩阵运算	层内通信
流水线并行	不同网络层	流水线空泡与调度
状态分片	参数、梯度或优化器状态	必要的聚合与通信
激活重计算	中间激活存储	额外前向计算

表 16.1 并行和节省显存的方法可以组合，但通信与调度也会成为瓶颈。

16.4 稀疏专家与训练监控

混合专家模型 MoE 为部分层设置多个 FFN 专家，路由器给每个 token 选择少数专家。一个简化表达是

$$y(x) = \sum_{e \in \text{TopK}(g(x))} \tilde{g}_e(x) f_e(x), \quad (16.13)$$

$g(x)$ 是路由分数， $\tilde{g}$ 是选中专家后的归一化权重。总参数量可以很大，而每个 token 只激活其中一部分。

总参数和激活参数应分别报告。专家之间负载不均会让少数设备过忙、其他设备空闲。负载均衡损失、容量限制和路由设计都影响训练。稀疏计算也不等于免费扩容，跨设备传输 token 可能带来明显通信成本。

数学直觉 | 更多专家提供容量，路由决定怎么用

把很多专家放进模型，只是增加了可用函数。每个 token 选择哪些专家、各专家能否获得足够训练，才决定这些容量是否真正发挥作用。

训练监控不能只看总损失。还应观察梯度范数、学习率、有效 token 吞吐、激活异常、不同数据源的验证损失和检查点恢复情况。如果损失突然下降，应排查数据泄漏或目标错位；如果突然升高，应检查学习率、异常批次和数值溢出。

小规模试验怎样帮助大规模训练

在扩大训练前，可以先验证：一小批数据是否能够被过拟合；因果掩码和标签移位是否正确；不同设备的梯度平均是否一致；检查点保存后是否能继续同一训练。小试验不是用来证明最终能力，而是降低基础错误被放大的风险。

算法实现 | 按 token 累积的训练逻辑

```
optimizer.zero_grad()
for microbatch in accumulation_group:
    logits = model(microbatch.inputs)
    loss_sum = token_cross_entropy(logits, microbatch.labels,
                                   reduction="sum")
    (loss_sum / group_valid_tokens).backward()
clip_grad_norm_(model.parameters(), max_norm)
optimizer.step()
scheduler.step()
```

分布式框架可能自动平均梯度，必须据此调整归一化，避免重复除以设备数。日志中的损失分母也应采用同一有效 token 口径。

预算算例：参数、token 与设备时间

设稠密模型有 $N = 7 \times 10^9$ 参数，计划读取 $D = 1.4 \times 10^{11}$ token。用 $6ND$ 粗估：

$$C \approx 6 \times 7 \times 10^9 \times 1.4 \times 10^{11} = 5.88 \times 10^{21} \text{ FLOPs.} \quad (16.14)$$

假设每个设备的持续有效吞吐为 10^{14} FLOP/s，32 个设备理想地线性扩展，则所需时间约为

$$\frac{5.88 \times 10^{21}}{32 \times 10^{14}} \approx 1.84 \times 10^6 \text{ s} \approx 21.3 \text{ 天.} \quad (16.15)$$

这是用于理解量纲的假设算例，不是某种硬件的实测承诺。通信、检查点、重计算和故障恢复都会增加实际时间。

如果持续有效吞吐只达到假设值的一半，时间就翻倍。预算应使用测量或合理估计的持续吞吐，而不是直接采用设备峰值。

Adam 第一步为什么需要偏置校正

设初始 $m_0 = v_0 = 0$ ，梯度 $g_1 = 2$ ， $\beta_1 = 0.9, \beta_2 = 0.999$ 。则

$$m_1 = 0.2, \quad v_1 = 0.004. \quad (16.16)$$

校正后 $\hat{m}_1 = 2$ 、 $\hat{v}_1 = 4$ 。忽略 ϵ ，自适应更新方向大小为 $2/\sqrt{4} = 1$ 。若学习率为 0.1，参数从 1 更新到 0.9。

若还有 AdamW 衰减 $\lambda = 0.01$ ，则先前参数乘 $1 - 0.1 \times 0.01 = 0.999$ ，再减 0.1，得到 0.899。这个小例子有助于检查代码中的校正、衰减和学习率顺序。

梯度累积中的长度偏置

两个微批次分别有 2 个和 8 个有效 token，平均损失为 1 和 3。按 token 平均应得到

$$\frac{2 \times 1 + 8 \times 3}{10} = 2.6. \quad (16.17)$$

如果简单对两次平均损失各乘 1/2，得到 2。短微批次中每个 token 的权重变成了长微批次的四倍。

这不是单纯的日志差异，梯度也会按不同权重累积。使用变长样本和 packing 时，应明确全局有效 token 数如何计算，以及分布式梯度平均是否又引入一次缩放。

参数预算与激活预算的不同增长

扩大词表主要增加嵌入和输出矩阵；扩大层数同时增加参数与各层激活；扩大序列长度不增加模型参数，却增加激活和注意力成本。仅用“模型有多少 B 参数”无法描述训练显存。

对固定参数模型，批量与长度也不是完全可互换的。相同总 token 数下，少量长序列的注意力平方项可能比大量短序列更昂贵。若长度分别为 $T_1, \dots, T_B$ ，注意力工作量更接近 $\sum_i T_i^2$ ，而不是只由 $\sum_i T_i$ 决定。

如何读训练与验证曲线

训练损失持续下降、验证损失上升，提示对训练分布的拟合开始损害泛化。两者都高而下降缓慢，可能是优化设置不合适，也可能是模型容量或数据质量限制。验证损失突然异常低，则应检查是否与训练重复。

改变一个因素后，应同时比较按更新步、按已读 token 和按实际时间绘制的曲线。某方法每步更有效，却每步慢很多，未必在固定时间预算下更好。

推演与自检

若模型参数翻倍、训练 token 减半， $6ND$ 估计的计算量不变。但激活内存、矩阵乘法效率和最终损失都不一定不变。这个例子说明：粗略预算公式用于筛选方案，最终决策仍需实验与系统测量。

第 17 章 提示与上下文学习

预训练模型已经学会根据前缀预测文本。提示利用这一点，把任务说明、输入和示例放进前缀，让模型的续写更接近我们想要的答案。参数不变，条件信息却变了。

理解提示时，既不能把它看成神奇咒语，也不能把它简化为无关紧要的措辞。任务定义、示例选择、输出格式和评价协议，都可能通过条件分布影响结果。

17.1 把任务写成条件生成

设任务输入为 x ，提示模板为 $T(x)$ ，答案为 y 。提示式预测使用

$$p_{\theta}(y | T(x)). \quad (17.1)$$

对于情感分类，可以把评论放在“判断以下评论的情感，只输出正面或负面”后面。模板明确了要做什么，标签词则规定答案如何编码。

如果使用标签词映射 $v(c)$ ，可比较

$$\hat{c} = \arg \max_c \log p_{\theta}(v(c) | T(x)). \quad (17.2)$$

标签词可能包含多个 token，此时需要累加各 token 的条件对数概率。若标签长短不同，直接比较序列概率可能偏向短标签；若改用长度归一化，则又改变了评分目标，应明确约定。

一个明确的抽取提示

对“张三于 2024 年加入甲公司”，任务可以写为：抽取 person、organization、year；只使用原文出现的信息；缺失字段填 null；输出固定 JSON 字段。这里真正有用的是输出对象和证据范围，而不是增加很多“请认真”“你很专业”之类的修饰。

结构化输出最好配合解析器或语法约束。仅要求“输出 JSON”仍可能产生多余解释、缺失字段或类型错误。解析失败应作为系统失败记录，不能在评测时悄悄丢弃。

数学直觉 | 提示改变模型面对的问题

“继续这篇文章”与“从这篇文章中提取三个事实”可以使用同一模型和同一原文，却对应不同条件分布。清楚的提示首先是在减少任务歧义。

17.2 少样本示例与上下文学习

把示例 $D_c = \{(x_i, y_i)\}_{i=1}^k$ 拼入上下文，得到

$$p_{\theta}(y | D_c, x). \quad (17.3)$$

这称为上下文学习 ICL。推断时没有梯度更新，示例通过注意力和隐藏状态影响输出。与此相对，微调通过优化步骤改变模型参数。



图 17.1 微调用示例更新参数，上下文学习把示例留在当前输入中。（课件 lec11_after-gpt2，第 12 页）

示例可以传达多种信息：标签含义、任务格式、输入分布、边界情况和解题方式。如果任务标签是任意符号，例如 A 表示正面、B 表示负面，示例还负责建立这次调用中的符号映射。

一种帮助理解的视角是把任务 z 看作隐变量：

$$p(y | x, D_c) = \sum_z p(y | x, z) p(z | D_c). \quad (17.4)$$

示例帮助推断当前任务是什么。这个式子提供了概率直觉，但不能据此断言真实 Transformer 必然在内部执行严格的贝叶斯算法。

相似示例不一定是最好的示例

相似示例有助于提供局部模式，但若全部来自同一标签，模型可能形成偏置。更好的集合常需要兼顾相似度、标签覆盖和多样性。可以先检索候选，再选出覆盖不同边界情况的少量示例。

示例顺序也可能影响输出。模型对最近内容、位置和模板具有偏好，因此应在验证集上检查多个顺序，或采用稳定的排序规则。测试标签不能用于选择示例；否则会引入难以察觉的泄漏。

备注 | 示例可能传达了错误的规律

若所有正面例子都很短、所有负面例子都很长，模型可能利用长度而非情感。提示中的少样本学习也会出现数据捷径，不能因为没有梯度更新就忽略实验设计。

17.3 提示设计、校准与软提示

一个实用提示可以按任务、输入、约束和输出格式组织。分隔符用于区分指令与待处理材料。若材料中包含别人的指令，应明确它只是被分析的内容，而不是系统应执行的新任务。

例如，摘要任务可说明目标读者、长度和必须保留的事实范围。过多互相冲突的要求会增加失败概率。提示越长不代表越清晰，关键是每条约束是否能够被检查。

标签偏置与上下文校准

模型可能在没有有效输入时也偏爱某个标签词。设真实输入下的标签分数为 $p(c | x)$ ，空白或中性输入下为 $p(c | x_0)$ ，一种启发式校准使用

$$s(c, x) = \log p(c | x) - \log p(c | x_0). \quad (17.5)$$

它试图扣除模板和标签词的基础偏好。校准输入怎样选择、是否适合当前任务，都需要验证，不能把这个差值当成普遍正确的后验。

如果正面标签的基础概率为 0.8，负面为 0.2，某评论下变成 (0.6, 0.4)，原始预测仍偏向正面；但相对基础分布，负面的提升更大。这个例子解释了校准为什么有时会改变决策。

离散提示与可学习提示

硬提示使用人类可读的 token。软提示则在输入嵌入前加入可学习向量 $P \in \mathbb{R}^{m \times d}$ ，冻结主体模型，只更新 P：

$$\min_P - \sum_i \log p_\theta(y_i | [P; E(x_i)]). \quad (17.6)$$

这些向量不必对应真实词。Prefix tuning 还可以在多层中引入可学习的前缀键值。它们都改变可训练参数的一部分，因而与纯粹的 ICL 不同。

算法实现 | 示例检索与提示构造

```
candidates = retrieve_examples(query, pool, top_k=30)
examples = select_diverse(candidates, count=4)
prompt = task_instruction + "\n"
for ex in examples:
    prompt += format_example(ex.input, ex.output)
prompt += format_query(query)
answer = generate(prompt, stop=stop_markers)
result = parse_and_validate(answer)
```

示例池只包含允许使用的训练材料。解析器、停止符和失败处理也属于系统，应在测试前固定。

17.4 推理提示与实验方法

给出中间步骤示例，可能让模型在复杂任务上生成更多有用的中间计算。这个办法通常称为 chain-of-thought 提示。它改变了模型输出的文本过程，也增加了推断 token 数。

对需要计算的任务，简短公式、表格或可执行程序常比长篇解释更易检查。例如“买三件每件 18 元的商品，优惠 5 元”可先写成 $3 \times 18 - 5$ ，再计算为 49。任务需要的是可核验的步骤，不是越长越好的文字。

可见推理文本不一定忠实反映模型内部的全部机制，也可能包含事后合理化。因此，评价应检查最终答案与可验证的中间结果，而不是把解释长度当作推理能力指标。程序辅助、搜索与多样本选择会在后面单独讨论。

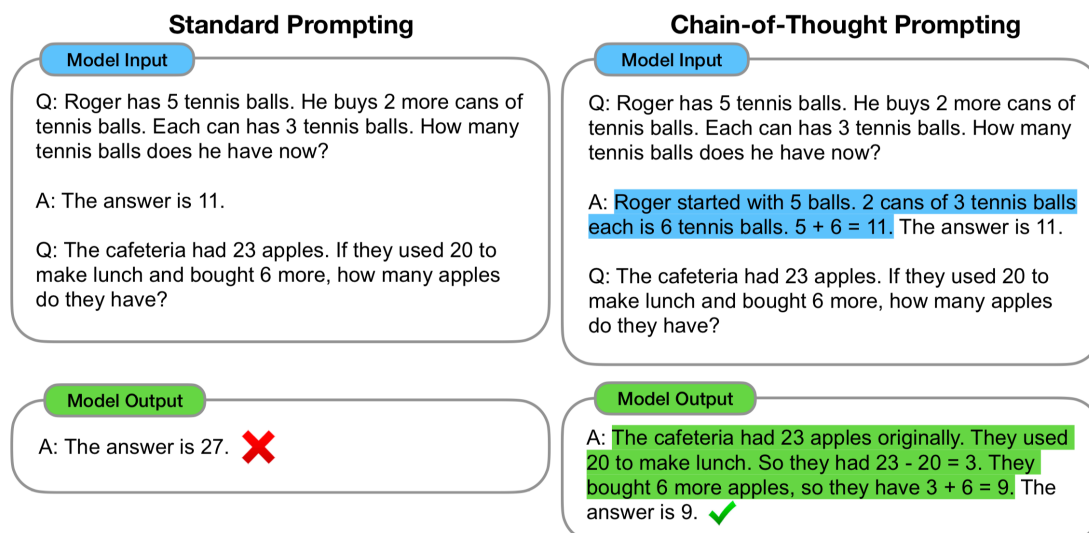


图 17.2 示例中的中间计算让模型有机会把复合问题拆开处理。可见过程仍需检查，不能只凭流畅程度判断正确。（课件 lec12_ppt，第 26 页）

怎样比较两个提示

应在固定模型、解码参数、示例池和评测集下比较。只挑几个看起来成功的案例，很容易高估效果。若不断试提示并观察同一测试集，它实际上已经成了验证集，需要另外保留测试集。

对随机生成，要么固定随机种子并认识其局限，要么多次采样后报告平均与波动。提示 A 偶然答对一次、提示 B 偶然答错一次，不足以支持稳定优劣判断。

常见失败怎样定位

如果模型输出格式正确但内容错误，优先检查任务知识与证据；内容正确但结构不合法，检查格式约束与解析；少样本示例加入后反而下降，检查标签分布、示例相关性与顺序；长提示中遗忘早期要求，检查信息位置和上下文负担。

一个任务识别的概率例子

假设当前任务可能使用两种标签映射： z_1 把正面映射到 A， z_2 把正面映射到 B。先验各为 $1/2$ 。一个清楚的正面示例被标为 A；假设这个示例在正确映射下出现概率为 0.9，错误映射下为 0.1。

看到示例后， $p(z_1 | D) = 0.9$ 。若又看到一条条件独立、同样支持 z_1 的示例，后验优势变为 $9^2 = 81$ ，后验为 $81/82 \approx 0.988$ 。

这个小模型说明示例怎样减少任务不确定性。真实 ICL 还涉及复杂表示与示例依赖，不能把这两个数当作 Transformer 的内部实测值。它们只是帮助理解“示例提供任务信息”的概率模型。

示例重复为什么不等于信息翻倍

如果第二条示例只是第一条的完全复制，它没有提供独立的新证据。把两者当作独立观测，会产生与朴素贝叶斯重复特征相似的过度自信。

因此，示例选择应避免只检索到大量近重复材料。用一条正面、一条负面和一条边界案例，有时比放三条几乎相同的正面评论更有帮助。原因不只在标签平衡，还在于覆盖了更多决策边界。

标签概率的校准手算

模板在空输入上给 A、B 概率为 (0.8, 0.2)，某真实输入上为 (0.6, 0.4)。按概率比较准，未归一化得分为

$$\left(\frac{0.6}{0.8}, \frac{0.4}{0.2}\right) = (0.75, 2). \quad (17.7)$$

归一化后为 (3/11, 8/11)，预测从 A 改为 B。它去掉了模板原本强烈偏爱 A 的影响。

但如果空输入并不能代表模板偏置，这个修正也可能引入错误。应使用验证集检验校准是否改善概率或分类指标，而不是仅因为公式看起来合理就默认有效。

提示优化也会对验证集过拟合

假设尝试 100 个提示，每个提示真实平均性能相近，但小验证集上的分数有随机波动。选择最高分的那个，往往也选中了正向噪声。随后在新测试集上，优势可能消失。

这个现象与超参数搜索相同。可以扩大验证集、限制尝试次数、使用多个切分，或在最后保留完全未参与选择的测试集。没有参数更新，并不意味着没有模型选择过拟合。

格式约束怎样与语义约束配合

要求输出 JSON 可以检查字段和类型，但不能检查内容是否正确。对抽取任务，先验证结构，再验证字段值是否来自输入，最后验证关系是否符合任务。

例如输出 `year: 2024` 类型正确，也确实出现在原文中，但若问题问毕业年份、原文中的 2024 是入职年份，仍然错误。结构化输出减少一种失败，不会自动解决语义对齐。

推演与自检

设计一个情感分类提示，要求标签只用 A、B。先用正确示例建立映射，再交换两类标签映射。若模型仍坚持原来的自然语言标签习惯，说明它没有充分利用当前示例。这个实验把“理解任务内容”和“遵循本次标签约定”区分开来。

第 18 章 生成算法与高效推断

语言模型给出下一 token 的概率，生成算法决定怎样使用这些概率。选最高概率、随机采样、保留多个候选，都会产生不同文本。模型不变，解码方法也可能明显改变准确性、多样性和长度。

本章先区分搜索与采样，再介绍 KV cache、量化和推测解码。需要始终记住：提高速度和改变输出分布是两类不同操作。

18.1 贪心搜索与束搜索

完整序列的概率为

$$p(y \mid x) = \prod_{t=1}^{|y|} p(y_t \mid x, y_{<t}). \quad (18.1)$$

贪心搜索每一步选 $\arg \max_v p(v \mid x, y_{<t})$ 。它很快，但局部最优未必产生全局最高概率序列。

例如第一步 A 的概率为 0.6、B 为 0.4。A 分支最好的完整后续条件概率只有 0.5，B 分支则为 0.9。贪心先选 A，最终概率最多 0.3；以 B 开头的最佳序列概率为 0.36，反而更高。

束搜索每步保留 b 个高分前缀。扩展候选的累计分数为

$$S(y_{1:t}) = S(y_{1:t-1}) + \log p(y_t \mid x, y_{<t}). \quad (18.2)$$

$b = 1$ 退化为贪心。有限束宽仍是近似搜索，早期被删除的分支不能恢复。束宽增加也不保证实际任务质量单调提高，因为模型概率与人工质量并非同一目标。

结束与长度偏好

每多乘一个不大于 1 的条件概率，序列概率不会增加，所以直接比较完整序列时容易偏向短答案。结束符概率本身也反映模型的长度偏好。常见启发式用

$$S_{\text{norm}}(y) = \frac{\sum_t \log p(y_t \mid x, y_{<t})}{|y|^\alpha} \quad (18.3)$$

或其他长度惩罚调整排序。这样做可能改善任务指标，但评分已不再是原始序列对数概率。

备注 | 搜索错误与模型错误

如果更强搜索找到更高模型概率却更差的人类答案，问题可能出在模型目标或校准。不能把所有低质量输出都归咎于搜索不充分。

算法实现 | 束搜索的最小骨架

```

beams = [(start_tokens, 0.0)]
finished = []
for step in range(max_new_tokens):
    candidates = []
    for prefix, score in beams:
        logp = model_next_logprobs(prefix)
        for token in top_tokens(logp, beam_size):
            item = (prefix + [token], score + logp[token])
            (finished if token == eos else candidates).append(item)
    beams = best_by_score(candidates, beam_size)
    if not beams:
        break
return best_completed_or_partial(finished, beams)

```

真实实现应明确完成候选的保存数、长度惩罚和提前停止准则。不同实现即使束宽相同，也可能给出不同答案。

18.2 温度、Top-k 与 Top-p 采样

采样从分布中随机选择 token。温度缩放为

$$p_T(v) = \frac{e^{z_v/T}}{\sum_u e^{z_u/T}}, \quad T > 0. \quad (18.4)$$

$T < 1$ 使分布更集中， $T > 1$ 更平坦。 $T \rightarrow 0^+$ 时集中到最大 logit；若存在并列最大值，极限并不唯一指向某个固定 token。

Top-k 只保留概率最高的 k 个 token，再归一化。Top-p 先排序，取累计概率首次达到阈值 p 的最小前缀集合，再归一化。二者都截去了原分布的一部分。

设概率为 (0.5, 0.3, 0.15, 0.05)。Top-k 取 $k = 2$ 后变成 (0.625, 0.375, 0, 0)。Top-p 取 $p = 0.9$ 时需要前三个 token，因为前两个只累计到 0.8，归一化后约为 (0.526, 0.316, 0.158, 0)。

数学直觉 | 固定人数与固定概率质量

Top-k 每次保留相同数量候选。Top-p 在模型很确定时保留较少候选，不确定时保留更多候选。它们提供不同的截断规则，不直接保证事实正确。

重复惩罚、禁词和格式约束还会进一步修改 logits。应用顺序会影响最终分布：先温度缩放再 Top-p，与相反顺序通常不同。评测中应固定并报告这些设置。

为什么多样性不能无限提高

开放式写作可能受益于多样采样，但高温会提高低概率候选的机会，其中也包含错误和不连贯内容。事实问答和代码任务应根据可验证的成功率调节采样，不能只凭文本“更有创意”判断。

18.3 KV cache 与两阶段推断

因果 Transformer 生成到第 t 步时，过去位置的表示不会因为新 token 到来而改变。过去的键和值因此可以缓存。新一步只计算当前 token 的 Q、K、V，再让当前查询读取所有缓存。

$$K_{1:t} = [K_{1:t-1}; k_t], \quad V_{1:t} = [V_{1:t-1}; v_t], \quad (18.5)$$

$$o_t = \text{softmax} \left(\frac{q_t K_{1:t}^\top}{\sqrt{d_k}} \right) V_{1:t}. \quad (18.6)$$

每一层都有自己的缓存。过去的查询通常不需要保存，因为后续步骤只要计算新位置的输出。

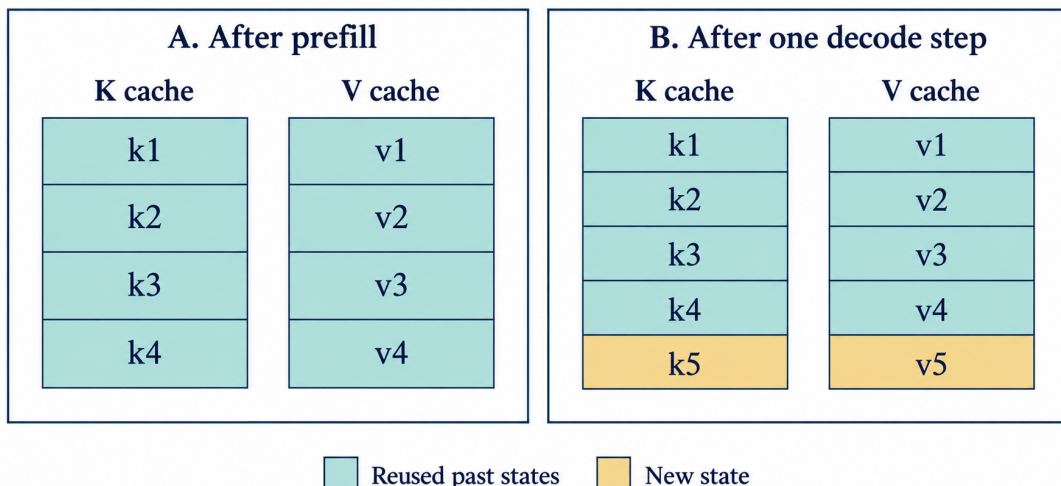


图 18.1 预填充保存提示中各位置的键和值；下一步追加一对新状态，旧状态继续复用。图中省略了层维度和批量维度。

prefill 一次处理整个提示，通常适合较大的矩阵运算；decode 每次处理少量新 token，常受内存带宽和缓存访问影响。首 token 延迟与后续 token 吞吐因此是不同指标。

设层数为 L 、批量为 B 、缓存长度为 T 、键值头数为 H_{kv} 、每头维度为 d_h ，每元素占 s 字节，则

$$\text{Memory}_{KV} \approx 2LBTH_{kv}d_h s. \quad (18.7)$$

系数 2 来自 K 与 V。这个估计不含分配碎片、页表和临时缓冲。缓存随长度与并发数线性增长，因此模型权重能放进显存，并不代表任意长上下文和任意并发都能运行。

一个显存估算

若 $L = 32, B = 1, T = 4096, H_{kv} = 32, d_h = 128, s = 2$ ，缓存占

$$2 \times 32 \times 4096 \times 32 \times 128 \times 2 = 2^{31} \text{ bytes} = 2 \text{ GiB}. \quad (18.8)$$

如果键值头数降为 8，其他量不变，缓存变为 0.5 GiB。分组查询注意力 GQA 让多个查询头共享一组键值头；多查询注意力 MQA 是键值头数为 1 的情形。它们改变了架构中的共享方式，不只是对缓存做无损压缩。

缓存复用的条件

同一个前缀可被多个请求复用，但必须保证 token、位置编号、模型参数和相关推断设置一致。RoPE 的位置偏移错误，会让缓存中的键与新查询使用不一致的位置含义。

KV cache 不会消除所有长序列代价。新查询仍要读取过去的键值，单步注意力成本随缓存长度增长。滑动窗口可以限制读取范围，但它也限制可直接访问的历史信息。

18.4 量化、批处理与推测解码

量化用较少位数近似保存权重或激活。对称线性量化的一种形式为

$$q = \text{clip}(\text{round}(x/s), -Q, Q), \quad \hat{x} = sq. \quad (18.9)$$

s 是尺度， Q 由位宽决定。分组尺度可让不同通道或块使用不同范围。低位宽减少存储和传输，但误差与实际速度取决于分组、异常值处理和硬件算子。

连续批处理在某个请求结束后及时加入新请求，提高设备利用率。批量过大又会提高单请求等待时间。吞吐、首 token 延迟、每 token 延迟与并发容量，需要一起衡量。

推测解码的基本思想

小模型先提出多个 token，大模型一次验证这些候选。如果草稿经常被接受，就能减少大模型按步调用的开销。对于采样任务，要用正确的接受拒绝规则，才能保留目标模型的分布 [22]。

设草稿分布为 q ，目标分布为 p ，候选 $x \sim q$ 。接受概率为

$$a(x) = \min\left(1, \frac{p(x)}{q(x)}\right). \quad (18.10)$$

若拒绝，从归一化残差分布

$$r(x) = \frac{\max(p(x) - q(x), 0)}{\sum_v \max(p(v) - q(v), 0)} \quad (18.11)$$

采样。接受产生的质量为 $\min(p(x), q(x))$ ，拒绝后的补偿质量为 $\max(p(x) - q(x), 0)$ ，两者相加恰好是 $p(x)$ 。

这个证明针对理想精确计算和相同条件前缀。实际多 token 验证还需在首次拒绝处停止接受草稿，并从正确位置继续。若省略校正而直接采纳“看起来差不多”的草稿，就改变了目标分布。

数学直觉 | 便宜模型先试，贵模型保留决定权

草稿模型负责提出可能的续写，目标模型负责确认。速度收益来自一次检查多个位置，而不是让小模型替代所有判断。

束搜索中的一个早期剪枝例子

设第一步三个 token 概率为 A:0.5、B:0.3、C:0.2。束宽为 2 时，C 被删除。假设 A 和 B 的最佳完整后续概率分别为 0.1 和 0.2，C 的最佳后续为 0.9，则三个最佳完整序列概率为

$$0.05, \quad 0.06, \quad 0.18. \quad (18.12)$$

全局最优来自 C，但有限束宽已在第一步丢弃它。因此束宽 2 虽比贪心保留更多候选，仍没有最优保证。

如果评分是未归一化的累计对数概率，继续扩展只能增加非正项。一个已完成序列的分数若高于所有未完成前缀的分数，就可作为安全停止条件之一。加入长度归一化后，分数可能因分母变化而上升，原停止逻辑就不一定适用。

温度如何改变一个二类分布

原始概率为 (0.8, 0.2)，两类 logit 差为 $\log 4$ 。温度 $T = 2$ 后，优势变成 $e^{\log 4/2} = 2$ ，新概率为 (2/3, 1/3)。温度 $T = 1/2$ 时，优势变成 16，新概率为 (16/17, 1/17)。

这说明温度作用于 logit 差，等价于对原概率做 $1/T$ 次幂再归一化：

$$p_T(v) = \frac{p(v)^{1/T}}{\sum_u p(u)^{1/T}}. \quad (18.13)$$

因此不能把“温度 2”理解成简单把概率乘 2。

KV cache 节省了哪些重复工作

没有缓存时，第 t 步可能重新计算整个前缀在每层的键和值。若每一步都这样做，长度为 M 的生成会重复处理大量旧 token。

有缓存时，旧位置的各层 K、V 被保留，新位置只增加一行。但新查询仍要与全部旧键匹配，并加权全部旧值。缓存主要消除重复的前缀投影和状态计算，不会让所有注意力计算都消失。

prefill 后缓存对应输入提示。生成第一个新 token 的分布来自提示最后位置的输出；把该 token 送入下一步后，才追加它自己的 K、V。实现中把“已经生成的 token”和“已经写入缓存的 token”错位，是常见的一步偏移错误。

量化误差的一个小例子

设对称量化尺度 $s = 0.1$ ，原值为 $x = 0.26$ ，量化整数 $q = 3$ ，恢复值为 0.3，误差为 0.04。若不发生截断，四舍五入误差绝对值最多约为 $s/2$ 。

若数值超出可表示范围，截断误差可以远大于 $s/2$ 。异常大值会迫使统一尺度变大，进而降低多数小值的分辨率。这解释了为什么分组量化和异常值处理重要。

推测采样的一般质量守恒

候选 x 从 q 产生，接受概率为 $\min(1, p(x)/q(x))$ ，所以接受到 x 的总质量为 $\min(p(x), q(x))$ 。总拒绝概率为

$$R = 1 - \sum_x \min(p(x), q(x)) = \sum_x \max(p(x) - q(x), 0). \quad (18.14)$$

拒绝后按残差分布采样，给 x 补回的质量为 $Rr(x) = \max(p(x) - q(x), 0)$ 。两部分相加等于 p ，证明了单步分布正确。

接受率越高，潜在加速越大，但草稿模型自身也有成本。草稿更强通常更贵，选择时应比较整体延迟，而不是只看接受率。

推演与自检

若草稿分布 $q = (0.8, 0.2)$ ，目标分布 $p = (0.6, 0.4)$ ，第一个 token 的接受概率为 0.75，第二个为 1。接受质量为 (0.6, 0.2)，剩余 0.2 全补给第二个 token，最终恢复 (0.6, 0.4)。这个二类例子可用于检查实现是否保留目标分布。

第 19 章 监督微调与参数高效适配

基础语言模型擅长继续文本，却未必按用户要求回答。监督微调用“指令—回答”示例明确告诉模型：遇到这样的输入，应当生成怎样的输出。它把通用预测能力引向具体的交互方式。

本章先解释训练样本和损失掩码，再介绍全量微调与 LoRA。关键问题是：哪些 token 提供监督，哪些参数允许更新，以及更新后怎样判断模型真正变好了。

19.1 指令数据与回答损失

一个 SFT 样本可写为 (x, y) ，其中 x 包含系统要求、用户问题与必要上下文， y 是目标回答。常见目标只在回答位置计算损失：

$$\mathcal{L}_{\text{SFT}} = - \sum_{t=1}^{|y|} \log \pi_{\theta}(y_t \mid x, y_{<t}). \quad (19.1)$$

输入 x 仍参与注意力，只是不直接作为要拟合的目标。把用户部分标签设为忽略值，不代表从模型输入中删除用户问题。

对多轮对话，可以选择监督每一轮助手回答，也可以只监督最后一轮。两个策略都合理，但会产生不同权重。角色分隔符、轮次结束符和整体结束符必须与模型预期格式一致。

一个损失掩码例子

假设 token 序列简写为

USER 问题 ASSISTANT 回答 EOS

回答损失应覆盖“回答”及约定的结束符。标签还需要与前一位置 logits 对齐。忽略的用户 token 可以出现在输入中，但对应目标标签设为 -100 一类的框架约定值。

算法实现 | 回答区域监督

```
labels = input_ids.clone()
labels[~assistant_target_mask] = -100
logits = model(input_ids, attention_mask=attention_mask).logits
loss = cross_entropy(logits[:, :-1].reshape(-1, V),
                     labels[:, 1:].reshape(-1),
                     ignore_index=-100)
```

部分模型接口会在内部完成移位。若使用这种接口，就不要再次手动移位。先用短样本核对每个 logit 对应的目标 token。

数学直觉 | SFT 学的是条件回答

预训练让模型熟悉大量文本的延续方式。SFT 进一步提高符合任务要求的回答在条件分布中的概率。它仍使用最大似然，只是样本构造和被监督的位置更有针对性。

19.2 数据质量、覆盖与训练强度

指令数据既可以由人编写，也可以来自已有 NLP 数据集、用户对话或模型生成。把分类样本转成指令时，不应只机械套用同一个模板；任务描述、输入形式和标签解释都要清楚。

质量至少包含事实正确、任务一致、表达清楚和格式合法。多样性则包括任务、领域、长度、语言和难度。只有短问短答的数据，未必能教好长文总结；只有标准答案，也未必能教会模型处理缺失信息与歧义。

课件中的 Self-Instruct 一类方法先生成指令和实例，再过滤、去重并训练。合成数据的教师能力限制了学生可得到的监督。盲目增加类似样本，可能放大教师的错误和风格偏置。

一条长回答应该有多大权重

按所有回答 token 平均损失，长回答自然贡献更多监督位置。先按每个样本平均、再按样本平均，则每条回答的总权重相同。两种目标分别是

$$\mathcal{L}_{\text{token}} = \frac{\sum_i \sum_t \ell_{it}}{\sum_i |y_i|}, \quad \mathcal{L}_{\text{sample}} = \frac{1}{N} \sum_i \frac{1}{|y_i|} \sum_t \ell_{it}. \quad (19.2)$$

它们不等价。选择哪一种，应结合任务和长度分布，并在实验中保持一致。

过拟合表现不只在验证损失上

模型可能更爱使用训练集的固定开头，回答变长，或在本应简短的任务里生成模板解释。这些变化未必提高实际成功率。应同时检查任务指标、长度分布、格式失败率与典型错误，而不只观察训练损失。

备注 | 新知识与旧能力

SFT 可以让模型学习新模式或新事实，但少量示例未必足以稳定注入大量知识。频繁变化的事实可能更适合通过检索提供；稳定的行为和格式要求则常适合微调。两者可以组合。

19.3 LoRA 的低秩更新

全量微调更新整个参数矩阵 $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ 。LoRA 冻结 W_0 ，把更新限制为低秩乘积 [14]：

$$W = W_0 + \Delta W, \quad \Delta W = \frac{\alpha}{r} BA, \quad (19.3)$$

其中 $A \in \mathbb{R}^{r \times d_{\text{in}}}$ 、 $B \in \mathbb{R}^{d_{\text{out}} \times r}$ ， r 远小于两侧维度。输出为

$$y = W_0 x + \frac{\alpha}{r} B(Ax). \quad (19.4)$$

先用 A 把输入投影到 r 维，再用 B 映回输出空间。可训练参数从 $d_{\text{out}} d_{\text{in}}$ 变为 $r(d_{\text{in}} + d_{\text{out}})$ 。

若输入输出均为 4096、 $r = 8$ ，原矩阵有 16,777,216 个参数，低秩分支只有 65,536 个，约为原来的 0.39%。这只是该矩阵的比例，不是整个训练显存的节省比例。

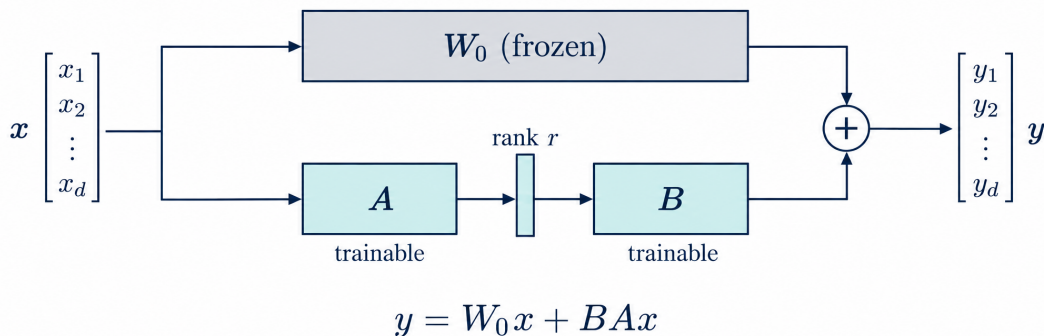


图 19.1 冻结原矩阵，另加一条低秩分支。图中示意输入输出同维，省略缩放系数 α/r 。

数学直觉 | 限制的是更新方向

LoRA 不要求预训练权重 W_0 本身低秩，而是认为适配当前任务所需的增量，可以用较少方向表示。它保留原模型，再学习一组受限制的修正。

为什么不能把两组矩阵都初始化为零

常见初始化让 A 随机、B 为零，因此初始 $\Delta W = 0$ ，模型最初与基础模型相同。设输出梯度为 $g = \partial \ell / \partial y$ ，忽略缩放后有

$$\frac{\partial \ell}{\partial B} = g(Ax)^\top, \quad \frac{\partial \ell}{\partial A} = B^\top g x^\top. \quad (19.5)$$

初始 B 为零时，A 的梯度暂为零，但 B 能得到非零梯度；下一步之后 A 也可学习。若 A 与 B 都为零，两者梯度都为零，低秩分支无法启动。

合并权重与多个适配器

推断时可把 ΔW 加入 W_0 ，得到一个合并矩阵。若不合并，则保留两条分支，方便切换任务适配器。量化权重的合并涉及额外的解量化、重数量化及误差，不应简单认为所有配置都能无代价合并。

选择插入哪些矩阵、秩 r 、缩放 α 和 dropout，会影响效果与成本。低秩不保证所有任务都足够；对于差异很大的任务，可能需要更高秩或更新更多模块。

算法实现 | LoRA 层的概念实现

```
class LoRALinear:
    def forward(self, x):
        base = x @ self.W0.T      # W0 is frozen
        low_rank = (x @ self.A.T) @ self.B.T
        return base + (self.alpha / self.rank) * low_rank
```

冻结表示不更新该参数，不表示整个基础层不参与反向计算。为更新前面的适配器，梯度仍可能需要穿过冻结层。

19.4 QLoRA、蒸馏与适配选择

QLoRA 将冻结的基础权重以低位宽形式保存，同时训练 LoRA 分支 [15]。其核心收益是减少基础权重存储；优化器状态主要服务于少量可训练参数。低位宽存储并不意味着所有运算都在相同低精度下完成。

还可以只训练任务头、部分层或软提示。冻结越多，训练成本通常越低，但可改变的行为也受到限制。选择应基于可用数据、任务差异、显存和部署方式，而不是只按参数更新比例排序。

知识蒸馏让学生拟合教师的输出分布。温度为 T 时，可使用

$$\mathcal{L}_{\text{KD}} = T^2 D_{\text{KL}} \left(p_T^{\text{teacher}} \| p_T^{\text{student}} \right). \quad (19.6)$$

前面的 T^2 用来补偿温度对梯度尺度的影响，是常见约定。实际也可混合真实标签交叉熵。教师的软分布提供类别之间的相对关系，但也可能传递教师偏差。

序列级蒸馏用教师生成的完整回答作为训练目标。它不需要保存词表级概率，但只展示了教师分布中的若干样本。若教师生成错误答案而没有验证，学生会直接学习这些错误。

怎样评价微调是否成功

至少保留三类检查：目标任务是否提高，通用能力是否明显下降，格式与行为要求是否稳定满足。还应使用与训练来源不同的样本，防止模型只是记住模板或答案。

对于专业问答，可以同时测闭卷回答与提供证据后的回答。若前者差、后者好，问题可能是知识覆盖；若有证据仍答错，问题更可能是读取、推理或遵循任务的能力。不同错误需要不同改进方法。

完整算例：一个秩一更新做了什么

令基础矩阵为单位阵，输入输出均为二维。取

$$W_0 = I, \quad A = \begin{pmatrix} 1 & -1 \end{pmatrix}, \quad B = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \quad (19.7)$$

并暂取缩放系数为 1。则

$$BA = \begin{pmatrix} 2 & -2 \\ 1 & -1 \end{pmatrix}. \quad (19.8)$$

对输入 $x = (3, 1)^\top$ ，低维投影为 $Ax = 2$ ，更新输出为 $BAx = (4, 2)^\top$ ，最终 $y = (7, 3)^\top$ 。

对输入 $x' = (2, 2)^\top$ ， $Ax' = 0$ ，低秩分支不改变输出。这个更新只读取输入在 $(1, -1)$ 方向上的差异，并沿 $(2, 1)$ 方向写入输出。秩一并不意味着只改一个矩阵元素，而是只使用一条输入输出组合方向。

同一个例子的梯度

设目标为 $y^* = (5, 2)^\top$ ，平方损失为 $\ell = \frac{1}{2} \|y - y^*\|^2$ 。当前输出梯度 $g = (2, 1)^\top$ 。

$$\frac{\partial \ell}{\partial B} = g(Ax)^\top = \begin{pmatrix} 4 \\ 2 \end{pmatrix}, \quad (19.9)$$

$$\frac{\partial \ell}{\partial A} = B^\top g x^\top = 5 \begin{pmatrix} 3 & 1 \end{pmatrix} = \begin{pmatrix} 15 & 5 \end{pmatrix}. \quad (19.10)$$

基础矩阵被冻结，所以不更新 W_0 ；但梯度对输入仍包含 $W_0^\top g$ 。这说明冻结参数与阻断整个计算图是不同操作。

为什么 rank 相同也可能效果不同

低秩分支插在 Q、K、V、输出投影或 FFN 上，改变的是不同计算环节。只调整 value，主要改变被读取的内容；调整 query 或 key，还可以改变匹配权重；调整 FFN，则改变逐位置的特征变换。

实际网络中这些作用会互相影响，不能完全分离。但它解释了为什么“都使用 rank 8”不足以说明两个 LoRA 实验配置相同，还要记录目标模块与缩放方式。

SFT 中长答案的权重示例

两条回答长度分别为 2 和 8，平均 token 损失分别为 1 和 3。token 平均目标为 2.6，样本平均目标为 2。前者让长回答拥有四倍总权重，后者让两条回答总权重相同。

如果偏好数据中正例总比负例长，后续训练又使用不同长度归一化，长度因素就会与内容偏好混在一起。可以按长度分组观察改善，或构造长度接近的偏好对，检查模型是否主要学会“写得更长”。

蒸馏中的软标签信息

教师对三类输出 (0.7, 0.2, 0.1)，硬标签只保留第一类正确，软标签还表达第二类比第三类更接近。学生拟合软分布时，会收到这部分相对关系。

但这些关系也可能来自教师偏差。若教师把某个群体或领域系统性误判，学生并不会因为模型更小就自动消除偏差。蒸馏评价应保留真实标签或外部验证，而不能只测学生与教师的一致率。

推演与自检

一个 1024×1024 矩阵使用秩 16 的 LoRA，可训练参数为 $16(1024 + 1024) = 32768$ ，约占原矩阵的 3.125%。如果把秩翻倍，参数翻倍，但是否改善性能仍需验证；低秩维度不是直接的“知识容量单位”。

第 20 章 偏好学习、RLHF 与 DPO

监督微调需要一份目标回答，但很多问题没有唯一标准答案。两段回答可能都正确，一段更清楚，另一段更冗长；也可能一段诚实承认信息不足，另一段给出看似完整的猜测。让标注者比较两段回答，往往比从头写出最优回答容易。

偏好学习利用这种比较信号。本章先把“更喜欢”写成概率模型，再解释怎样用奖励改进生成策略，最后推导 DPO。需要特别区分：奖励模型是人类偏好的近似，不是真实质量本身。

20.1 从比较数据到奖励模型

偏好数据写为 (x, y^+, y^-) ，其中 y^+ 是对同一输入 x 更受偏好的回答。奖励模型 $r_\phi(x, y) \in \mathbb{R}$ 为回答给出一个实数分数。

Bradley-Terry 模型用分数差预测偏好：

$$p_\phi(y^+ \succ y^- | x) = \text{sigmoid}(r_\phi(x, y^+) - r_\phi(x, y^-)). \quad (20.1)$$

若两个分数相同，预测偏好概率为 1/2；差值越大，越确信正例更好。对应损失为

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x, y^+, y^-)} \log \text{sigmoid}(r_\phi(x, y^+) - r_\phi(x, y^-)). \quad (20.2)$$

奖励网络可以用语言模型的最后状态接一个标量头。训练时只需要比较数据，不要求标注者给出可跨题目比较的绝对分数。

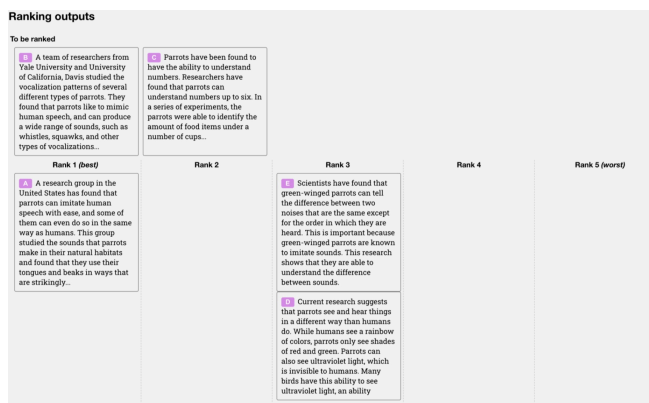


图 20.1 比较标注把多个回答排出相对顺序。训练目标通常使用同一提示下的回答对。（课件 lec13_posttraining, 第 21 页）

奖励的不可识别性

对同一个提示，给所有回答的奖励都加 $c(x)$ ，差值不变，偏好概率不变。因此比较数据不能确定每个提示的绝对奖励零点。奖励尺度也与偏好概率模型和训练约定相关，不能随意把不同奖励模型的分数直接比较。

例如奖励为 2 和 1 时，偏好概率约 0.731；奖励改成 102 和 101，概率仍相同。若改成 4 和 2，概率变为约 0.881，说明缩放与平移有不同作用。

数学直觉 | 学的是相对好坏

标注者告诉我们“这段比那段好”，奖励模型据此学习排序。分数 7 并不天然表示某种客观质量等级，它只有在训练协议与比较关系中才有意义。

20.2 策略梯度与 KL 约束

把生成模型记为策略 $\pi_\theta(y | x)$ 。如果只最大化平均奖励，目标是

$$J(\theta) = \mathbb{E}_x \mathbb{E}_{y \sim \pi_\theta(\cdot | x)} [r(x, y)]. \quad (20.3)$$

奖励可以由完整回答计算，也可以在中间步骤提供。由于 token 是离散采样，不能像普通连续层那样直接对采样结果求导。

利用 $\nabla \pi = \pi \nabla \log \pi$ ，固定提示时有

$$\nabla_\theta J = \sum_y r(x, y) \nabla_\theta \pi_\theta(y | x) \quad (20.4)$$

$$= \mathbb{E}_{y \sim \pi_\theta} [r(x, y) \nabla_\theta \log \pi_\theta(y | x)]. \quad (20.5)$$

再用自回归分解：

$$\nabla_\theta \log \pi_\theta(y | x) = \sum_t \nabla_\theta \log \pi_\theta(y_t | x, y_{<t}). \quad (20.6)$$

高奖励样本推动自身概率上升。问题是奖励噪声大，整段回答的信用分配也不容易。

基线为什么能降方差

用不依赖当前动作的基线 $b(x)$ 替代奖励零点：

$$\nabla J = \mathbb{E}[(r(x, y) - b(x)) \nabla \log \pi_\theta(y | x)]. \quad (20.7)$$

因为

$$\mathbb{E}[b(x) \nabla \log \pi] = b(x) \nabla \sum_y \pi(y | x) = 0, \quad (20.8)$$

减去基线不改变期望梯度。 $r - b$ 常称为优势的简化形式。逐 token 强化学习中，基线通常依赖当前状态，即提示与已生成前缀。

若只追逐奖励，策略可能找到奖励模型的漏洞。例如某些重复措辞恰好被高估，模型就不断使用它。一个常见约束是让策略不要远离参考模型 π_{ref} ：

$$J(\pi) = \mathbb{E}_{y \sim \pi} [r(x, y)] - \beta D_{\text{KL}}(\pi(\cdot | x) \| \pi_{\text{ref}}(\cdot | x)), \quad \beta > 0. \quad (20.9)$$

KL 不是证明输出正确的约束，它限制分布变化幅度。 β 越大，在这个理想目标中越强调保持参考分布。

序列 KL 与 token 级估计

序列对数比可分解为

$$\log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)} = \sum_t \log \frac{\pi(y_t | x, y_{<t})}{\pi_{\text{ref}}(y_t | x, y_{<t})}. \quad (20.10)$$

在 $y \sim \pi$ 下取期望，得到序列 KL。单个采样回答的对数比可以为负，尽管期望 KL 非负。不能把“每个样本的估计值必须非负”当作检查标准。

20.3 PPO 与完整的 RLHF 流程

典型 RLHF 流程先做 SFT，再训练奖励模型，最后使用强化学习优化策略。参考模型通常固定，奖励模型在相应阶段也常固定。经典流程的代表性实例见 [16]。

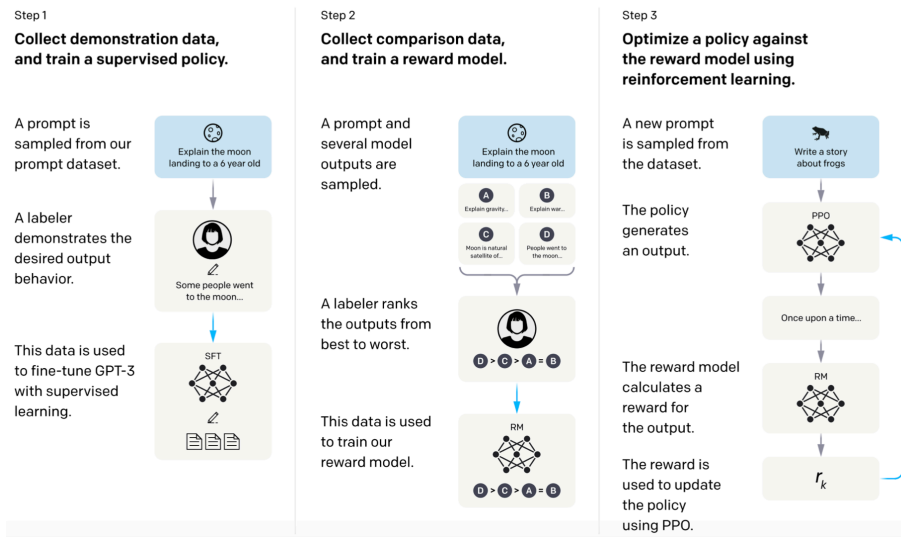


图 20.2 示范数据训练初始策略，比较数据训练奖励模型，随后用奖励优化生成策略。（课件 lec13_posttraining, 第 17 页）

PPO 从旧策略 π_{old} 采样轨迹，再限制新策略对这些动作概率的更新幅度。对状态 s_t 与动作 a_t ，定义比率

$$\rho_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}. \quad (20.11)$$

常见裁剪代理目标为

$$L^{\text{clip}} = \mathbb{E}_t \min \left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right). \quad (20.12)$$

训练最大化该式。若优势为正，增大动作概率是有益的，但超过阈值后目标不再继续奖励这一方向；若优势为负，降低动作概率也受到相应限制。

数学直觉 | 避免一次更新改得太猛

旧策略提供了训练样本。如果新策略很快变得完全不同，这些样本就难以代表当前行为。PPO 的裁剪是一种控制更新的办法，但不是严格保证所有状态的 KL 都小于某个界。

三个容易混淆的模型

参考模型用于 KL 约束，常固定为 SFT 模型。旧策略用于重要性比率，是生成当前训练批次时的策略快照。当前策略是正在更新的模型。参考模型与旧策略可能起初相同，但角色不同，不能在代码中混用。

价值模型估计未来回报，为优势估计提供基线。奖励模型评价回答质量，价值模型预测从当前前缀继续能得到多少回报。二者都输出标量，但监督目标不同。

奖励投机与分布外优化

奖励模型只在有限回答上训练。策略优化会主动寻找高分区域，可能进入奖励模型不熟悉分布。训练奖励上升而人工评价下降，就是代理目标失真的一种表现。应定期检查独立偏好集和实际任务成功率，而不是只看奖励曲线。

20.4 DPO：把奖励关系写进策略概率

DPO 从式 (20.9) 出发。固定提示 x ，先假设奖励已知、策略可以在参考分布的支持集上自由选择。展开目标并加入概率和为 1 的约束：

$$\mathcal{J} = \sum_y \pi(y)r(y) - \beta \sum_y \pi(y) \log \frac{\pi(y)}{\pi_{\text{ref}}(y)} + \lambda \left(\sum_y \pi(y) - 1 \right). \quad (20.13)$$

对 $\pi(y)$ 求导：

$$r(y) - \beta \left(\log \frac{\pi(y)}{\pi_{\text{ref}}(y)} + 1 \right) + \lambda = 0. \quad (20.14)$$

整理并归一化，得到

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y | x) \exp(r(x, y)/\beta). \quad (20.15)$$

奖励越高，参考概率被放大的倍数越大； β 越大，这种放大越温和。

反过来表示奖励：

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x). \quad (20.16)$$

对同一提示的两个回答取差，未知的 $\log Z(x)$ 消失。将策略 π_θ 代入这个奖励参数化，再放入 Bradley-Terry 模型，就得到 DPO 损失 [17]：

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E} \log \text{sigmoid}(\beta \Delta_\theta), \quad (20.17)$$

$$\Delta_\theta = \log \frac{\pi_\theta(y^+ | x)}{\pi_{\text{ref}}(y^+ | x)} - \log \frac{\pi_\theta(y^- | x)}{\pi_{\text{ref}}(y^- | x)}. \quad (20.18)$$

DPO 直接使用偏好对训练策略，无需在每一步先拟合显式奖励再进行在线采样。

梯度在推动什么

记 $m = \beta \Delta_\theta$ ，则

$$\nabla_\theta \mathcal{L} = -\beta \text{sigmoid}(-m) [\nabla \log \pi_\theta(y^+ | x) - \nabla \log \pi_\theta(y^- | x)]. \quad (20.19)$$

这个梯度会提高正例相对负例的对数概率。间隔 m 已经很大时，样本权重趋近于零，更新也更小。

相对差增加，不保证正例的绝对概率每次都上升。共享参数还会影响其他回答，词表归一化也会随之改变。

一个数值例子

参考模型对两个回答的概率为 0.2 和 0.1，当前策略为 0.3 和 0.1。相对改善差为 $\Delta = \log 1.5 - \log 1 = \log 1.5$ 。若 $\beta = 1$ ，偏好概率为 $\text{sigmoid}(\log 1.5) = 0.6$ ，损失约为 0.511。

初始策略等于参考模型时， $\Delta = 0$ ，每对样本损失为 $\log 2$ 。这为实现提供了一个简单的检查点。但初始损失正确，仍不能保证标签掩码、截断和长度处理都正确。

算法实现 | DPO 的核心损失

```
# Each log probability is a SUM over response tokens only.
chosen_ratio = policy_chosen_logp - ref_chosen_logp
rejected_ratio = policy_rejected_logp - ref_rejected_logp
margin = beta * (chosen_ratio - rejected_ratio)
loss = -torch.nn.functional.logsigmoid(margin).mean()
```

参考模型不更新。提示 token 不计入回答概率和，padding 也应屏蔽。若把求和换为按长度平均，目标就发生了变化。

20.5 方法选择与偏好数据的边界

SFT 使用示范回答，DPO 使用离线偏好对，在线 RL 则可对当前策略新生成的回答进行反馈。在线采样能探索新的行为，成本和稳定性要求也更高。离线方法更简单，却受已有回答覆盖范围限制。

偏好标注可能受长度、格式、语气和位置影响。若标注者偏爱看起来详细的回答，模型可能学会变长；若所有胜者都来自更强模型，数据也可能混入来源风格。应对调候选顺序、使用明确评价维度，并检查标注一致性。

“对齐到人类偏好”不是单一客观目标。不同人、任务和场景对清楚、简短、谨慎、完整的权衡不同。课程中更有用的问题是：偏好由谁提供，比较什么，数据覆盖哪些情况，优化后在哪些独立测试上改善？

完整算例：KL 约束怎样重新分配回答概率

假设只有三个完整回答，参考概率为 $(0.5, 0.3, 0.2)$ ，奖励为 $(0, \log 2, \log 3)$ ，并取 $\beta = 1$ 。最优策略的未归一化质量为

$$\pi_{\text{ref}}(y)e^{r(y)} = (0.5, 0.6, 0.6). \quad (20.20)$$

归一化常数为 $Z = 1.7$ ，所以

$$\pi^* = \left(\frac{5}{17}, \frac{6}{17}, \frac{6}{17} \right). \quad (20.21)$$

第三个回答奖励最高，却与第二个得到相同最终概率，因为它的参考概率更低。KL 正则使优化同时考虑奖励和偏离原分布的代价。

若令 $\beta \rightarrow \infty$ ，指数权重趋于 1，策略接近参考分布；若 $\beta \rightarrow 0^+$ ，策略更集中到最高奖励回答，但仍受参考支持集限制。参考概率严格为零的回答，不能在有限 KL 下获得正概率。

最优值与配分函数的关系

利用最优策略的定义，有

$$\log \frac{\pi^*(y)}{\pi_{\text{ref}}(y)} = \frac{r(y)}{\beta} - \log Z. \quad (20.22)$$

代入目标：

$$J(\pi^*) = \mathbb{E}_{\pi^*}[r] - \beta \mathbb{E}_{\pi^*} \left[\frac{r}{\beta} - \log Z \right] \quad (20.23)$$

$$= \beta \log Z. \quad (20.24)$$

更进一步，对任意策略 π 都可以整理为

$$J(\pi) = \beta \log Z - \beta D_{\text{KL}}(\pi \| \pi^*). \quad (20.25)$$

由于 KL 非负，最优点确实为 π^* 。这比只令导数为零更完整地说明了全局最优性。

DPO 的 beta 怎样解释

在奖励已知的 KL 目标中，较大 β 意味着更强地保持参考分布。DPO 损失中同一个符号又直接乘在分类间隔上，所以它也影响梯度尺度和饱和速度。

不能只看到 β 乘在梯度前面，就得出“越大一定离参考越远”。实际训练涉及有限数据、参数化、学习率和早停，效果要通过验证集与分布变化一起观察。

偏好差异接近零时的梯度

策略与参考相同时， $m = 0$ ， $\text{sigmoid}(-m) = 1/2$ 。对每个偏好对，梯度系数为 $-\beta/2$ 。若当前模型已经大幅提高正例相对概率， m 很大，系数接近零。

如果偏好标签本身有噪声，极端拉大间隔可能过拟合。对同一问题收集多个比较、允许平局或使用更明确的标注准则，可以比单纯训练更多轮更有价值。

同提示比较为什么重要

DPO 中消去的 $\log Z(x)$ 依赖提示 x 。只有正负回答来自同一个提示，它们的归一化常数才相同。把两个不同问题的回答直接凑成偏好对，会破坏这条推导。

数据中还要确保被偏好的内容确实针对同一任务条件。例如一个回答基于旧政策、另一个基于新政策，比较前应明确哪种政策版本是当前问题的条件。

推演与自检

若给每个提示的所有奖励增加同一个常数，式 (20.15) 的分子和归一化分母同时乘同一因子，策略不变。若把奖励乘十倍但保持 β 不变，策略会更集中。这再次说明奖励的零点与尺度有不同作用。

第 21 章 推理时计算、可验证奖励与工具使用

面对一道复杂题，模型可以直接给出答案，也可以先写出中间计算、尝试多个候选、运行程序或查询资料。后面这些做法增加了推理时计算，常能提高某些任务的成功率，但它们依赖额外的选择和验证机制。

本章把“多想一会儿”拆成可分析的算法。我们分别看多样本投票、程序辅助、搜索、可验证奖励和工具交互，避免把所有改进都归为一个模糊的“推理能力”。

21.1 多条候选与答案聚合

设模型生成中间过程 z 和答案 a ，则

$$p(a \mid x) = \sum_z p(z, a \mid x). \quad (21.1)$$

一个答案可能由多种过程支持。只取最高概率过程的答案，与对所有过程边缘化后的最佳答案不一定相同。

Self-consistency 采样多条过程，提取最终答案，再按答案频率投票 [19]。若样本为 $(z^{(i)}, a^{(i)})$ ，则

$$\hat{a} = \arg \max_a \sum_{i=1}^M \mathbf{1}[a^{(i)} = a]. \quad (21.2)$$

它用采样近似答案层面的聚合。数值答案应先统一格式，例如 1/2、0.5 与 50% 在合适问题中可能表示同一结果。

多数票为什么有时有效

假设每条独立候选以概率 $p > 1/2$ 答对，且错误可以按二元对错处理，奇数 M 次采样的多数正确概率为

$$\sum_{j=(M+1)/2}^M \binom{M}{j} p^j (1-p)^{M-j}. \quad (21.3)$$

当 $p = 0.7, M = 3$ ，结果为 $3(0.7)^2(0.3) + (0.7)^3 = 0.784$ ，高于单次的 0.7。

真实模型的错误往往相关，甚至所有样本都会重复同一个误解。此时独立假设不成立，收益远小于上式。若错误答案更容易形成一个大簇，投票也可能稳定地选错。

数学直觉 | 多采样需要多样性，也需要识别能力

生成十份几乎相同的答案，提供的新信息很少。即使候选不同，如果没有可靠办法选出正确者，增加候选也未必提高最终成功率。

Best-of- N 用评分器或验证器从 N 个候选中选最好者。它与多数票不同：一个少见但可证明正确的答案，也可以胜过多数错误答案。收益受到候选覆盖和评分器准确性的共同限制。

21.2 程序辅助与树搜索

自然语言擅长表达任务，精确计算则可以交给程序。模型先把问题转换成可执行步骤，再由解释器返回结果。例如折扣计算可以生成一个算术表达式，统计问题可以生成数据处理代码。

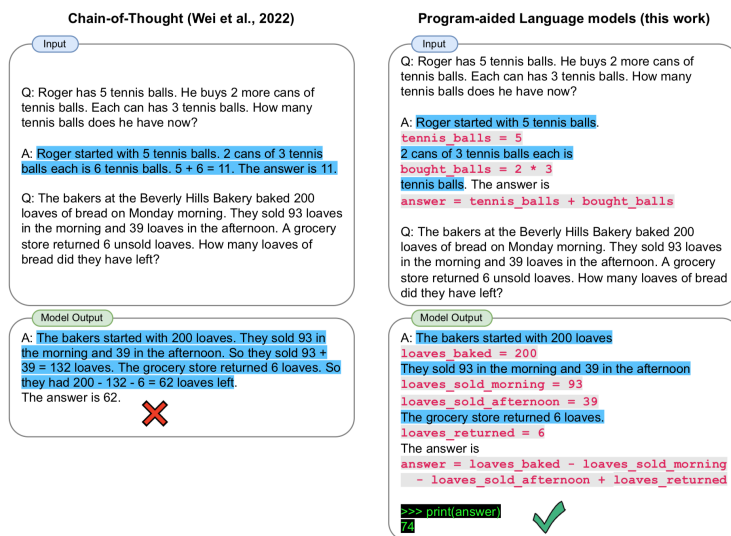


图 21.1 程序辅助把部分计算交给执行环境，模型负责把问题映射为正确程序。（课件 lec12_ppt，第 31 页）

执行成功不等于程序语义正确。程序可能使用了错误公式、漏掉条件或访问了错误字段。因此应区分语法有效、运行成功、测试通过和真实任务完成四个层次。

一个可验证的小任务

问题为“每箱 12 瓶，买 7 箱后分给 5 人，每人 8 瓶，还剩多少？”程序可以是

```
remaining = 12 * 7 - 5 * 8
assert remaining >= 0
print(remaining) # 44
```

验证器可以独立检查两个数量关系，而不是只看最终输出是否为一个整数。如果题目还要求均分剩余部分，则原程序还未完成全部任务。

树搜索把中间状态作为节点，候选步骤作为边。搜索可以同时保留多个思路，使用启发式或价值函数选择展开方向，遇到低质量分支时回退。

这种搜索与 Viterbi 有相似处：都避免只保留一个早期选择。区别是自然语言中间状态往往没有精确、紧凑的可合并状态，评分器也未必可靠。因此一般不能获得 HMM 那样的精确动态规划保证。

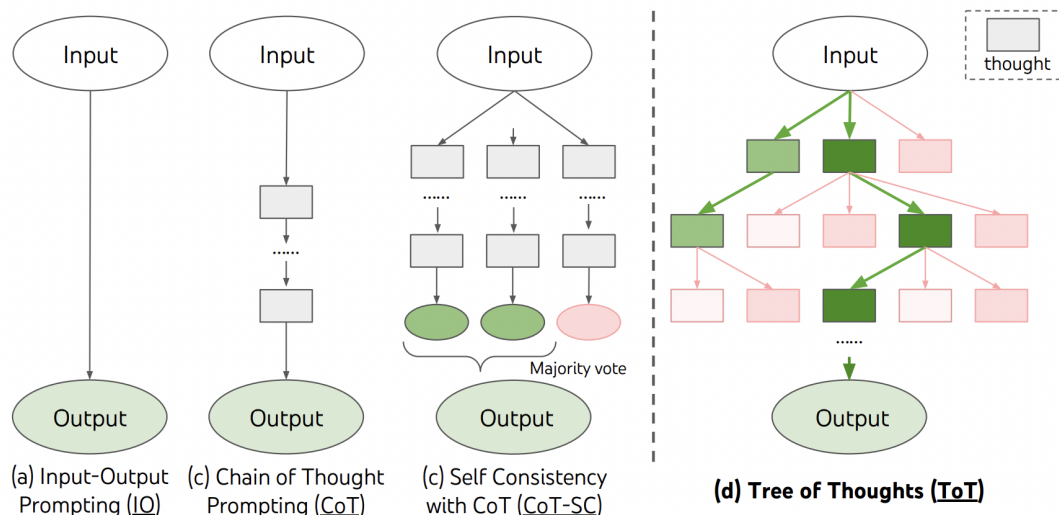


图 21.2 直接生成、单链过程、多样本聚合与树搜索使用不同的候选保留方式。（课件 lec12_ppt, 第 33 页）

算法实现 | 候选扩展与验证

```
frontier = [initial_state(problem)]
for depth in range(max_depth):
    proposals = [nxt for s in frontier for nxt in propose(s)]
    valid = [s for s in proposals if passes_local_checks(s)]
    solved = [s for s in valid if complete_and_verified(s)]
    if solved:
        return choose(solved)
    frontier = top_states(valid, budget=beam_width)
return failure_or_best_partial(frontier)
```

局部检查和最终检查不应混用。某步格式正确，只说明它值得继续考虑，不能提前当作整题成功。

21.3 可验证奖励与组相对优化

对于数学、代码和某些结构化任务，结果可以由规则或程序核验。强化学习可使用这种反馈，常称为可验证奖励。最简单的奖励是答案正确为 1，否则为 0；也可以组合格式、测试覆盖和资源限制等指标。

过程监督评价中间步骤，结果监督只评价最终答案。过程奖励能提供更密集的信号，但步骤标签更昂贵，也可能误奖看似合理的错误过程。结果奖励更容易自动化，却难以定位是哪一步导致成功或失败。

GRPO 使用同一提示下的多条回答构造相对优势，减少对独立价值模型的依赖 [18]。对一组 G 条回答，奖励为 $r_1, \dots, r_G$ ，一种常见形式为

$$\hat{A}_i = \frac{r_i - \bar{r}}{s_r + \epsilon}, \quad \bar{r} = \frac{1}{G} \sum_j r_j. \quad (21.4)$$

这里 s_r 为组内奖励标准差。奖励高于组均值的回答得到正优势，低于均值的得到负优势。

一个组内标准化例子

若四条回答奖励为 $(1, 1, 0, 0)$ ，均值为 0.5。采用总体标准差约定， $s_r = 0.5$ ，忽略极小 ϵ 后优势为 $(1, 1, -1, -1)$ 。若全部回答奖励都为 0，所有中心化奖励也为 0，没有相对区分信号。

逐 token 比率为

$$\rho_{it}(\theta) = \frac{\pi_{\theta}(y_{it} \mid x, y_{i,<t})}{\pi_{\text{old}}(y_{it} \mid x, y_{i,<t})}. \quad (21.5)$$

可用 PPO 风格的裁剪目标，将回答级优势用于各个 token，再按约定聚合。具体算法对长度归一化、KL 项和组标准化有不同选择；改变这些细节会改变优化行为。

备注 | 组均值不是免费的无偏基线

组统计量由采样回答本身计算，包含当前样本的信息。不能直接套用“任何与动作无关基线都不改变期望梯度”的结论。GRPO 的实用目标及其各种变体，需要按实际公式分析。

验证器也有漏洞

代码只通过少量公开测试，可能是硬编码；数学只检查最终数字，可能奖励碰巧猜对；格式奖励过强，可能让模型优先输出漂亮格式而放弃解题。奖励越容易自动计算，越要检查它是否真的覆盖目标能力。

长度也会影响奖励与梯度。给每条回答同等总权重，与给每个 token 同等权重，可能对长过程产生不同偏好。比较算法时，应固定这些口径，不能只比较名字。

21.4 工具与智能体的状态循环

工具使用让模型的动作不再只有输出文字，还可以查询搜索引擎、执行代码或读取数据库。一个简化循环是：观察当前状态，选择动作，接收工具结果，更新状态，再决定下一步。

可写为

$$a_t \sim \pi_{\theta}(\cdot \mid h_t), \quad o_{t+1} \sim P(\cdot \mid h_t, a_t), \quad h_{t+1} = (h_t, a_t, o_{t+1}). \quad (21.6)$$

历史 h_t 包含任务和已获得的证据。实际环境常只能部分观察，因此当前上下文不一定反映世界的全部状态。ReAct 一类工作把语言推理与行动交替组织 [20]。

工具返回内容是新的观测，不自动成为更高优先级的任务指令。系统还需要判断来源是否可靠、结果是否过时、调用是否成功。模型把失败信息误当成功，就可能在后面建立一整段错误推断。

何时调用工具

若问题只需已有常识，工具调用可能增加延迟；若需要最新事实、精确计算或环境状态，工具可能不可替代。可以把选择看成效用权衡：

$$U(a) = \mathbb{E}[\text{任务收益} \mid a] - \lambda_c \text{cost}(a) - \lambda_l \text{latency}(a). \quad (21.7)$$

这个式子不要求实际系统都显式计算效用，但提醒我们：工具的价值应由新增信息和任务成功衡量，而不是调用次数。

停止与任务完成

智能体应有明确的完成条件。代码任务可以要求测试通过，检索任务可以要求找到可支持答案的证据，操作任务可以要求环境状态达到目标。模型自己说“完成了”，只是一个输出，不足以证明任务已经完成。

推理预算也应有限。重复搜索、重复尝试同一失败动作，不一定带来新信息。可以设置最大步骤数、成本上限和重复检测，并在证据不足时返回可说明的部分结果。

候选覆盖与最终选择的两个概率

设单个候选正确概率为 p ，独立生成 N 个候选，至少一个正确的概率为

$$P(\text{存在正确候选}) = 1 - (1 - p)^N. \quad (21.8)$$

若 $p = 0.2$, $N = 5$ ，该概率约为 0.672。但系统还需要从候选中选出正确者。

设在存在正确候选的条件下，选择器以概率 s 选中正确答案。若没有正确候选时不可能成功，最终成功率为 $[1 - (1 - p)^N]s$ 。取 $s = 0.6$ ，结果约为 0.403，远低于候选覆盖率。

因此 $\text{pass}@N$ 与最终单次提交准确率之间的差距，可以揭示选择器的价值。模型有时“会做但没选对”，有时则根本没有生成正确候选，两种情况需要不同改进。

组内标准化对奖励尺度的影响

奖励组为 $(2, 2, 0, 0)$ ，与 $(1, 1, 0, 0)$ 的中心化标准化结果相同。正的线性缩放被标准差抵消，加常数被均值抵消。因此组相对优势主要关注相对差异，而不保留原奖励的绝对尺度。

这也会带来问题。一个组的奖励差异可能来自微小噪声，标准化后却被放大到与清晰正确错误差异类似的量级。 ϵ 、零方差组处理和奖励定义都可能影响训练。

结果奖励无法区分所有过程

两条过程都得到正确答案 42，一条使用正确推导，另一条先算错再碰巧抵消。只有最终数值奖励时，它们都得到 1。

如果任务目标还包括可解释、可迁移的步骤，就需要更丰富的验证，例如检查关键中间等式、随机替换题目数字，或在一组相关问题上重复测试。过程文本更长并不能解决这个识别问题。

工具调用中的状态误差

假设模型请求读取一个文件，工具返回“文件不存在”。如果模型仍把预期内容写入后续状态，之后的推理就建立在虚构观测上。正确的状态更新应包含失败事实，并据此选择检查路径、换来源或报告缺失。

同样，执行代码返回异常，不等于代码已经完成任务。智能体应把工具状态码、输出内容与任务完成条件分别处理。外部反馈只有被正确解释，才真正增加可靠性。

推演与自检

一个系统采样 20 个程序，至少一个通过测试，却总是选错最终提交。候选生成能力已经较好，瓶颈在选择器。若 20 个程序全部错误，改进排序没有帮助，应优先改善生成、任务理解或工具反馈。把流水线拆开评价，才能找到真正瓶颈。

第 22 章 检索增强生成与长上下文

模型参数中的知识更新较慢，也很难逐条追踪来源。检索增强生成把相关材料放到当前上下文中，再让模型据此回答。这样，知识库可以独立更新，答案也有机会连接到具体证据。

但检索到文档不等于回答正确。文档可能不相关，证据可能只支持答案的一部分，模型也可能忽略材料。本章按检索、组织和生成三个阶段分析 RAG，并比较它与长上下文和微调的关系。

22.1 从隐文档到条件生成

设问题为 x 、检索文档为 z 、答案为 y 。一种概率形式是

$$p(y | x) = \sum_z p_\eta(z | x) p_\theta(y | x, z). \quad (22.1)$$

检索器 p_η 提供文档分布，生成器 p_θ 在文档条件下回答。实际通常只对 Top-k 文档近似求和。原始 RAG 将检索与生成连接成可训练系统 [21]。

常见工程实现则直接取若干文档，拼接到提示中：

$$\hat{z}_{1:k} = \text{Retrieve}(x), \quad \hat{y} \sim p_\theta(\cdot | x, \hat{z}_{1:k}). \quad (22.2)$$

它未必显式对文档进行概率边缘化，也未必端到端训练。两种写法相关，但不是完全相同的实现。

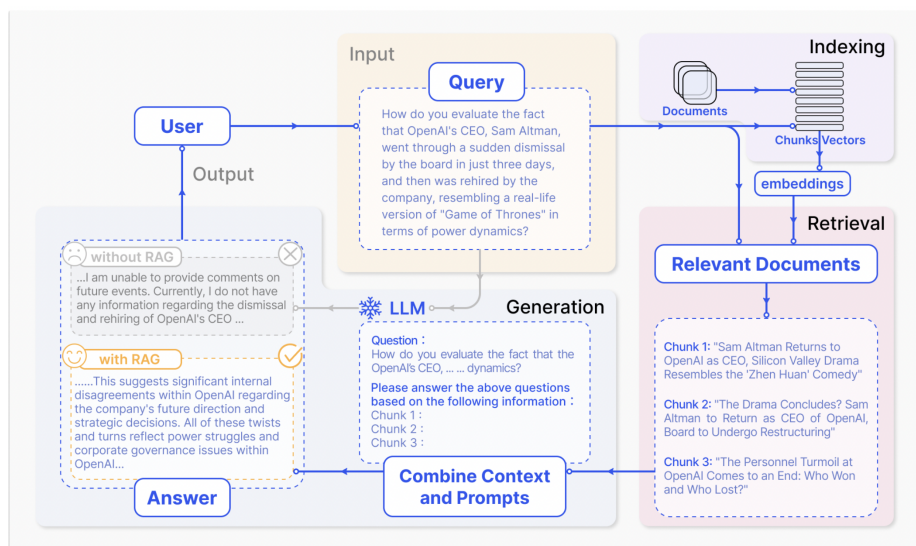


图 22.1 文档先进入索引，查询检索相关片段，再将证据与问题一起交给生成模型。（课件 lec12_ppt，第 44 页）

数学直觉 | 把知识作为本次作答的条件

RAG 不要求模型把每条新事实都写入参数。它把外部资料变成当前可读取的上下文。真正要学好的，是如何找到、理解和正确使用这些资料。

22.2 切分、召回与重排

知识库需要先划分为检索单元。片段太短，关键定义和条件可能被切开；片段太长，无关内容增多，也更容易挤占上下文。可以按标题、段落和语义边界切分，再用重叠区减少边界损失。

每个片段应保留文档编号、标题、章节、时间和位置等元数据。向量只支持相似度搜索，不能替代这些来源信息。相同一句话出现在不同版本文档里，时间和版本可能决定哪条证据有效。

一个切分失败例子

第一段写“本政策适用于 2025 年后入学的学生”，第二段写“可申请两次延期”。如果只检索到第二段，模型可能把结论误用于所有学生。切分时保留适用条件，或检索后补回父章节上下文，可以减少这种错误。

查询改写可以补全缩写、解析指代或产生多个搜索视角。对“它的报名截止时间”这类问题，应先从对话中确定“它”指什么，再检索。直接用孤立句子检索，可能得到语义相近却对象错误的材料。

混合检索结合 BM25 与稠密向量，重排器再读取查询与候选片段。难例通常不是“完全不相关”，而是主题正确、实体或时间条件错误。因此重排训练需要覆盖这些细微差别。

去重与多样性

Top-k 结果可能来自同一篇文档的相邻片段，信息高度重复。保留少量多样结果，可以提高证据覆盖。但过度追求多样性也可能降低相关性。

最大边际相关性 MMR 的一个形式是

$$\text{MMR}(d) = \lambda s(q, d) - (1 - \lambda) \max_{d' \in S} s(d, d'), \quad (22.3)$$

其中 S 是已选文档集合。第一项鼓励相关，第二项惩罚与已有结果重复。不同相似度的尺度需要兼容，否则 λ 难以解释。

22.3 证据组织、引用与回答约束

找到材料后，应按问题需要组织上下文。重复段落可以压缩，表格可保留相关行，但不能在压缩时丢掉否定、条件、单位和时间范围。摘要器本身也会犯错，因此关键事实最好保留可回查的原文位置。

一种清晰的上下文格式是给每个证据片段编号，并保留简短标题。生成时要求每个重要结论连接到能支持它的证据。引用不是装饰：编号存在，只说明模型指向了某段文字，还需要检查该段是否真正支持结论。

支持关系比主题相似更严格

证据说“2024 年试点中有 60% 的参与者选择方案 A”，不能直接支持“多数用户一直偏好 A”。后者扩大了时间、群体和稳定性范围。基于证据回答，需要控制结论的适用范围。

当资料不足或互相冲突时，系统应说明缺少哪项信息，或区分不同来源的条件。强行给出单一确定答案，会把检索系统的局限转化为看似权威的幻觉。

备注 | 外部内容与任务指令

检索文档可能包含引用的命令、广告或恶意插入的指令。它们应当作为待分析内容处理。系统需要把“用户要做的任务”和“文档里写的话”分清，避免资料改变原任务。

算法实现 | 一个可检查的 RAG 流程

```
query = resolve_question(question, conversation)
candidates = hybrid_retrieve(query, top_k=50)
ranked = rerank(query, candidates)
evidence = select_under_budget(ranked, token_budget=6000)
answer = generate_with_evidence(question, evidence)
claims = extract_checkable_claims(answer)
report = verify_support(claims, evidence)
return answer, report
```

每一步都可能失败。验证支持关系可以由规则、模型与人工抽查配合完成，不能假设另一个模型的判断必然正确。

22.4 长上下文、迭代检索与系统评价

长上下文允许放入更多原文，减少切分和召回遗漏，但会增加成本。模型也未必能同等利用所有位置的信息。把整本书塞进上下文，并不自动解决跨章节证据整合。

RAG 适合从大库中选择少量材料；长上下文适合一次读取较完整的相关材料；微调适合稳定改变行为或学习任务模式。三者可以组合。例如先检索相关文档，再把完整章节放入长上下文，并使用已微调的模型生成规范答案。

多跳问题需要迭代

问题“某奖项得主的导师毕业于哪所大学”可能需要先找得主，再找导师，最后查导师教育经历。一次查询未必能覆盖全部关系。迭代检索在获得中间实体后再发起下一步查询。

但每一步都可能传播错误。若第一步找错人物，后续检索仍可能得到逻辑连贯的答案。系统应保留每一步证据，必要时对关键实体进行交叉检查。

把端到端分数拆开

可以分别测：相关证据是否被召回，证据是否被保留进上下文，答案是否使用了证据，结论是否正确且有支持。一个简单分解是

$$P(\text{成功}) = P(E)P(\text{成功} | E) + P(\neg E)P(\text{成功} | \neg E), \quad (22.4)$$

其中 E 表示检索到了必要证据。只看最终准确率，无法知道系统是在依赖证据，还是碰巧用参数知识答对。

可以做三个对照：只给问题，给正常检索结果，给人工确认的正确证据。若正确证据显著改善结果，检索可能是瓶颈；若给正确证据仍失败，应检查阅读、推理和任务格式。

引用评价的两个方向

引用精确率衡量被引用材料是否支持关联结论；引用覆盖率衡量需要证据的重要结论中，有多少获得支持。只给一条正确引用可能精确率高，却漏掉其他关键断言。两项应同时观察。

检索延迟、上下文 token 数、生成成本和知识更新时延，也是系统指标。一个准确但昂贵到无法使用的流程，未必满足实际需求。

完整案例：一条带时间条件的回答

假设知识库有三段材料：A 写“2024 年试点允许一次延期”；B 写“自 2025 年起，新入学学生可申请两次延期”；C 写“延期申请需导师签字”。用户问“2026 年入学的学生最多能延期几次，需要什么手续？”

B 支持次数，C 支持签字要求，A 是旧规则。答案应组合 B 与 C，并保留适用对象。若只按关键词“延期”召回 A，很容易给出过时次数。

这个例子说明，相关性包含实体、时间和条件匹配。三段材料都与延期高度相关，只有部分内容适合当前问题。

证据预算怎样分配

上下文预算有限时，可以把片段选择写成约束问题。设片段成本为 token 数 c_i ，预计收益为 u_i ，选择变量 $z_i \in \{0, 1\}$ ：

$$\max_z \sum_i u_i z_i, \quad \sum_i c_i z_i \leq B. \quad (22.5)$$

这是一个简化的背包形式。真实收益还依赖片段之间的互补与重复，不能总独立相加。例如 B 给次数、C 给手续，二者的联合价值高于两个重复的次数片段。

工程上可先按相关性筛选，再去重、保留条件和补足缺失事实。重要的是不要只截取前 B 个 token，因为截断可能恰好删掉决定适用范围的句子。

RAG 结果的错误分解

设 100 个问题中，80 个检索到必要证据。生成器在这 80 个中答对 60 个，在其余 20 个中凭参数知识答对 5 个。总体准确率是 65%，有证据条件下准确率是 75%。

若把检索全部提升为正确，并假设生成器条件成功率仍为 75%，粗略上限约为 75%。这不是严格预测，因为证据质量与问题难度相关；但它说明当前系统还存在明显生成端错误，不能指望只改检索就达到满分。

引用覆盖的一个计数例子

答案有 5 个需要支持的关键断言，3 个带引用，其中 2 个引用确实支持对应断言。若按这些断言计数，引用精确率为 $2/3$ ，支持覆盖率为 $2/5$ 。

如果再给另外两条断言补上不相关引用，引用数量增加了，精确率却降到 $2/5$ ，覆盖率没有改善。评价引用应检查支持关系，而不是统计方括号出现次数。

第 23 章 评价：分数到底说明了什么

“模型 A 比模型 B 好”只有在任务、数据和协议明确后才有意义。分类要看标签，生成要看内容与表达，工具任务还要看环境是否真的达到目标。同一个模型可能在一种指标上领先，在另一种指标上落后。

本章先讲常用指标的数学含义，再讨论基准设计、统计波动和大模型裁判。评价的目的不是收集漂亮数字，而是知道系统在哪些条件下可靠，在哪些地方还会失败。

23.1 分类、结构预测与排序

二分类混淆矩阵包含 TP、FP、FN、TN。准确率与正类指标为

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}, \quad (23.1)$$

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad (23.2)$$

$$F_\beta = \frac{(1 + \beta^2)PR}{\beta^2 P + R}. \quad (23.3)$$

$\beta > 1$ 更强调召回， $\beta < 1$ 更强调精确率。分母为零时的处理应事先规定，不能按有利于结果的方式临时选择。

若 1000 个样本只有 10 个正例，全预测负例可得 99% 准确率，正类召回却为零。任务关心稀有正例时，准确率可能掩盖最重要的失败。

多类宏平均先对每类算指标，再让各类等权平均；微平均先汇总计数。加权宏平均再按类频率加权，又是不同指标。报告时必须写清楚，不能只写“F1”。

一个实体识别例子

真实实体有 5 个，系统预测 6 个，其中跨度和类型都正确的有 4 个，则 $P = 4/6$ 、 $R = 4/5$ 、 $F_1 = 8/11 \approx 0.727$ 。如果把错误跨度里的部分正确 token 也算成实体正确，指标会被错误抬高。

句法分析按成分跨度或依存弧评价，检索按排名评价。它们都把复杂输出拆成某种单位，但拆分方式决定了哪些错误更重。完全匹配率则把整份输出作为一个单位，严格但有时不够细致。

置信度与校准

若模型把一批预测都赋予约 0.8 的置信度，理想校准要求其中约 80% 正确。校准好不等于准确率高：一个永远输出基础正确率的模型也可能校准，但没有区分能力。

多类 Brier 分数为

$$\text{Brier} = \frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K (p_{ik} - \mathbf{1}[y_i = k])^2. \quad (23.4)$$

它同时反映概率误差。ECE 把样本按置信度分箱，比较每箱平均置信度与准确率，但结果依赖分箱数量和方式。评价概率时，可同时查看负对数似然、Brier 和可靠性图。

23.2 编辑距离与文本重叠指标

语音识别常使用词错误率

$$\text{WER} = \frac{S + D + I}{N}, \quad (23.5)$$

其中 S, D, I 分别为替换、删除、插入次数, N 为参考文本词数。若插入很多词, WER 可以超过 1, 不能把它误认为必在 $[0, 1]$ 的准确率。

编辑距离的动态规划为

$$d(i, j) = \min \begin{cases} d(i-1, j) + 1, \\ d(i, j-1) + 1, \\ d(i-1, j-1) + 1[r_i \neq h_j]. \end{cases} \quad (23.6)$$

初始化 $d(i, 0) = i, d(0, j) = j$ 。三种转移分别对应删除、插入和替换 / 匹配。最终距离给出最少编辑数; 若要分开统计 S, D, I , 还需回溯一条最优对齐路径。

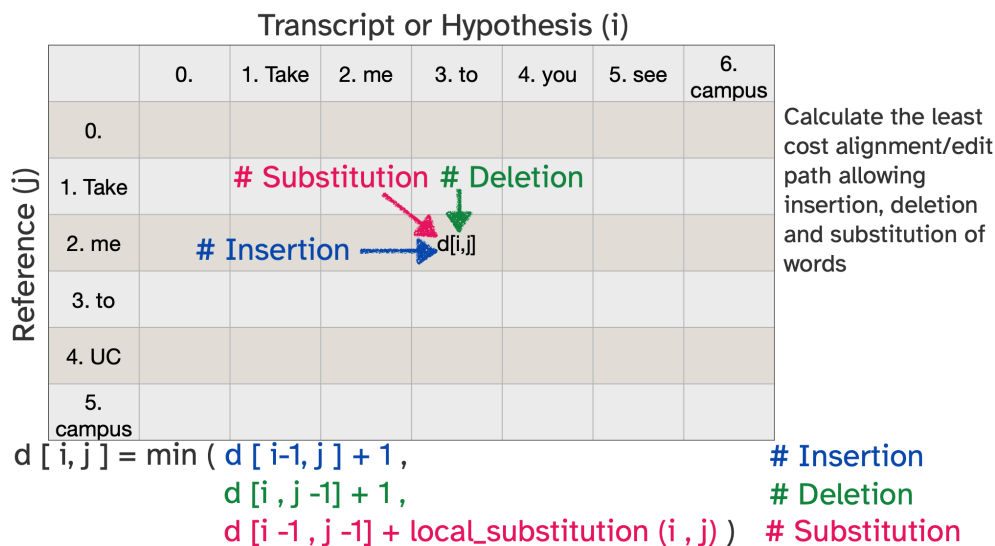


图 23.1 编辑距离沿前缀长度递推, 对齐路径决定错误类型。(课件 lec13_tasks-evaluation-benchmark, 第 9 页)

BLEU: 修正精确率与长度惩罚

BLEU 比较候选与参考的 n -gram 重叠。每个候选 n -gram 的正确计数被参考中允许的次數截断, 防止重复一个词无限获得分数。记修正精确率为 p_n , 则

$$\text{BLEU} = BP \exp \left(\sum_{n=1}^N w_n \log p_n \right), \quad (23.7)$$

$$BP = \begin{cases} 1, & c > r, \\ \exp(1 - r/c), & c \leq r, \end{cases} \quad (23.8)$$

其中 c 是候选总长度, r 是按协议选择的有效参考总长度。常用语料级 BLEU 先汇总计数, 不等于逐句 BLEU 的平均。

如果候选只有 “the the the”, 参考只有一个 the, 候选 unigram 中最多只有一次命中。若高阶 $p_n = 0$, 未经平滑的几何平均为零。句子级平滑方法有多种, 不能混用后直接比较。

数学直觉 | 重叠是代理指标

词面接近常意味着内容接近，但同义改写可能重叠很少；包含否定词的错误句子也可能与正确句子高度重叠。指标便于自动化，却只测量质量的一部分。

ROUGE 常用于摘要，关注参考内容被覆盖的程度。ROUGE-N 统计 n -gram，ROUGE-L 使用最长公共子序列。具体实现可能报告召回、精确率或 F 值，必须注明。chrF 在字符 n -gram 上计算匹配，对形态变化和部分语言的分词差异更稳健。

语义相似度指标使用上下文化 token 表示。例如 BERTScore 为候选 token 寻找参考中最相似的 token，再聚合。它能容忍一些词面改写，但仍可能忽略否定、实体替换和数字错误。高语义相似度不是事实正确证明。

23.3 可执行任务与多样本评价

代码生成可以运行测试，SQL 可以执行查询，数学题可以检查最终答案或证明。执行评价更接近实际用途，但受测试覆盖限制。一个错误程序可能通过有限测试，一个正确但资源超限的程序也可能无法使用。

$\text{pass}@k$ 衡量从 k 个候选中至少有一个正确的概率。若生成 n 个样本，其中 c 个通过测试，常用无放回估计为

$$\widehat{\text{pass}@k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}, \quad n \geq k. \quad (23.9)$$

当 $n - c < k$ 时，失败组合数为零，估计为 1。该指标描述候选覆盖，不代表系统能自动选中正确答案。

一次成功与稳定成功

同一任务反复运行可能有不同结果。除了 $\text{pass}@1$ ，还应关注随机种子波动、失败是否集中在特定输入、同一任务是否需要很多次重试。一个偶尔成功的智能体，与几乎每次都成功的系统，使用价值不同。

智能体任务应检查环境状态。例如修改代码后是否通过测试，数据库查询是否返回正确记录，文件操作是否产生预期结果。仅评价最后一段自然语言报告，会漏掉“声称完成但没有完成”的失败。

备注 | 预算属于评价协议

允许多少生成 token、采样几次、能调用哪些工具、运行多久，都会影响结果。不同预算下的分数不应被当作同一条件下的能力比较。

23.4 基准设计与大模型裁判

一个基准至少要明确目标能力、样本来源、标注规范、测试协议和评分规则。分类、知识问答、数学、代码、开放对话与工具操作，适合不同设计。

GLUE 把若干语言理解任务组织成集合；MMLU 覆盖多个学科的选择题；GSM8K 与 MATH 强调不同难度的数学题；SWE-bench 关注真实软件问题。它们测量不同的任务分布，不能用一个总排名替代全部能力描述。

题目难不等于测得准

一道题可能难在冷僻事实，也可能难在题目歧义、格式解析或错误标注。设计能力测试时，应控制不希望测量的因素。例如研究逻辑推理，可以使用明确给出的规则和虚构实体，减少常识记忆的干扰。

标注需要试运行。先让多人独立标一小批，分析分歧，再修改规范。若专家之间也难一致，单一硬标签可能无法充分表达合理答案范围。可以保留多参考、部分分值或分歧信息。

大模型作为裁判

模型裁判可以按相关性、正确性、完整性和表达清楚度比较回答。它降低评价成本，却可能偏爱更长的文本、某种风格或某个候选位置。还可能把流畅的错误解释判断为高质量。

应随机交换候选顺序，使用明确量表，并用人工标注的小集合校验。若裁判能看到系统名称，还可能受到来源暗示。对于可执行任务，应优先使用执行证据，而不是让裁判凭文本猜程序是否正确。

偏好胜率可以用 Bradley–Terry 模型拟合相对强度：

$$p(A \succ B) = \text{sigmoid}(s_A - s_B). \quad (23.10)$$

胜率依赖问题分布、比较对象与裁判，不能脱离这些条件解释为绝对能力。不同榜单如果提示集、预算或裁判不同，数字未必可直接对照。

23.5 统计波动、污染与错误分析

测试集是有限样本。若 N 个独立二元样本上的准确率为 $\hat{p}$ ，粗略标准误为

$$\text{SE}(\hat{p}) \approx \sqrt{\frac{\hat{p}(1 - \hat{p})}{N}}. \quad (23.11)$$

例如 $N = 400, \hat{p} = 0.8$ ，标准误约为 0.02。相差一个百分点可能只是抽样波动。小样本或边界概率时，应使用更合适的区间估计，不能机械依赖正态近似。

比较两个系统最好利用同一批样本上的配对结果。若它们恰好在不同题上出错，配对差异的信息比两个独立均值更丰富。bootstrap 可重采样样本并重复计算差值，但重采样单位要与数据依赖结构一致。

算法实现 | 配对 bootstrap 的思路

```
# a[i], b[i]: per-example scores on the same examples
rng = np.random.default_rng(0)
delta = []
for _ in range(2000):
    ids = rng.integers(0, len(a), size=len(a))
    delta.append(np.mean(a[ids] - b[ids]))
interval = np.quantile(delta, [0.025, 0.975])
```

对同一文档的多个句子，应考虑按文档重采样。复杂非线性指标则需在每个重采样集合上重新计算完整指标。

污染与捷径

测试内容出现在训练阶段，会让记忆能力混入泛化评价。即使没有逐字重复，题目模板、答案解释和近似改写也可能泄漏。时间切分、新构造样本和去重检查能降低风险，但未必彻底排除污染。

数据捷径则让系统利用与目标能力无关的相关性。例如某类标签总对应更长句子，模型可凭长度取巧。反事实改写、实体替换、标签平衡和跨领域测试，有助于发现这些问题。

怎样做有用的错误分析

先按输入长度、领域、语言、频率和任务类型分组，再检查错误。把“答案错了”细分为检索失败、条件遗漏、计算错误、格式错误或工具执行失败，通常更容易决定下一步改进。

还应保留少量完整案例：输入、系统输出、参考答案、所用证据和评分理由。只有聚合分数，无法说明错误机制；只有精选案例，也无法说明发生频率。两者需要配合。

完整算例：宏平均和微平均为什么不同

有两个类别，多数类 90 个样本、少数类 10 个。系统把全部样本预测成多数类。多数类精确率为 $90/100 = 0.9$ ，召回率为 1， $F_1 = 1.8/1.9 \approx 0.947$ 。少数类召回为 0，按常见零除约定其 $F_1 = 0$ 。

宏平均 F_1 约为 0.474，微平均 F_1 与准确率一样为 0.9。两个数字都计算正确，只是类别权重不同。若少数类是实际关心的对象，单报微平均会产生误导。

BLEU 的简化手算

参考为“the cat is on the mat”，候选为“the cat is on mat”。unigram 全部可匹配， $p_1 = 1$ 。候选四个 bigram 中，“on mat”未出现在参考中，其余三个命中，所以 $p_2 = 3/4$ 。

只用一、二元组并等权时，几何平均为 $\sqrt{3/4} \approx 0.866$ 。候选长度为 5，参考为 6，长度惩罚为 $e^{1-6/5} = e^{-0.2} \approx 0.819$ 。最终简化 BLEU 约为 0.709。

这个算例不是标准四元 BLEU 的分数，它只用于看清两个机制：重叠衡量局部匹配，长度惩罚防止通过过短输出来提高精确率。真实报告必须注明 n-gram 阶数、平滑与分词协议。

编辑距离的边界条件

参考为空、候选有 m 个词时，编辑距离是 m 次插入；候选为空、参考有 n 个词时，是 n 次删除。这给出 $d(0, m) = m, d(n, 0) = n$ 。

如果错误初始化成全零，算法相当于允许免费删除或插入前缀，最终会低估距离。动态规划中的边界条件不是附属细节，它定义了允许的路径和代价。

配对比较比独立区间更有信息

同样 100 道题，A 与 B 都答对 80 道。情形一是它们在完全相同题目上答对，行为差异很小；情形二是有 20 道题 A 对 B 错、另有 20 道题 B 对 A 错。平均分相同，但系统错误互补性很不一样。

如果考虑组合或路由，两种情况的价值不同。保存逐样本结果，才能研究这种互补；仅保存一个平均分就丢掉了信息。

多次挑选“最好结果”的偏差

假设不同随机种子带来几分波动，只报告最好的一个种子，会高估平均表现。对多个模型、提示或超参数反复尝试后选最高测试分，也会产生类似偏差。

应报告预先约定的汇总方式，并将模型选择放在验证集。对于昂贵实验，可以用少量重复估计波动，再将差异解释到与证据相称的程度。

平均质量与最坏情形

部署任务有时更关心严重错误的频率，而不是平均语言质量。可以同时报告任务成功率、严重错误率和拒答率。降低错误但大量拒答，也会改变系统的覆盖范围。

选择性预测可画风险随覆盖率的变化：只回答最有把握的样本时，错误率通常降低；覆盖更多样本时，风险上升。这个关系比单一阈值下的准确率更全面地说明系统可用性。

推演与自检

两个系统平均分相同，一个在常见问题上稳定正确、在罕见问题上失败；另一个各组表现均匀。哪一个更好取决于应用分布与错误代价。评价最后仍要回到任务，而不是让平均数替我们做全部判断。

第 24 章 综合实验：把模型放回任务中

单个公式通常很清楚，完整系统却包含数据处理、训练、解码和评价。一个看似来自模型的提升，可能实际来自更好的分词、更多训练数据或不同的输出长度。因此，理解课程内容还需要把这些环节放在一起。

本章用三个实验方案串起前面的知识。重点是定义可检验的问题、建立足够简单的基线，并通过错误分析决定下一步。这里不预设哪一种模型一定获胜。

24.1 实验一：从词袋分类到上下文表示

选择一个有明确标签的文本分类任务，例如短评论情感。先按文档或来源划分训练、验证和测试，再建立三个基线：多数类、TF-IDF 加逻辑回归、预训练编码器微调。

多数类基线告诉我们类别偏斜能贡献多少分数。线性模型告诉我们词汇线索是否已经足够。编码器模型则检验上下文表示能否处理词序、否定和更复杂的表达。

先固定数据口径

只用训练集学习词表与 IDF。去重应覆盖划分之间的近重复。验证集用于选择正则化、学习与停止时刻，测试集只用于最终比较。

对每个样本保留原文、标签、来源和长度。这样，后续可以检查短句与长句、常见词与罕见词、不同来源之间的差异。没有这些元数据，许多错误会被平均数淹没。

建立诊断样本

构造几组最小对照：“很好”与“不好”，“不是很好”与“不是不好”，“屏幕不错但续航很差”与“续航不错但屏幕很差”。它们用于检查模型是否利用否定、转折和评价对象。

诊断集不是正式测试集的替代。它的问题少、针对性强，用来解释机制；大测试集用来估计总体表现。二者各有用途。

概率与阈值

如果负面评论的漏检代价更高，验证集上可以选择更低的负面判定阈值。阈值变化会移动精确率与召回率的平衡，而不是凭空提高模型排序能力。

可以在验证集上做温度校准，再在测试集上检查负对数似然与 Brier 分数。准确率不变而概率质量改善，也可能有实际价值，例如帮助人工复核挑选不确定样本。

算法实现 | 分类实验的最小流程

```
train, dev, test = split_by_source(dataset)
vectorizer.fit(train.text)
X_train = vectorizer.transform(train.text)
X_dev = vectorizer.transform(dev.text)
model = choose_regularization(X_train, train.label, X_dev, dev.label)
threshold = choose_threshold(model, X_dev, dev.label)
report = evaluate_once(model, vectorizer, test, threshold)
```

所有可选择的步骤在查看测试结果前固定。若根据测试错误继续改模型，之后需要新的独立测试。

24.2 实验二：序列标注中的结构是否有用

选择命名实体识别。比较独立 token 分类器与同一编码器加 CRF。保持编码器、数据和训练预算尽量一致，才能判断结构层带来了什么。

独立分类器优化各位置标签交叉熵，CRF 优化完整序列条件概率。前者可能预测非法 BIO 转移，后者可在解码中直接排除。两者差异既来自训练目标，也来自解码约束，应分别分析。

三个有帮助的对照

第一组是独立分类、无约束 argmax 。第二组保留独立训练，但解码时加入合法转移约束。第三组使用完整 CRF 训练与解码。这样能区分“约束减少非法输出”和“联合训练改善评分”两种作用。

实体级 F1 是主要指标，也可记录非法转移率、边界错误率和类型混淆。若 CRF 主要减少非法转移，却没有明显改善实体 F1，说明结构合法只是成功的一部分。

子词与实体边界

编码器可能把一个词拆成多个子词。标签可以只监督首子词，也可以传播到所有子词，但实体评价最终仍需回到原始词或字符跨度。训练对齐、解码对齐和评测对齐必须一致。

例如一个人名被拆成三片，如果把后两片错误标成新的 B-PER，会制造三个实体。即使 token 级指标看起来不错，最终抽取结果仍然错误。

上下文范围实验

先用句子级输入，再加入相邻句或文档上下文。新增上下文可能帮助识别代词、缩写和跨句实体，但也增加计算与噪声。应按实体是否需要跨句信息分组，判断提升是否来自预期机制。

数学直觉 | 结构与表示各自承担一部分工作

编码器判断某个位置像什么标签，结构层判断这些标签能否组成合理输出。更强表示可能减少结构错误，但不会使显式约束在所有场景下都失去意义。

24.3 实验三：一个有证据的课程问答系统

以课程资料为知识库，问题包括概念解释、公式推导和跨章节比较。系统先检索课件或讲义片段，再生成带证据的回答。为了便于验证，先从答案确实存在于材料中的问题开始。

把问题分为单片段事实、跨片段组合、计算推导和材料不足四类。“HMM 有哪两个独立性假设”属于前一类；“HMM 与 CRF 为什么共享动态规划”需要组合；给定转移概率求最佳路径需要计算；资料未说明的问题则应允许回答不足。

建立三条基线

第一条只检索并返回原文；第二条让模型闭卷回答；第三条检索后生成。第一条检查检索本身，第二条测参数知识，第三条测组合系统。再加“人工选择正确证据”的上界对照，定位检索与生成的瓶颈。

对于公式问题，切分时应让公式与变量说明保持在一起。单独检索到 $\alpha_t(j)$ 的递推式，却没有其定义，容易让模型把前向概率和 Viterbi 分数混淆。

答案应该怎样评分

概念题检查关键条件是否完整；计算题检查数值与步骤；比较题检查共同点和差异是否准确；材料不足题检查是否凭空编造。引用再单独检查支持关系。

例如回答“HMM 和 CRF 都用 Viterbi，所以二者相同”应判错。共同推断算法不意味着共同概率模型。这个题目比简单询问缩写更能检查课程主线是否被理解。

最小改动与消融

先固定生成模型，只改变检索方式，比较 BM25、稠密和混合检索。再固定检索结果，比较证据组织与提示。最后才改变模型或做微调。一次同时改变所有模块，即使分数上升，也难知道原因。

观察到的问题	优先检查	有针对性的实验
答案所在片段没被找到	召回与切分	人工证据对照、扩大候选数
证据正确但答案遗漏条件	阅读与上下文组织	保留定义、调整证据顺序
推导过程数值错误	计算与验证	接入计算工具、检查中间量
引用存在但不支持结论	证据匹配	逐断言支持检查
格式经常失败	输出约束	结构化解码与解析器

表 24.1 错误类型决定改进位置。

24.4 读懂实验结果的共同方法

三个实验都需要记录随机种子、数据版本、超参数、停止规则和评价脚本。复现不仅是能重新运行命令，还要能解释为何得到这个结果。

先看总体指标，再看分组结果，最后回到具体案例。若只有少数极端样本贡献了提升，应说明这一点；若某组明显退步，也不应被总分掩盖。

从错误提出下一个问题

“更换更大模型”是一个可能方案，但不是唯一方案。词表遗漏可以改分词，检索漏召回可以改索引，标签冲突可以改规范，输出非法可以加结构约束。改动越贴近已经观察到的错误机制，实验越容易解释。

连接经典模型与大模型

朴素贝叶斯通过计数共享证据，神经模型通过参数共享证据；HMM 与 CKY 利用分解进行精确搜索，生成模型常使用近似搜索；对数线性模型、CRF、对比学习和 DPO 都包含归一化评分与相对比较。它们不是互不相干的技术清单，而是围绕表示、概率、优化和推断的不同选择。

最终应能够对一个新方法回答四个问题：它读到什么信息，给什么对象打分，训练目标奖励什么，推断过程怎样得到最终输出。再加上评价协议，就能较有条理地判断它解决了什么问题，还留下哪些限制。

综合练习：一个提升究竟来自哪里

假设新系统同时更换了分词器、训练语料、模型大小和解码策略，最终指标提高。这个结果可以证明完整系统在该协议下更好，却不能证明其中某一项是原因。

可以分阶段做对照。先固定数据与解码，只换模型；再固定模型与解码，只换数据；最后检查两项组合是否产生交互。如果资源不足以做全部组合，至少选择与核心结论直接相关的对照。

数据量与训练预算的公平性

一个模型训练 1 轮，另一个训练 10 轮，比较相同步数或相同 epoch 都可能不公平。应根据问题选择预算：固定读取 token、固定计算量、固定实际时间，或固定最终部署成本。

没有唯一的公平定义，但定义必须与研究问题一致。例如比较训练效率，应看固定计算预算下的结果；比较最大可达性能，可以允许不同预算，但要同时报告成本。

从一条失败案例提出可证伪假设

系统把“不是所有申请都被批准”理解成“所有申请都未获批准”。可以提出假设：模型混淆了否定与全称量词的作用域。

下一步不应只增加这一个句子的训练次数，而应构造一组作用域对照：并非所有、所有都不、至少一个不、没有一个。若模型只在原句上改善、对新措辞仍失败，说明它可能记住了句子，尚未学会该逻辑区别。

记录失败比只保存成功输出更有价值

训练与评测日志应保留解析失败、超时、无效工具调用和空答案。若只统计成功返回的样本，困难案例会被系统性排除，指标被抬高。

对失败样本明确给分规则，并报告失败率。若某方法通过多次重试减少失败，还应计入重试预算。一个可复现的实验，不仅要复现成功路径，也要复现失败如何被处理。

附录 A 推导专题：把几条共同思路连起来

本附录集中展开几条在不同章节反复出现的推导。它们不依赖某一个具体网络，而是帮助我们理解：隐变量怎样被学习，结构空间怎样被求和，以及梯度为什么常常表现为两个期望之差。

A.1 隐变量、下界与 EM

有观测 x 和隐变量 z ，模型似然为 $p_\theta(x) = \sum_z p_\theta(x, z)$ 。难点在于对数在求和之外：

$$\log p_\theta(x) = \log \sum_z p_\theta(x, z). \quad (\text{A.1})$$

如果直接把对数移进求和，就会改变目标。引入任意分布 $q(z)$ ，在相关支持集上改写为

$$\log p_\theta(x) = \log \mathbb{E}_{z \sim q} \left[\frac{p_\theta(x, z)}{q(z)} \right]. \quad (\text{A.2})$$

由 $\log$ 的凹性，Jensen 不等式给出下界

$$\log p_\theta(x) \geq \mathbb{E}_q[\log p_\theta(x, z)] - \mathbb{E}_q[\log q(z)] \equiv \mathcal{F}(q, \theta). \quad (\text{A.3})$$

第二项是 q 的熵。它避免把所有质量任意集中到某一个隐解释，而不考虑这种选择的不确定性。

下界与真实似然相差多少

使用 $p_\theta(z | x) = p_\theta(x, z)/p_\theta(x)$ ，可直接展开

$$D_{\text{KL}}(q \| p_\theta(z | x)) = \mathbb{E}_q[\log q(z) - \log p_\theta(x, z) + \log p_\theta(x)] \quad (\text{A.4})$$

$$= \log p_\theta(x) - \mathcal{F}(q, \theta). \quad (\text{A.5})$$

因此有精确恒等式

$$\log p_\theta(x) = \mathcal{F}(q, \theta) + D_{\text{KL}}(q \| p_\theta(z | x)). \quad (\text{A.6})$$

当 q 等于真实后验时，KL 为零，下界贴住似然。E 步就是在当前参数下取这个后验；M 步固定 q ，提高下界中的参数相关项。

记旧参数为 θ^{old} ，E 步取 $q = p_{\theta^{\text{old}}}(z | x)$ 。若 M 步找到新参数使下界不下降，则

$$\log p_{\theta^{\text{new}}}(x) \geq \mathcal{F}(q, \theta^{\text{new}}) \geq \mathcal{F}(q, \theta^{\text{old}}) = \log p_{\theta^{\text{old}}}(x). \quad (\text{A.7})$$

这解释了 EM 的单调性。它不意味着全局最优，也不意味着每轮都会显著改善。

在 HMM 中，E 步具体算什么

隐变量是整条状态序列。直接列出后验有指数多个结果，但 M 步只需要初始状态、转移和发射的期望计数。前向后向提供 $\gamma_t(j)$ 与 $\xi_t(i, j)$ ，正好足够。

$$\pi_j^{\text{new}} \propto \sum_{\text{句子}} \gamma_1(j), \quad (\text{A.8})$$

$$A_{ij}^{\text{new}} \propto \sum_{\text{句子}} \sum_t \xi_t(i, j), \quad (\text{A.9})$$

$$B_j^{\text{new}}(w) \propto \sum_{\text{句子}} \sum_t \gamma_t(j) \mathbf{1}[x_t = w]. \quad (\text{A.10})$$

每组按其条件变量归一化。隐变量本身很多，并不代表学习一定需要保存所有后验结构；关键是找到参数更新真正需要的统计量。

硬 EM 与软 EM

如果 E 步只保留最佳隐解释，就得到一种硬分配训练。它通常更便宜，但不再与标准 EM 的后验期望相同。某条解释略胜其他解释时，硬分配也会给它全部权重，从而更容易放大早期选择。

在词义聚类中，软分配允许一个上下文暂时同时属于多个义项；在文法学习中，软分配保留多棵树的贡献。是否值得保留这些不确定性，取决于计算成本和任务。

A.2 配分函数、边缘与期望梯度

考虑一类统一模型

$$p_{\theta}(y | x) = \exp(\theta^{\top} f(x, y) - A_{\theta}(x)), \quad (\text{A.11})$$

其中 $A_{\theta}(x) = \log \sum_y \exp(\theta^{\top} f(x, y))$ 。对参数求导：

$$\nabla_{\theta} A_{\theta}(x) = \mathbb{E}_{p_{\theta}}[f(x, Y)]. \quad (\text{A.12})$$

再次求导得到特征协方差。由此，分类器、CRF 和其他指数族结构模型共享同一套训练逻辑。

一个局部势的导数就是其出现概率

把某条边或某个标签的局部分数记为 s ，完整结构是否包含它由指示函数 $I_s(y)$ 表示。若总分对 s 的导数为 $I_s(y)$ ，则

$$\frac{\partial \log Z}{\partial s} = \mathbb{E}[I_s(Y)] = P(I_s(Y) = 1). \quad (\text{A.13})$$

如果同一个规则可重复出现，则导数变为期望出现次数。于是，求配分函数和求边缘并不是完全分离的两件事：后者可以通过前者的导数得到。

最大化对应另一种传播

将 $\log\text{-sum-exp}$ 换成 $\max$ ，计算的是最佳结构分数。若最优结构唯一，对局部分数的梯度是该最优结构中的指示值；存在并列最优时，函数不可微，可讨论次梯度。

求和传播保留所有候选的软权重，最大传播保留最优候选的硬选择。这解释了似然训练与某些最大间隔训练为何会产生不同的更新信号。

半环的简单理解

动态规划把小问题的结果组合成大问题。沿同一条路径组合局部因子是一种运算，在不同候选路径之间汇总是另一种运算。不必先掌握抽象代数才能使用这个观点。只要明确在一条结构内

目标	路径内组合	候选间汇总	无解值
可达性	逻辑与	逻辑或	假
总概率	乘法	加法	0
最佳概率	乘法	最大	0
最佳对数分数	加法	最大	$-\infty$
对数总概率	加法	log-sum-exp	$-\infty$

表 A.1 同一结构分解可以支持不同计算问题。

部怎样组合、不同结构之间怎样比较或累加，就能减少把 forward 与 Viterbi 混淆的机会。

A.3 对比学习中的梯度与负例分布

对一个查询 q 、正例 d^+ 和候选集合 D ，令分数为 $s_j = u^\top v_j / \tau$ ，损失为 $-\log \text{softmax}(s)_+$ 。对查询向量求导：

$$\frac{\partial \ell}{\partial u} = \frac{1}{\tau} \left(\sum_j p_j v_j - v_+ \right). \quad (\text{A.14})$$

正例向量提供吸引方向，模型加权平均的候选提供相对排斥方向。权重来自模型当前分数，因此高分负例对更新更重要。

温度同时改变分布和梯度尺度

较小 τ 让 softmax 更集中，还在导数前引入 $1/\tau$ 。不能仅说它让梯度“变大”或“变小”，因为概率 p_j 也随温度改变。容易样本可能很快饱和，困难样本则可能产生较强更新。

如果向量没有归一化，增大向量范数也会扩大点积分数，其作用与降低温度有联系。归一化约束将这种尺度自由度移除，让相似度主要由方向决定。

负例从哪里来，决定要区分什么

随机文档负例训练模型区分大主题，困难负例训练模型区分主题中的细节。若评测需要找同一实体的最新政策，负例最好包含同实体的旧政策，而不只是无关新闻。

但负例分布不能一味追求最难。极难负例可能标注有误，也可能超出当前模型可学习范围。随机、稀疏检索与模型挖掘负例可以混合，形成不同难度的监督。

批内负例的概率解释

批内目标把“正确配对是哪一个”变成分类问题。候选集合改变时，分母也改变。因此同一个正例在不同批次里的损失可能不同，并非模型突然对它学坏了。

如果目标是完整语料库上的检索，批内目标只是一个可计算的训练替代。应在独立完整候选库上评价，不能把训练批次准确率当作最终检索召回率。

A.4 策略梯度的基线、重要性采样与支持集

策略梯度恒等式要求能够对策略概率求导。离散采样本身不可微，但采样分布的对数概率可微。对固定奖励函数：

$$\nabla_{\theta} \mathbb{E}_{Y \sim \pi_{\theta}}[r(Y)] = \mathbb{E}_{Y \sim \pi_{\theta}}[r(Y) \nabla_{\theta} \log \pi_{\theta}(Y)]. \quad (\text{A.15})$$

如果奖励函数本身还显式依赖 θ ，求导时通常还需要其直接导数项。把包含策略对数比的 KL 惩罚当作奖励时，应与具体实现的梯度处理保持一致。

常数基线与价值基线

常数基线不改变期望梯度，但不同基线影响方差。对逐步决策，状态 s_t 已在动作采样前确定，所以 $b(s_t)$ 可作为基线；若基线依赖本次动作，则一般不能直接消去。

价值函数估计从当前状态继续的期望回报。优势 $A(s, a) = Q(s, a) - V(s)$ 衡量某个动作比当前平均水平好多少。它强调相对收益，能减少不同问题难度造成的整体奖励差异。

从旧策略样本估计新策略期望

若样本来自 q ，而目标是 p ，在支持集覆盖满足时，

$$\mathbb{E}_{Y \sim p}[f(Y)] = \mathbb{E}_{Y \sim q} \left[\frac{p(Y)}{q(Y)} f(Y) \right]. \quad (\text{A.16})$$

比率称为重要性权重。若 q 在某处为零而 p 为正，就无法仅靠 q 的样本估计该区域贡献。若比率很大，方差也会很大。

PPO 的比率利用旧策略采样，但通过裁剪构造更稳定的代理目标。裁剪引入了目标变化，不能再声称它是未经修改的精确重要性估计。算法的实用性与估计的无偏性，需要分别讨论。

为什么在线采样可能有帮助

离线偏好数据只包含已有回答。新策略若走到未覆盖区域，监督可能不足。在线采样能针对当前策略的实际错误收集反馈，但要付出生成和评价成本。

这与主动学习有相似之处：不是随机增加任意样本，而是选择当前模型最需要的监督。不过，反馈噪声与奖励投机仍然存在，在线并不自动代表正确。

A.5 数值稳定：数学相等不代表浮点计算相同

计算 $\log(1 + e^x)$ 时，较大 x 会使指数溢出。稳定形式为

$$\text{softplus}(x) = \max(x, 0) + \log(1 + e^{-|x|}). \quad (\text{A.17})$$

于是 $\log \text{sigmoid}(x) = -\text{softplus}(-x)$ 。负采样、奖励模型和 DPO 都应使用这类稳定原语。

全为负无穷的一行

若掩码把某个注意力行的所有键都设为 $-\infty$, 普通 softmax 中减去最大值会产生 $-\infty - (-\infty)$, 可能得到 NaN。正确处理应从任务约束出发：保证至少一个合法键，或对空行使用明确的零输出规则。

同样，Viterbi 的最终状态全为 $-\infty$ 表示没有合法路径。不能继续回溯并把默认索引当成答案。数值异常常常暴露结构或掩码错误，而不只是精度不够。

求和顺序与混合精度

浮点加法不严格满足结合律。大量很小值与很大值相加时，小值可能被舍去。归一化、方差和长序列损失累计常需要较高精度，即使矩阵乘法使用较低精度。

验证算法时，应先用小规模双精度实现建立参考，再比较低精度或优化版本。比较应使用与问题尺度相称的容差，而不是要求所有浮点位完全相同。

附录 B 算法实验：用小输入检查大公式

实现算法时，最有效的第一步往往不是训练大模型，而是构造一个足够小、可以枚举或手算的问题。这样，代码输出与数学定义能够逐项比较。

下面的实验围绕三条原则展开：用枚举核对动态规划，用等价实现核对优化，用有限差分核对梯度。输入规模很小，普通 CPU 即可完成。

B.1 概率模型：先检查归一化与边界

任何声称输出概率的函数，都应检查非负性和总和。更重要的是检查边界情形：空输入、未见历史、零计数、所有路径非法。正常样本通过，不代表边界处理正确。

实验一：朴素贝叶斯的完整预测

使用正文中的两类词频矩阵。输出后验应约为 (0.8974, 0.1026)。

```
counts = np.array([[6., 0., 4.], [1., 8., 1.]])
x = np.array([1., 0., 1.])
prob = counts + 1
prob /= prob.sum(axis=1, keepdims=True)
score = np.log([0.5, 0.5]) + np.log(prob) @ x
posterior = np.exp(score - score.max())
posterior /= posterior.sum()
```

将输入改为 [1, 1, 0]，金融义概率应降到 0.28。这检查了特征顺序是否与词表一致。如果结果不变，可能是代码只使用了先验，或输入向量没有真正参与计算。

再把两个类别的计数行交换，并同时交换先验。后验应按同样方式交换。这个对称性检查不依赖某一个固定数值，有助于发现标签索引错误。

实验二：平滑分布的行和

对一个历史枚举完整输出词表，计算平滑概率之和。加性平滑应满足

$$\sum_w \frac{C(h, w) + \alpha}{C(h) + \alpha V} = 1. \quad (\text{B.1})$$

Kneser–Ney 则应核对折扣后剩余质量加回退质量也为 1。若只枚举训练中见过的后继词，会漏掉回退分配给其他词的质量，不能据此判断模型未归一化。

一个合理的测试应包含未见历史。此时回退分布仍应归一化，不能产生除零。还应检查 BOS 是否被错误加入可输出词表。

B.2 动态规划：用穷举作参考答案

小规模穷举很慢，但定义清楚。它非常适合检查动态规划是否正确合并候选、是否漏乘一个因子，以及回溯是否恢复了正确结构。

实验三：枚举 HMM 的全部路径

状态数 K 、长度 T 很小时，可列出 K^T 条路径，逐条计算联合概率。然后比较总和与前向算法，比较最大值与 Viterbi。

```
from itertools import product
weights = {}
for path in product(range(K), repeat=T):
    p = start[path[0]] * emission[0, path[0]]
    for t in range(1, T):
        p *= transition[path[t-1], path[t]]
        p *= emission[t, path[t]]
    weights[path] = p
brute_total = sum(weights.values())
brute_best = max(weights, key=weights.get)
```

正文的两状态例子应得到总概率 0.091，最佳路径 N,V 的概率 0.054。若前向与枚举一致、Viterbi 不一致，优先检查最大化方向和回溯表；若二者都错，优先检查模型因子和初始化。

还可从枚举结果直接计算每个标签边缘，再与前向后向比较。这样既验证归一化常数，也验证局部后验。

实验四：CKY 的最大值与求和

使用望远镜附着文法。编写一个通用填表过程，分别采用 $\max$ 与 $\sum$ 汇总候选。最终应得到最佳树概率 0.00192，inside 总概率 0.00336。

将 $VP \rightarrow VP PP$ 的概率降低时，必须同步调整同左侧的其他规则概率，才能仍是 PCFG。若只改一条规则而破坏归一化，仍可作为任意结构评分模型，但不能继续解释为同一生成概率文法。

一个额外检查是删除 $NP \rightarrow NP PP$ 规则，并重新约定剩余 NP 规则的概率。此时名词附着应不再可解析。若仍输出该结构，可能是旧 chart 或回溯状态没有清空。

实验五：编辑距离的最小边界集

至少测试空串对空串、空串对非空串、完全相同序列、一次插入、一次删除和一次替换。然后用含多个最优对齐的例子检查距离值稳定。

若要报告 S、D、I，多个最优对齐可能给出不同错误分解，而总距离相同。评价协议需要固定回溯的并列选择规则，尤其是在分析错误类型时。

B.3 神经计算：检查因果性与等价性

神经层通常由成熟库提供，但张量维度、掩码、缓存和标签移位仍由使用者组织。小张量实验可以在不训练的情况下发现很多错误。

实验六：修改未来，过去输出应不变

随机生成长度为 4 的输入，计算因果自注意力输出。再只修改最后一个 token，重新计算。前三个位置输出应在数值容差内不变。

```
first = causal_attention(X)
changed = X.copy()
changed[-1] = [100.0, -30.0]
second = causal_attention(changed)
np.testing.assert_allclose(first[:-1], second[:-1])
```

如果检查失败，可能是掩码方向反了、softmax 轴错了，或某个归一化 / 池化步骤跨位置使用了未来信息。这个检查比仅观察训练损失更直接。

实验七：缓存与全量前缀应一致

对一个固定的单层注意力，先一次性计算整个因果输出，再逐位置追加 K、V，比较每一步新输出。两个实现应一致。

对于多层 Transformer，每层都要保存自己的缓存，且位置编码保持一致。比较时关闭 dropout，使用相同数据类型和模型权重。训练模式下的随机 dropout 会让两次运行不同，不能误判成缓存公式错误。

实验八：LoRA 合并不改变线性输出

直接计算两条分支与先合并矩阵再计算：

```
unmerged = W0 @ x + scale * (B @ (A @ x))
merged = (W0 + scale * B @ A) @ x
np.testing.assert_allclose(unmerged, merged)
```

在相同精度的纯线性运算中，两者应接近。若训练分支包含 dropout，必须切到推断行为后再比较。低精度重数量化可能引入额外误差，也要与纯浮点合并区别开来。

实验九：softmax 梯度的有限差分

使用 logits (2, 1, 0)、真实标签为第三类。解析梯度约为 (0.6652, 0.2447, -0.9100)。对每个 logit 做中心差分，与解析梯度比较。

梯度之和应接近零。这个性质来自 logits 的共同平移不改变损失。如果梯度和明显不为零，可能是目标分布未归一化，或实现遗漏了分母的导数。

B.4 偏好与采样：检查目标是否真的一致

偏好学习与推测采样都使用概率比，因此很容易把正负方向、参考模型或归一化弄反。小分布上的精确计算尤其重要。

实验十：DPO 在参考点的损失

策略与参考的两个回答对数概率完全相同时，DPO 间隔为零，损失应为 $\log 2$ 。把偏好回答相对参考的概率提高、其他量固定，损失应下降。

```
margin = beta * ((policy_pos-ref_pos) - (policy_neg-ref_neg))
loss = np.logaddexp(0.0, -margin)
```

把正负回答交换后，间隔变号。若损失完全不变，代码可能丢掉了标签方向，或错误地对差值取了绝对值。

实验十一：推测采样的概率质量

设 $p = (0.6, 0.4)$ 、 $q = (0.8, 0.2)$ 。接受部分为 $\min(p, q) = (0.6, 0.2)$ ，拒绝概率为 0.2，残差分布为 $(0, 1)$ 。两部分相加恢复 p 。

这个检查无需运行随机采样，就能确认单步概率守恒。随后可大量采样，检查经验频率是否接近 p ，但随机试验只提供统计证据，不能代替前面的代数核对。

实验十二：指标实现的自洽性

对完全正确的分类与结构输出，指标应达到协议中的满分；对完全错误且无重叠的输出，应得到相应低分。检索指标还应检查结果去重，避免同一相关文档重复出现而重复计分。

$\text{pass}@k$ 在 $c = 0$ 时为零，在 $c = n$ 时为 1；随 k 增大不应下降。若计算大组合数出现溢出，可用乘积形式计算失败概率：

$$\frac{\binom{n-c}{k}}{\binom{n}{k}} = \prod_{j=0}^{k-1} \frac{n-c-j}{n-j}, \quad (\text{B.2})$$

并单独处理 $n - c < k$ 的情况。

从小实验走向真实训练

小实验通过后，再依次增加词表、长度、批量和模型层数。每次只扩一个维度，更容易定位新问题。最后才加入混合精度、分布式和优化算子。

这种顺序不是要求所有研究都重复实现底层工具，而是为复杂系统建立一个可相信的参照。模型失败时，我们至少能区分数学实现错误与统计学习本身的困难。

附录 C 综合习题与解答

这些题目用于检查能否把公式迁移到稍有变化的问题中。先独立作答，再阅读解答。重点不在记住结果，而在说明所用假设、计算步骤和结论的适用范围。

C.1 概率与分类

题一：两个模型谁更好

四个样本的真实标签都为正类。模型 A 给真实标签的概率为 $(0.9, 0.9, 0.9, 0.01)$ ，模型 B 全部给 0.6。使用 0.5 阈值时，分别计算准确率与平均负对数似然，并解释为何两种指标可能不同。

A 的准确率为 $3/4$ ，B 为 1。平均负对数似然分别为

$$L_A = -\frac{1}{4}(3 \log 0.9 + \log 0.01) \approx 1.230, \quad (\text{C.1})$$

$$L_B = -\log 0.6 \approx 0.511. \quad (\text{C.2})$$

A 在一个样本上自信地犯错，代价很大。若把 A 的最后一个概率改为 0.49，准确率仍为 $3/4$ ，但损失会显著下降。这说明分类正确性和概率质量测量不同方面。

题二：复制特征会不会增加信息

二类模型的先验相同。某线索在正类下概率为 0.7，负类下为 0.3。看到一次线索后，朴素贝叶斯给正类多大概率？若同一线索被复制三次并错误地视为独立，结果是什么？

一次线索给出后验 0.7。三次独立计入后的优势为 $(7/3)^3$ ，后验为

$$\frac{0.7^3}{0.7^3 + 0.3^3} = \frac{343}{370} \approx 0.927. \quad (\text{C.3})$$

真实信息没有增加，增加的是模型因错误假设产生的确信。若三条线索是不同来源且在给定标签后确实独立，乘法才具有相应统计依据。

题三：拒答阈值与校准

一个三分类系统对某样本输出 $(0.55, 0.35, 0.10)$ 。误分类代价为 1，人工复核代价为 0.2。在概率可信的前提下应直接预测还是复核？

直接选第一类的期望错误代价为 $1 - 0.55 = 0.45$ ，大于复核代价，所以应复核。一般阈值为 $p_{\max} \geq 0.8$ 。若模型未校准，0.55 不能直接代表真实正确率，仍需在代表性验证集上检查。

C.2 表示与语言模型

题四：一组词向量的共同旋转

将所有词向量同时乘同一个正交矩阵 R。点积、余弦和欧氏距离是否改变？如果只旋转查询而不旋转文档，又会怎样？

因为 $R^T R = I$ ，对列向量有 $(Ru)^T(Rv) = u^T v$ ，长度与距离也保持。因此共同旋转不改变这些几何关系。只旋转一侧时，点积变成 $u^T R^T v$ ，通常改变检索结果。

这解释了为什么不同训练运行得到的坐标不宜逐维比较，也解释了查询编码器与文档编码器必须处在兼容空间中。

题五：平滑是否只是给零计数加概率

词表有 4 个词，某历史的计数为 (4, 2, 0, 0)。计算 MLE 与加一平滑分布。说明已见事件发生了什么变化。

MLE 为 (2/3, 1/3, 0, 0)，加一平滑为 (5, 3, 1, 1)/10。前两项由约 (0.667, 0.333) 变成 (0.5, 0.3)。新增质量必须来自已有质量的重新分配，否则不能保持归一化。

若改为 $\alpha = 0.1$ ，分布为 (4.1, 2.1, 0.1, 0.1)/6.4。平滑更弱，保留更多经验频率差异，但对未见事件的保护也更小。

题六：困惑度的单位

同一批原始文本，用两个分词器得到 1000 和 1500 个 token。两个模型的总负对数似然分别为 2000 和 2400。哪个 token 级 PPL 更低？能否直接说该模型更好？

两个 PPL 为 $e^{2000/1000} = e^2$ 与 $e^{2400/1500} = e^{1.6}$ ，第二个更低。但总编码代价反而更高。若编码、文本与概率定义可比较，应换算到相同的字符或字节单位；不能仅用不同分词器的 token PPL 排名。

题七：负采样数翻倍

在理想独立点积近似下，噪声分布不变，负采样数从 k 变为 $2k$ 。最佳点积会怎样变化？是否说明词语关系变弱？

由 $s^* = \log[p(c | w)/(kq(c))]$ ，新值为 $s^* - \log 2$ 。变化来自数据与噪声的类先验改变，而不是语料共现关系改变。比较不同训练设置的原始点积时，应考虑这种尺度与偏移变化。

C.3 结构预测与语义

题八：改变 HMM 的依赖结构

若观测发射依赖当前标签和前一标签，即 $p(x_t | y_{t-1}, y_t)$ ，但标签仍为一阶转移，Viterbi 是否必须把两个标签都放进当前状态？

不一定。递推到当前标签 j 时，可以在枚举前驱 i 的过程中加入发射分数：

$$\delta_t(j) = \max_i [\delta_{t-1}(i) + \log A_{ij} + \log B_{ij}(x_t)]. \quad (\text{C.4})$$

未来一步仍只需要知道当前 j ，所以状态可保持单标签。与此不同，若下一标签分布依赖前两个标签，未来就需要更长历史，必须扩充状态。判断依据是未来计算需要什么，而不是局部公式里出现几个变量。

题九：局部分数共同平移

在一个固定长度 CRF 中，给位置 t 的所有标签发射分数都加 5。最佳序列、条件概率和负对数似然是否改变？

每条合法路径都增加 5，因此最佳序列不变。配分函数乘 e^5 ，真实路径分子也乘 e^5 ，条件概率不变。负对数似然中的两项抵消，也不变。

如果合法输出长度不同，且每个输出位置都加常数，则不同长度获得不同总增量，结论不再自动成立。共同平移不变性必须说明哪些候选受到相同增量。

题十：树与路径的求和

为什么 HMM 前向算法和 PCFG inside 都能对指数多个结构求和，却不显式枚举它们？给出二者各自共享的子问题。

HMM 共享的是“已读到位置 t 、当前标签为 j ”的前缀总概率。未来只通过 j 与过去相连。PCFG 共享的是“某非终结符覆盖某个跨度”的子树总概率。上层只通过根类别和跨度使用该子树。

能够合并的前提是模型因子只依赖保留在状态中的信息。若上层还依赖子树内部某个未记录属性，就不能继续使用原来的状态而仍声称精确。

题十一：空集合与全称命题

如果论域中没有学生，“所有学生都读过这本书”在标准一阶逻辑中是否为真？自然语言使用中可能出现什么差异？

公式 $\forall x[\text{Student}(x) \Rightarrow \text{Read}(x, b)]$ 在没有学生时为真，因为没有反例。这称为空真。自然语言句子可能还带有“存在学生”的语用预设，逻辑式若要表达这种要求，需要另加存在条件。

这个例子提醒我们，真值条件、预设与会话含义不是完全相同的层面。形式化表示必须明确希望保留哪部分语言意义。

题十二：正确答案与正确程序

一个文本到 SQL 系统在测试数据库上返回正确结果，为什么仍可能生成错误语义？设计一个可以揭示错误的额外测试。

程序可能碰巧利用当前数据分布。例如问题要求北京学生，程序直接选择分数最高的学生，而当前最高分恰好来自北京。加入一名非北京的更高分学生，正确查询结果应保持相应城市条件，错误程序却改变。

这种反事实数据库测试检查程序是否表达条件，而不只是当前输出是否匹配。

C.4 Transformer 与训练

题十三：没有位置编码就完全不知道顺序吗

一个使用因果掩码、但没有显式位置编码的注意力网络，与完全无掩码、无位置的注意力网络相同吗？

不相同。因果掩码让不同位置可见的前缀集合不同，已经引入了方向和边界结构。因此“无位置编码就是完全置换等变”只适用于没有其他破坏排列对称性的条件。

不过，因果可见范围不等于提供了丰富、稳定的相对距离表示。是否能够学习某种位置任务，仍要看架构和训练数据。

题十四：注意力矩阵沿哪个方向归一化

查询长度为 3、键长度为 5，注意力分数矩阵为 3×5 。如果错误地沿查询维度归一化，会得到什么含义？

正确做法让每个查询对 5 个键分配总和为 1 的权重。沿查询维度归一化则让每个键在 3 个查询之间分配权重，输出不再是原定义下每个查询的加权平均。

矩阵乘法的形状可能仍合法，所以仅靠程序不报错无法发现。应检查每一行权重和是否为 1，并用一个手算查询验证。

题十五：LoRA 的两组零矩阵

为何 A、B 都初始化为零会让低秩分支无法学习，而仅令 B 为零通常可以？

因为 $\partial \ell / \partial B = g(Ax)^\top$, $\partial \ell / \partial A = B^\top g x^\top$ 。两者都为零时，两组梯度都为零。若 A 非零、B 为零，B 仍可获得梯度，更新后 A 也能获得信号。

这里假设没有另外直接作用于 A、B 的特殊目标。讨论初始化时应从实际梯度看原因，而不是只说“随机初始化更好”。

题十六：训练 FLOPs 相同就等价吗

两种训练方案的 ND 相同，一个模型更大、数据更少，另一个模型更小、数据更多。除了验证损失，还应比较什么？

应比较显存、实际吞吐、通信、训练稳定性和推断成本。相同粗估 FLOPs 不保证设备利用率相同。若部署调用很多次，小模型即使训练更久，也可能具有更低总成本。

还应检查数据是否新鲜。更多 token 若主要来自重复语料，其统计收益与更多独特文本不同。

C.5 偏好、推理与检索

题十七：奖励平移与策略平移

在 KL 正则奖励目标中，给固定提示的所有回答奖励加常数 c ，最优策略是否改变？目标最优值是否改变？

策略不变，因为 $e^{(r+c)/\beta} = e^{c/\beta} e^{r/\beta}$ ，共同因子被归一化消去。目标值整体增加 c ，因为任何策略的期望奖励都增加 c ，而 KL 不变。

这说明“策略相同”不意味着所有报告的奖励数值相同。比较奖励曲线前，应确认分数零点一致。

题十八：DPO 正例概率一定上升吗

DPO 是否保证每个偏好回答的绝对概率在每步更新后都提高？请区分目标推动方向与参数化模型的实际变化。

目标推动正例相对于负例、并相对于参考的对数概率差增加。共享参数更新还影响其他样本和词表归一化，正例绝对概率不一定单调上升。两者都下降、负例下降更多，也可能增加相对间隔。

因此诊断 DPO 时可以同时观察正负回答对数概率、间隔、长度和独立偏好胜率，而不只看训练损失。

题十九：投票为什么可能越投越错

如果一个错误答案在模型分布中占 0.6，正确答案占 0.4，增加独立采样次数再多数投票会怎样？

在二元情形下，多数票会越来越稳定地选择错误答案。投票只能放大已有的多数倾向，不能自动识别真理。独立验证器、外部证据或更好的候选分布，才可能改变这一趋势。

若错误答案分散在很多不同结果中，而正确答案集中，答案级投票仍可能有效。这也说明“单次正确率”并不足以完全决定多类答案投票效果。

题二十：RAG 的正确引用为何不够

答案引用了一个真实存在的段落，且该段与问题主题相关，是否足以认定引用有效？

不够。引用需要支持具体断言，并保持实体、时间、数量和条件范围一致。主题相关只是候选证据，逻辑支持才是最终要求。

可以把答案拆成原子断言，逐条检查引用。若一句话同时包含两个事实，一个引用只支持其中一个，应补充证据或拆开表达，不能让局部支持覆盖整个句子。

C.6 评价与实验解释

题二十一：pass@10 与可用代码率

某模型 pass@10 很高，用户每次只收到一个由模型自选的程序。为何体验仍可能很差？

pass@10 只说明十个候选中较常存在正确程序，不说明选择器能识别它。若提交选择随机或评分器偏好错误程序，实际成功率仍低。应分别评价候选覆盖、选择准确性和最终提交成功率。

还要计入十次生成和运行测试的成本。高 pass@10 可能来自大量推断预算，不能与单次生成的成本直接混同。

题二十二：一个百分点是否显著

在 100 道题上，A 答对 81 道，B 答对 80 道。是否足以断言 A 更好？需要哪些额外信息？

不足以。需要逐题配对结果、任务抽样方式、随机生成波动和预先确定的检验协议。一个样本的差异很容易受抽样影响。若 A、B 只在一题上不同，证据尤其有限。

统计显著也不等于实际重要。对高代价任务，一个百分点可能很重要；对噪声较大的开放评价，则可能小于评分器波动。

题二十三：正确标签也可能含糊

如果同一句话存在两种合理解释，而数据集只提供一个标签，模型预测另一种合理解释会怎样影响评价？应如何改进数据？

硬标签指标会把它记为错误，即使语义上合理。可以增加上下文、细化标注规范、允许多标签或多参考，并记录标注分歧。目标不是把所有分歧都消除，而是让评分规则与任务允许的答案一致。

题二十四：如何判断一个模块是否有贡献

一个 RAG 系统加入重排后准确率上升。除总体分数外，怎样验证提升确实来自更好的证据排序？

固定召回候选和生成设置，比较重排前后的必要证据位置、证据覆盖率以及最终答案支持情况。再使用人工正确证据作对照，判断剩余错误来自哪里。

若重排同时改变了上下文长度、片段数量和提示格式，就存在混杂因素。消融实验应尽量只改变需要研究的模块，或明确报告不可避免的联合变化。

参考文献

- [1] Yansong Feng. 自然语言处理基础 (Foundations of Natural Language Processing) . 2026 春课程课件.
- [2] Dan Jurafsky and James H. Martin. Speech and Language Processing. <https://web.stanford.edu/~jurafsky/slp3/>.
- [3] Yoshua Bengio et al. A Neural Probabilistic Language Model. JMLR, 2003.
- [4] Tomas Mikolov et al. Distributed Representations of Words and Phrases and their Compositionality. NeurIPS, 2013.
- [5] Jeffrey Pennington, Richard Socher, Christopher D. Manning. GloVe: Global Vectors for Word Representation. EMNLP, 2014.
- [6] John Lafferty, Andrew McCallum, Fernando Pereira. Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data. ICML, 2001.
- [7] Ashish Vaswani et al. Attention Is All You Need. 2017. <https://arxiv.org/abs/1706.03762>.
- [8] Jacob Devlin et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. 2018. <https://arxiv.org/abs/1810.04805>.
- [9] Colin Raffel et al. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. JMLR, 2020. <https://jmlr.org/papers/v21/20-074.html>.
- [10] Vladimir Karpukhin et al. Dense Passage Retrieval for Open-Domain Question Answering. EMNLP, 2020. <https://aclanthology.org/2020.emnlp-main.550/>.
- [11] Jianlin Su et al. RoFormer: Enhanced Transformer with Rotary Position Embedding. 2021. <https://arxiv.org/abs/2104.09864>.
- [12] Tri Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. 2022. <https://arxiv.org/abs/2205.14135>.
- [13] Jordan Hoffmann et al. Training Compute-Optimal Large Language Models. 2022. <https://arxiv.org/abs/2203.15556>.
- [14] Edward J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. 2021. <https://arxiv.org/abs/2106.09685>.
- [15] Tim Dettmers et al. QLoRA: Efficient Finetuning of Quantized LLMs. 2023. <https://arxiv.org/abs/2305.14314>.

-
- [16] Long Ouyang et al. Training language models to follow instructions with human feedback. 2022. <https://arxiv.org/abs/2203.02155>.
 - [17] Rafael Rafailov et al. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. 2023. <https://arxiv.org/abs/2305.18290>.
 - [18] Zhihong Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. 2024. <https://arxiv.org/abs/2402.03300>.
 - [19] Xuezhi Wang et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. 2022. <https://arxiv.org/abs/2203.11171>.
 - [20] Shunyu Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. 2022. <https://arxiv.org/abs/2210.03629>.
 - [21] Patrick Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. 2020. <https://arxiv.org/abs/2005.11401>.
 - [22] Charlie Chen et al. Accelerating Large Language Model Decoding with Speculative Sampling. 2023. <https://arxiv.org/abs/2302.01318>.
 - [23] Yaniv Leviathan, Matan Kalman, Yossi Matias. Fast Inference from Transformers via Speculative Decoding. 2022. <https://arxiv.org/abs/2211.17192>.